

RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



**CS23333 DATABASE MANAGEMENT
SYSTEMS LAB**

Laboratory Record Note Book

Name :...S.THANIKAIVEL.....

Year / Branch / Section : ...II / CSE.....

University Register No. : ...2116240701562.....

College Roll No. :

...240701562.....

Semester : ...III.....

Academic Year : ...2025 - 2026.....



**RAJALAKSHMI
ENGINEERING COLLEGE**

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME

ACADEMIC YEAR SEMESTER BRANCH.....

UNIVERSITY REGISTER No.

Certified that this is the bonafide record of work done by the above student in the

..... Laboratory during the year 20 - 20

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on

Internal Examiner

External Examiner

INDEX

Name : _____ Branch : _____ Sec : _____ Roll No : _____

[illegible]

Definition of a Relational Database

A relational database is a collection of relations or two-dimensional tables.

Terminologies Used in a Relational Database

1. A single **ROW** or table representing all data required for a particular employee. Each row should be identified by a primary key which allows no duplicate rows.
2. A **COLUMN** or attribute containing the employee number which identifies a unique employee. Here Employee number is designated as a primary key ,must contain a value and must be unique.
3. A column may contain foreign key. Here Dept_ID is a foreign key in employee table and it is a primary key in Department table.
4. A Field can be found at the intersection of a row and column. There can be only one value in it. Also it may have no value. This is called a null value.

EMP ID	FIRST NAME	LAST NAME	EMAIL
100	King	Steven	Sking
101	John	Smith	Jsmith
102	Neena	Bai	Neenba
103	Eex	De Haan	Ldehaan

Relational Database Properties

A relational database :

- Can be accessed and modified by executing structured query language (SQL) statements.

- Contains a collection of tables with no physical pointers.
- Uses a set of operators

Relational Database Management Systems

RDBMS refers to a relational database plus supporting software for managing users and processing SQL queries, performing backups/restores and associated tasks.

(Relational Database Management System) Software for storing data using SQL (structured query language). A relational database uses SQL to store data in a series of tables that not only record existing relationships between data items, but which also permit the data to be joined in new relationships. SQL (pronounced 'sequel') is based on a system of algebra developed by E F Codd, an IBM scientist who first defined the relational model in 1970. Relational databases are optimized for storing transactional data, and the majority of modern business software applications therefore use an RDBMS as their data store. The leading RDBMS vendors are Oracle, IBM and Microsoft.

The first commercial RDBMS was the Multics Relational Data Store, first sold in 1978.

INGRES, Oracle, Sybase, Inc., Microsoft Access, and Microsoft SQL Server are well-known database products and companies. Others include PostgreSQL, SQL/DS, and RDB.

A relational database management system (RDBMS) is a program that lets you create, update, and administer a relational database. Most commercial RDBMS's use the Structured Query Language (SQL) to access the database, although SQL was invented after the development of the relational model and is not necessary for its use.

The leading RDBMS products are Oracle, IBM's DB2 and Microsoft's SQL Server. Despite repeated challenges by competing technologies, as well as the claim by some experts that no current RDBMS has fully implemented relational principles, the majority of new corporate databases are still being created and managed with an RDBMS.

SQL Statements

1. Data Retrieval(DR)
2. Data Manipulation Language(DML)
3. Data Definition Language(DDL)
4. Data Control Language(DCL)
5. Transaction Control Language(TCL)

TYPE	STATEMENT	DESCRIPTION
DR	SELECT	Retrieves the data from the database
DML	1.INSERT 2.UPDATE 3.DELETE 4.MERGE	Enter new rows, changes existing rows, removes unwanted rows from tables in the database respectively.
DDL	1.CREATE 2.ALTER 3.DROP 4.RENAME	Sets up, changes and removes data structures from tables.

	5.TRUNCATE	
TCL	1.COMMIT 2.ROLLBACK 3.SAVEPOINT	Manages the changes made by DML statements. Changes to the data can be grouped together into logical transactions.
DCL	1.GRANT 2.RREVOKE	Gives or removes access rights to both the oracle database and the structures within it.

DATA TYPES

1. Character Data types:

- Char – fixed length character string that can varies between 1-2000 bytes
- Varchar / Varchar2 – variable length character string, size ranges from 1-4000 bytes.it saves the disk space(only length of the entered value will be assigned as the size of column)
- Long - variable length character string, maximum size is 2 GB

2. Number Data types : Can store +ve,-ve,zero,fixed point, floating point with 38 precision.

- Number – {p=38,s=0}
- Number(p) - fixed point
- Number(p,s) –floating point (p=1 to 38,s= -84 to 127)

3. Date Time Data type: used to store date and time in the table.

- DB uses its own format of storing in fixed length of 7 bytes for century, date, month, year, hour, minutes, and seconds.
- Default data type is “dd-mon-yy”
- New Date time data types have been introduced. They are
TIMESTAMP-Date with fractional seconds
INTERVAL YEAR TO MONTH-stored as an interval of years and months

INTERVAL DAY TO SECOND-stored as o interval of days to hour’s minutes and seconds

4. Raw Data type: used to store byte oriented data like binary data and byte string.

5. Other :

- CLOB – stores character object with single byte character.
- BLOB – stores large binary objects such as graphics, video, sounds.
- BFILE – stores file pointers to the LOB’s.

EXERCISE-1

Creating and Managing Tables

OBJECTIVE

After the completion of this exercise, students should be able to do the following:

- Create tables
- Describing the data types that can be used when specifying column definition
- Alter table definitions
- Drop, rename, and truncate tables

NAMING RULES

Table names and column names:

- Must begin with a letter
- Must be 1-30 characters long
- Must contain only A-Z, a-z, 0-9, _, \$, and #
- Must not duplicate the name of another object owned by the same user
- Must not be an oracle server reserve words
- 2 different tables should not have same name.
- Should specify a unique column name.
- Should specify proper data type along with width
- Can include “not null” condition when needed. By default it is ‘null’.

The CREATE TABLE Statement

Table: Basic unit of storage; composed of rows and columns

Syntax: 1 Create table table_name (column_name1 data_type (size) column_name2 data_type (size)...);

Syntax: 2 Create table table_name (column_name1 data_type (size) constraints, column_name2 data_type constraints ...);

Example:

Create table employees (employee_id number(6), first_name varchar2(20), ..job_id varchar2(10),
CONSTRAINT emp_emp_id_pk PRIMARY KEY (employee_id));

Tables Used in this course

Creating a table by using a Sub query

SYNTAX

// CREATE TABLE table_name(column_name type(size)...);

Create table table_name as select column_name1,column_name2,.....column_namen from table_name where predicate;

AS Subquery

Subquery is the select statement that defines the set of rows to be inserted into the new table.

Example

Create table dept80 as select employee_id, last_name, salary*12 Annsal, hire_date from employees where dept_id=80;

The ALTER TABLE Statement

The ALTER statement is used to

- Add a new column
- Modify an existing column
- Define a default value to the new column
- Drop a column
- To include or drop integrity constraint.

SYNTAX

ALTER TABLE table_name ADD /MODIFY(Column_name type(size));

ALTER TABLE table_name DROP COLUMN (Column_nname);

ALTER TABLE ADD CONSTRAINT Constraint_name PRIMARY KEY (Colum_Name);

Example:

Alter table dept80 add (jod_id varchar2(9));

Alter table dept80 modify (last_name varchar2(30));

Alter table dept80 drop column job_id;

NOTE: Once the column is dropped it cannot be recovered.

DROPPING A TABLE

- All data and structure in the table is deleted.
- Any pending transactions are committed.
- All indexes are dropped.
- Cannot roll back the drop table statement.

Syntax:

Drop table *tablename*;

Example:

Drop table dept80;

RENAMING A TABLE

To rename a table or view.

Syntax

RENAME old_name to new_name

Example:

Rename dept to detail_dept;

TRUNCATING A TABLE

Removes all rows from the table.

Releases the storage space used by that table.

Syntax

TRUNCATE TABLE *table_name*;

Example:

TRUNCATE TABLE copy_emp;

Find the Solution for the following:

Create the following tables with the given structure.

EMPLOYEES TABLE

NAME	NULL?	TYPE
Employee_id	Not null	Number(6)
First Name		Varchar(20)
Last Name	Not null	Varchar(25)

Email	Not null	Varchar(25)
Phone Number		Varchar(20)
Hire date	Not null	Date
Job id	Not null	Varchar(10)
Salary		Number(8,2)
Commission_pct		Number(2,2)
Manager id		Number(6)
Department_id		Number(4)

DEPARTMENT TABLE

NAME	NULL?	TYPE
Dept_id	Not null	Number(6)
Dept name	Not null	Varchar(20)
Manager id		Number(6)
Location_id		Number(4)

JOB_GRADE TABLE

NAME	NULL?	TYPE
Grade_level		Varchar(2)
Lowest_sal		Number
Highest_sal		Number

LOCATION TABLE

NAME	NULL?	TYPE
Location_id	Not null	Number(4)
St_addr		Varchar(40)
Postal_code		Varchar(12)
City	Not null	Varchar(30)
State_province		Varchar(25)
Country_id		Char(2)

1. Create the DEPT table based on the DEPARTMENT following the table instance chart below. Confirm that the table is created.

Column name	ID	NAME
Key Type		
Nulls/Unique		
FK table		
FK column		
Data Type	Number	Varchar2

Length	7	25
---------------	---	----

```
CREATE TABLE DEPT(Dept_id number(6) not null,Dept_name varchar(20) not null,manager_id
number(6),location_id number(4));
```

Table created.

2. Create the EMP table based on the following instance chart. Confirm that the table is created.

Column name	ID	LAST_NAME	FIRST_NAME	DEPT_ID
Key Type				
Nulls/Unique				
FK table				
FK column				
Data Type	Number	Varchar2	Varchar2	Number
Length	7	25	25	7

```
create table EMP(employee_id number(6) not null,first_name varchar(20),last_name
varchar(25) not null,email varchar(25) not null,phone_number varchar(20),hire_date Date not
null,job_id varchar(6) not null,salary number(8,2),commission_pct number(2,2),manager_id
```

Table created.

3. Modify the EMP table to allow for longer employee last names. Confirm the modification.(Hint: Increase the size to 50)

```
alter table EMP modify(last_name varchar(50));
```

Table altered.

4. Create the EMPLOYEES2 table based on the structure of EMPLOYEES table. Include Only the Employee_id, First_name, Last_name, Salary and Dept_id columns. Name the columns Id, First_name, Last_name, salary and Dept_id respectively.

```
create table EMPLOYEES2 as select employee_id as  
id,first_name,last_name,salary,department_id as dept_id from EMP;
```

Table created.

5. Drop the EMP table.

```
drop table EMP;
```

Table dropped.

6. Rename the EMPLOYEES2 table as EMP.

```
rename EMPLOYEES2 to EMP;
```

Table renamed.

7. Add a comment on DEPT and EMP tables. Confirm the modification by describing the table.

```
TABLE DEPT AND EMP WAS CREATED. EMP TABLE WAS DROPPED. TABLE  
EMPLOYEES2 WAS RENAMED AS EMP.
```

8. Drop the First_name column from the EMP table and confirm it.

```
ALTER TABLE EMP DROP column first_name;
```

Table altered.

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

EXERCISE-2

MANIPULATING DATA

OBJECTIVE

After, the completion of this exercise the students will be able to do the following

- Describe each DML statement
- Insert rows into tables
- Update rows into table
- Delete rows from table
- Control Transactions

A DML statement is executed when you:

- Add new rows to a table
- Modify existing rows
- Removing existing rows

A transaction consists of a collection of DML statements that form a logical unit of work.

To Add a New Row

INSERT Statement

Syntax

INSERT INTO table_name VALUES (column1 values, column2 values, ..., columnn values);

Example:

INSERT INTO department (70, 'Public relations', 100,1700);

Inserting rows with null values

Implicit Method: (Omit the column)

INSERT INTO department VALUES (30,'purchasing');

Explicit Method: (Specify NULL keyword)

INSERT INTO department VALUES (100,'finance', NULL, NULL);

Inserting Special Values

Example:

Using SYSDATE

```
INSERT INTO employees VALUES (113,'louis', 'popp', 'lpopp','5151244567',SYSDATE,
'ac_account', 6900, NULL, 205, 100);
```

Inserting Specific Date Values

Example:

```
INSERT INTO employees VALUES ( 114,'den', 'raphealy', 'drapheal', '5151274561',
TO_DATE('feb 3,1999','mon, dd ,yyyy'), 'ac_account', 11000,100,30);
```

To Insert Multiple Rows

& is the placeholder for the variable value

Example:

```
INSERT INTO department VALUES (&dept_id, &dept_name, &location);
```

Copying Rows from another table

➤ Using Subquery

Example:

```
INSERT INTO sales_reps(id, name, salary, commission_pct)
SELECT employee_id, Last_name, salary, commission_pct
FROM employees
WHERE job_id LIKE '%REP');
```

CHANGING DATA IN A TABLE

UPDATE Statement

Syntax1: (to update specific rows)

```
UPDATE table_name SET column=value WHERE condition;
```

Syntax 2: (To updae all rows)

```
UPDATE table_name SET column=value;
```

Updating columns with a subquery

```
UPDATE employees
SET job_id= (SELECT job_id
FROM employees
WHERE employee_id=205)
WHERE employee_id=114;
```

REMOVING A ROW FROM A TABLE

DELETE STATEMENT

Syntax

DELETE FROM table_name WHERE conditions;

Example:

DELETE FROM department WHERE dept_name='finance';

Find the Solution for the following:

1. Create MY_EMPLOYEE table with the following structure

NAME	NULL?	TYPE
ID	Not null	Number(4)
Last_name		Varchar(25)
First_name		Varchar(25)
Userid		Varchar(25)
Salary		Number(9,2)

create table MY_EMPLOYEE(id number(4) not null,last_name varchar(25),first_name varchar(25),userid varchar(25),salary number(9,2));

Table created.

2. Add the first and second rows data to MY_EMPLOYEE table from the following sample data.

ID	Last_name	First_name	Userid	salary
1	Patel	Ralph	rpatel	895
2	Dancs	Betty	bdancs	860
3	Biri	Ben	bbiri	1100
4	Newman	Chad	Cnewman	750

5	Ropebur	Audrey	aropebur	1550
---	---------	--------	----------	------

```
insert into MY_EMPLOYEE values(1,'Patel','Ralph','rpatel',895);
insert into MY_EMPLOYEE values(2,'Dancs','Betty','bdancs',860);
```

```
Query OK, 2 rows affected (0.257 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

3. Display the table with values.

```
select * from MY_EMPLOYEE;
```

```
+-----+-----+-----+-----+-----+
| id | last_name | first_name | userid | salary |
+-----+-----+-----+-----+-----+
| 1 | Patel | Ralph | rpatel | 895.00 |
| 2 | Dancs | Betty | bdancs | 860.00 |
+-----+-----+-----+-----+-----+
2 rows in set (0.010 sec)
```

4. Populate the next two rows of data from the sample data. Concatenate the first letter of the first_name with the first seven characters of the last_name to produce Userid.

```
insert into MY_EMPLOYEE
values(3,'Biri','Ben','bbiri',1100),(4,'Newman','Chad','cnewman',750);
```

```
Query OK, 2 rows affected (0.257 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

5. Make the data additions permanent.

```
select * from MY_EMPLOYEE;
```

```
+-----+-----+-----+-----+-----+
| id | last_name | first_name | userid | salary |
+-----+-----+-----+-----+-----+
| 1 | Patel | Ralph | rpatel | 895.00 |
| 2 | Dancs | Betty | bdancs | 860.00 |
| 3 | Biri | Ben | bbiri | 1100.00 |
| 4 | Newman | Chad | cnewman | 750.00 |
+-----+-----+-----+-----+-----+
4 rows in set (0.009 sec)
```

6. Change the last name of employee 3 to Drexler.

```
update MY_EMPLOYEE set last_name = 'Drexler' where id=3;
```

```
Query OK, 1 row affected (0.067 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

7. Change the salary to 1000 for all the employees with a salary less than 900.

```
update MY_EMPLOYEE set salary=1000 where salary<900;
```

```
Query OK, 3 rows affected (0.273 sec)
Rows matched: 3  Changed: 3  Warnings: 0
```

8. Delete Betty dances from MY_EMPLOYEE table.

```
delete from MY_EMPLOYEE where first_name='Betty';
```

```
Query OK, 1 row affected (0.245 sec)
```

9. Empty the fourth row of the emp table.

```
delete from MY_EMPLOYEE where id=4;
```

```
Query OK, 1 row affected (0.245 sec)
```

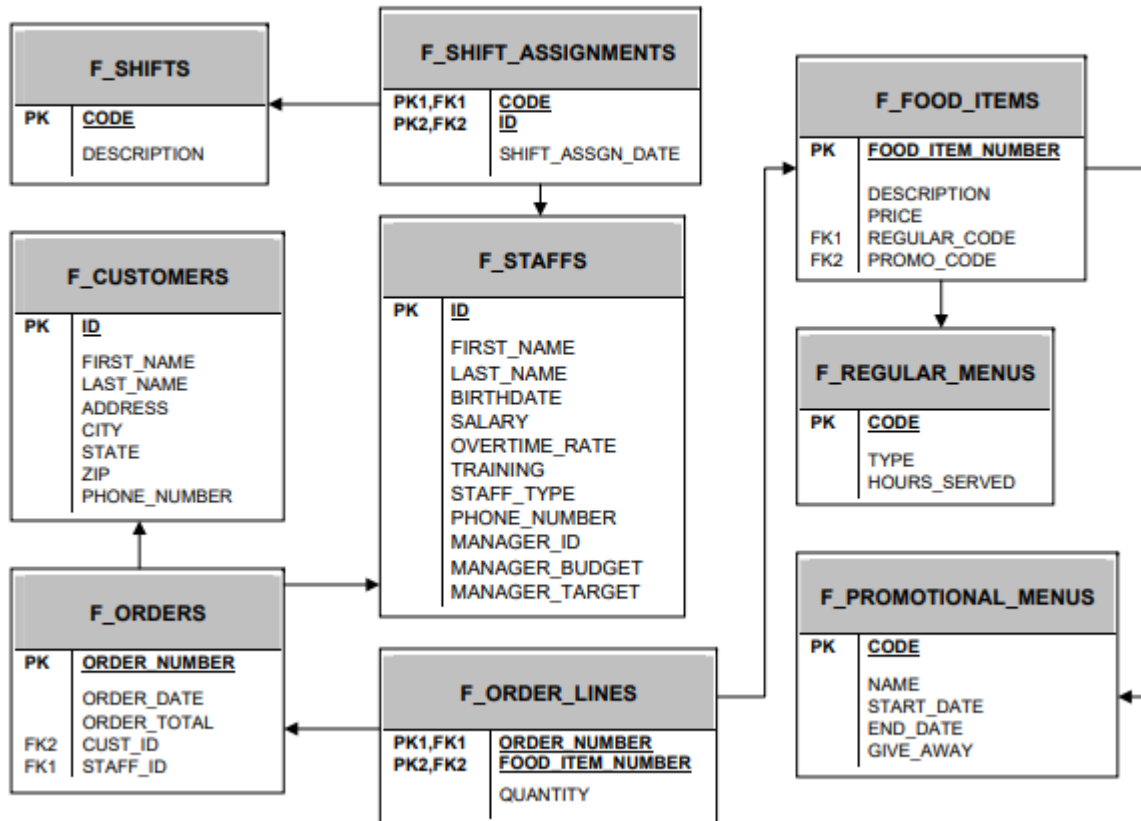
Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

PRACTICE QUESTIONS

Date:

Working with Columns, Characters, and Rows

Global Fast Foods Database Tables



1. The manager of Global Fast Foods would like to send out coupons for the upcoming sale. He wants to send one coupon to each household. Create the SELECT statement that returns the customer last name and a mailing address.

**SELECT DISTINCT last_name, address
FROM customers;**

2. Each statement below has errors. Correct the errors and execute the query in Oracle Application Express.

a.

SELECT first name FROM

f_staffs;

b.

```
SELECT first_name || " " || last_name AS "DJs on Demand Clients"  
FROM d_clients;
```

c.

```
SELECT DISCTINCT f_order_lines  
FROM quantity;
```

d.

```
SELECT order number  
FROM f_orders;
```

3. Sue, Bob, and Monique were the employees of the month. Using the f_staffs table, create a SELECT statement to display the results as shown in the Super Star chart.

Super Star
*** Sue *** Sue ***
*** Bob *** Bob ***
*** Monique *** Monique ***

```
SELECT '*** ' || first_name || ' *** ' || first_name || ' ***' AS "Super Star"  
FROM f_staffs  
WHERE first_name IN ('Sue', 'Bob', 'Monique');
```

4.Which of the following is TRUE about the following query?

```
SELECT first_name, DISTINCT birthdate  
FROM f_staffs;
```

- a. **Only two rows will be returned.**
- b. Four rows will be returned.
- c. Only Fred 05-Jan-1988 and Lizzie 10-Nov-1987 will be returned.

d. No rows will be returned.

5. Global Fast Foods has decided to give all staff members a 5% raise. Prepare a report that presents the output as shown in the chart.

EMPLOYEE LAST NAME	CURRENT SALARY	SALARY WITH 5% RAISE

```
SELECT last_name AS "EMPLOYEE LAST NAME",  
       salary AS "CURRENT SALARY",  
       salary * 1.05 AS "SALARY WITH 5% RAISE"  
FROM f_staffs;
```

6. Create a query that will return the structure of the Oracle database EMPLOYEES table. Which columns are marked “nullable”? What does this mean?

```
DESC employees;
```

7. The owners of DJs on Demand would like a report of all items in their D_CDs table with the following column headings: Inventory Item, CD Title, Music Producer, and Year Purchased. Prepare this report.

```
SELECT cd_number AS "Inventory Item", title AS "CD Title", producer AS "Music Producer",  
       year_purchased AS "Year Purchased" FROM d_cds;
```

8.True/False – The following SELECT statement executes successfully: SELECT last_name, job_id, salary AS Sal FROM employees;

```
SELECT last_name, job_id, salary AS Sal  
FROM employees;
```

9.True/False – The following SELECT statement executes successfully: SELECT

* FROM job_grades;

TRUE

10. There are four coding errors in this statement. Can you identify them?

SELECT employee_id, last_name sal x 12 ANNUAL SALARY FROM employees;

11. In the arithmetic expression salary*12 - 400, which operation will be evaluated first?

SELECT employee_id, last_name, salary * 12 AS "ANNUAL SALARY"

FROM employees;

12. Which of the following can be used in the SELECT statement to return all columns of data in the Global Fast Foods f_staffs table?

- a. column names
- b. *
- c. DISTINCT id
- d. **both a and b**

13. Using SQL to choose the columns in a table uses which capability?

- a. selection
- b. **projection**
- c. partitioning
- d. join

14. SELECT last_name AS "Employee". The column heading in the query result will appear as:

- a. EMPLOYEE
- b. employee
- c. **Employee**
- d. "Employee:"

15. Which expression below will produce the largest value?

- a. SELECT salary*6 + 100
- b. SELECT salary* (6 + 100)**
- c. SELECT 6(salary+ 100)
- d. SELECT salary+6*100

16. Which statement below will return a list of employees in the following format?

Mr./Ms. Steven King is an employee of our company.

- a. SELECT "Mr./Ms." || first_name || ' ' || last_name 'is an employee of our company.' AS "Employees" FROM employees;
- b. SELECT 'Mr./Ms. ' || first_name || last_name || ' ' || 'is an employee of our company.' FROM employees;
- c. SELECT 'Mr./Ms. ' || first_name || ' ' || last_name || ' ' || 'is an employee of our company.' AS "Employees" FROM employees ;**
- d. SELECT Mr./Ms. || first_name || ' ' || last_name || ' ' || "is an employee of our company." AS "Employees" FROM employees

17. Which is true about SQL statements?

- a. SQL statements are case-sensitive
- b. SQL clauses should not be written on separate lines.
- c. Keywords cannot be abbreviated or split across lines.**
- d. SQL keywords are typically entered in lowercase; all other words in uppercase.

18. Which queries will return three columns each with UPPERCASE column headings?

- a. SELECT "Department_id", "Last_name", "First_name" FROM employees;
- b. SELECT DEPARTMENT_ID, LAST_NAME, FIRST_NAME FROM employees;**
- c. SELECT department_id, last_name, first_name AS UPPER CASE FROM employees
- d. SELECT department_id, last_name, first_name FROM employees;

19. Which statement below will likely fail?

- a. SELECT * FROM employees;**
- b. Select * FROM employees;

- c. `SELECT * FROM EMPLOYEES;`
- d. `Select* FROM employees;`

20. Click on the History link at the bottom of the SQL Commands window. Scroll or use the arrows at the bottom of the page to find the statement you wrote to solve problem 3 above. (The one with the column heading SuperStar). Click on the statement to load it back into the command window. Execute the command again, just to make sure it is the correct one that works. Once you know it works, click on the SAVE button in the top right corner of the SQL Commands window, and enter a name for your saved statement. Use your own initials and “_superstar.sql”, so if your initials are CT then the filename will be CT_superstar.sql.

Log out of OAE, and log in again immediately. Navigate back to the SQL Commands window, click the Saved SQL link at the bottom of the page and load your saved SQL statement into the Edit window. This is done by clicking on the script name. Edit the statement, to make it display + instead of *. Run your amended statement and save it as initials_superplus.sql.

Steps Only:

- Click **History** at the bottom of SQL Commands.
- Locate and click the query from problem 3 (Super Star).
- Execute the query to verify it works.
- Click **SAVE** and name it initials_superstar.sql.
- Log out and log back in.
- Navigate to **Saved SQL**, load your saved script.
- Edit the statement to replace * with +.
- Run the amended query to verify output.
- Click **SAVE** and name it initials_superplus.sql.

Evaluation Procedure	Marks awarded
Practice Evaluation (5)	
Viva(5)	
Total (10)	
Faculty Signature	

EXERCISE-3

INCLUDING CONSTRAINTS

OBJECTIVE

After the completion of this exercise the students should be able to do the following

- Describe the constraints
- Create and maintain the constraints

What are Integrity constraints?

- Constraints enforce rules at the table level.
- Constraints prevent the deletion of a table if there are dependencies

The following types of integrity constraints are valid

a) **Domain Integrity**

- ✓ NOT NULL
- ✓ CHECK

b) **Entity Integrity**

- ✓ UNIQUE
- ✓ PRIMARY KEY

c) **Referential Integrity**

- ✓ FOREIGN KEY

Constraints can be created in either of two ways

1. At the same time as the table is created
2. After the table has been created.

Defining Constraints

Create table tablename (column_name1 data_type constraints, column_name2 data_type constraints ...);

Example:

Create table employees (employee_id number(6), first_name varchar2(20), ..job_id varchar2 (10),
CONSTRAINT emp_emp_id_pk PRIMARY KEY (employee_id));

Domain Integrity

This constraint sets a range and any violations that takes place will prevent the user from performing the manipulation that caused the breach.It includes:

NOT NULL Constraint

While creating tables, by default the rows can have null value.the enforcement of not null constraint in a table ensure that the table contains values.

Principle of null values:

- Setting null value is appropriate when the actual value is unknown, or when a value would not be meaningful.
- A null value is not equivalent to a value of zero.
- A null value will always evaluate to null in any expression.

- When a column name is defined as not null, that column becomes a mandatory i.e., the user has to enter data into it.
- Not null Integrity constraint cannot be defined using the alter table command when the table contain rows.

Example

```
CREATE TABLE employees (employee_id number (6), last_name varchar2(25) NOT NULL, salary
number(8,2), commission_pct number(2,2), hire_date date constraint emp_hire_date_nn NOT
NULL'....);
```

CHECK

Check constraint can be defined to allow only a particular range of values.when the manipulation violates this constraint,the record will be rejected.Check condition cannot contain sub queries.

```
CREATE TABLE employees (employee_id number (6), last_name varchar2 (25) NOT NULL, salary
number(8,2), commission_pct number(2,2), hire_date date constraint emp_hire_date_nn NOT
NULL'...,CONSTRAINT emp_salary_mi CHECK(salary > 0));
```

Entity Integrity

Maintains uniqueness in a record. An entity represents a table and each row of a table represents an instance of that entity. To identify each row in a table uniquely we need to use this constraint. There are 2 entity constraints:

a) Unique key constraint

It is used to ensure that information in the column for each record is unique, as with telephone or driver's license numbers. It prevents the duplication of value with rows of a specified column in a set of column. A column defined with the constraint can allow null value.

If unique key constraint is defined in more than one column i.e., combination of column cannot be specified. Maximum combination of columns that a composite unique key can contain is 16.

Example:

```
CREATE TABLE employees (employee_id number(6), last_name varchar2(25) NOT NULL,email
varchar2(25), salary number(8,2), commission_pct number(2,2), hire_date date constraint
emp_hire_date_nn NOT NULL' COSTRAINT emp_email_uk UNIQUE(email));
```

PRIMARY KEY CONSTRAINT

A primary key avoids duplication of rows and does not allow null values. Can be defined on one or more columns in a table and is used to uniquely identify each row in a table. These values should never be changed and should never be null.

A table should have only one primary key. If a primary key constraint is assigned to more than one column or combination of column is said to be composite primary key, which can contain 16 columns.

Example:

```
CREATE TABLE employees (employee_id number(6) , last_name varchar2(25) NOT NULL,email
varchar2(25), salary number(8,2), commission_pct number(2,2), hire_date date constraint
emp_hire_date_nn NOT NULL, Constraint emp_id pk PRIMARY KEY
(employee_id),CONSTRAINT emp_email_uk UNIQUE(email));
```

c) Referential Integrity

It enforces relationship between tables. To establish parent-child relationship between 2 tables having a common column definition, we make use of this constraint. To implement this, we should define the column in the parent table as primary key and same column in the child table as foreign key referring to the corresponding parent entry.

Foreign key

A column or combination of column included in the definition of referential integrity, which would refer to a referenced key.

Referenced key

It is a unique or primary key upon which is defined on a column belonging to the parent table.
Keywords:

FOREIGN KEY: Defines the column in the child table at the table level constraint.

REFERENCES: Identifies the table and column in the parent table.

ON DELETE CASCADE: Deletes the dependent rows in the child table when a row in the parent table is deleted.

ON DELETE SET NULL: converts dependent foreign key values to null when the parent value is removed.

```
CREATE TABLE employees (employee_id number(6) , last_name varchar2(25) NOT
NULL,email varchar2(25), salary number(8,2), commission_pct number(2,2), hire_date date
constraint emp_hire_date_nn NOT NULL, Constraint emp_id pk PRIMARY KEY
(employee_id),CONSTRAINT emp_email_uk UNIQUE(email),CONSTRAINT emp_dept_fk
FOREIGN KEY (department_id) references deparments(dept_id));
```

ADDING A CONSTRAINT

Use the ALTER to

- Add or Drop a constraint, but not modify the structure
- Enable or Disable the constraints
- Add a not null constraint by using the Modify clause

Syntax

ALTER TABLE table name ADD CONSTRAINT Cons_name type(column name);

Example:

ALTER TABLE employees ADD CONSTRAINT emp_manager_fk FOREIGN KEY (manager_id) REFERENCES employees (employee_id);

DROPPING A CONSTRAINT

Example:

ALTER TABLE employees DROP CONSTRAINT emp_manager_fk;

CASCADE IN DROP

- The CASCADE option of the DROP clause causes any dependent constraints also to be dropped.

Syntax

ALTER TABLE departments DROP PRIMARY KEY|UNIQUE (column)| CONSTRAINT constraint_name CASCADE;

DISABLING CONSTRAINTS

- Execute the DISABLE clause of the ALTER TABLE statement to deactivate an integrity constraint
- Apply the CASCADE option to disable dependent integrity constraints.

Example

ALTER TABLE employees DISABLE CONSTRAINT emp_emp_id_pk CASCADE;

ENABLING CONSTRAINTS

- Activate an integrity constraint currently disabled in the table definition by using the ENABLE clause.

Example

```
ALTER TABLE employees ENABLE CONSTRAINT emp_emp_id_pk CASCADE;
```

CASCADING CONSTRAINTS

The CASCADE CONSTRAINTS clause is used along with the DROP column clause. It drops all referential integrity constraints that refer to the primary and unique keys defined on the dropped Columns.

This clause also drops all multicolumn constraints defined on the dropped column.

Example:

Assume table TEST1 with the following structure

```
CREATE TABLE test1 ( pk number PRIMARY KEY, fk number, col1 number,col2 number,  
CONSTRAINT fk_constraint FOREIGN KEY(fk) references test1, CONSTRAINT ck1 CHECK  
(pk>0 and col1>0), CONSTRAINT ck2 CHECK (col2>0));
```

An error is returned for the following statements

```
ALTER TABLE test1 DROP (pk);
```

```
ALTER TABLE test1 DROP (col1);
```

The above statement can be written with CASCADE CONSTRAINT

```
ALTER TABLE test 1 DROP(pk) CASCADE CONSTRAINTS;
```

(OR)

```
ALTER TABLE test 1 DROP(pk, fk, col1) CASCADE CONSTRAINTS;
```

VIEWING CONSTRAINTS

Query the USER_CONSTRAINTS table to view all the constraints definition and names.

Example:

```
SELECT constraint_name, constraint_type, search_condition FROM user_constraints  
WHERE table_name='employees';
```

Viewing the columns associated with constraints

```
SELECT constraint_name, constraint_type, FROM user_cons_columns  
WHERE table_name='employees';
```

Find the Solution for the following:

1. Add a table-level PRIMARY KEY constraint to the EMP table on the ID column. The constraint should be named at creation. Name the constraint my_emp_id_pk.

```
create table EMP(employee_id number(6) not null,first_name varchar(20),last_name  
varchar(25) not null,email varchar(25) not null,phone_number varchar(20),hire_date Date  
not null,job_id varchar(10) not null,salary number(8,2),commission_pct  
number(2,2),manager_id number(6),department_id number(4),constraint my_emp_id_pk  
primary key(employee_id));
```

```
Query OK, 0 rows affected, 3 warnings (0.215 sec)
```

2. Create a PRIMARY KEY constraint to the DEPT table using the ID column. The constraint should be named at creation. Name the constraint my_dept_id_pk.

```
create table DEPT(dept_id number(6) not null,dept_name varchar(20) not null,manager_id  
number(6),id number(4),constraint my_dept_id_pk primary key(dept_id));
```

```
Query OK, 0 rows affected, 3 warnings (0.496 sec)
```

3. Add a column DEPT_ID to the EMP table. Add a foreign key reference on the EMP table that ensures that the employee is not assigned to nonexistent department. Name the constraint my_emp_dept_id_fk.

```
alter table EMP add column dept_id number(4);
```

```
Query OK, 0 rows affected, 1 warning (0.870 sec)  
Records: 0 Duplicates: 0 Warnings: 1
```

```
alter table EMP add constraint my_emp_dept_id_fk foreign key(dept_id) references  
dept(dept_id);
```

```
Query OK, 0 rows affected (0.619 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

4. Modify the EMP table. Add a COMMISSION column of NUMBER data type, precision 2, scale 2. Add a constraint to the commission column that ensures that a commission value is greater than zero.

```
alter table EMP add(commission decimal(2,2));
```

```
Query OK, 0 rows affected (0.619 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
alter table EMP add constraint positive check(commission>0);
```

```
Query OK, 0 rows affected (0.802 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

PRACTICE QUESTIONS

Limit Rows Selected

1. Using the Global Fast Foods database, retrieve the customer's first name, last name, and address for the customer who uses ID 456.

SELECT first_name, last_name, address FROM customer WHERE customer_id = 456;

2. Show the name, start date, and end date for Global Fast Foods' promotional item "ballpen and highlighter" giveaway.

SELECT promo_name, start_date, end_date FROM promotions WHERE promo_name = 'ballpen and highlighter';

3. Create a SQL statement that produces the following output:

Oldest

The 1997 recording in our database is The Celebrants Live in Concert

SELECT 'Oldest' AS "Oldest",

'The 1997 recording in our database is The Celebrants Live in Concert' AS "Recording"
FROM dual;

4. The following query was supposed to return the CD title "Carpe Diem" but no rows were returned. Correct the mistake in the statement and show the output.

SELECT produce, title
FROM d_cds
WHERE title = 'carpe diem' ;

SELECT producer, title FROM d_cds WHERE title = 'Carpe Diem';
--

5. The manager of DJs on Demand would like a report of all the CD titles and years of CDs that were produced before 2000.

SELECT title, year_purchased FROM d_cds WHERE year_purchased < 2000;

6. Which values will be selected in the following query?

SELECT salary

FROM employees
WHERE salary <= 5000;

- a. **5000**
- b. 0 - 4999
- c. 2500
- d. 5

7. Write a SQL statement that will display the student number (studentno), first name (fname), and last name (lname) for all students who are female (F) in the table named students.

SELECT studentno, fname, lname FROM students WHERE gender = 'F';

8. Write a SQL statement that will display the student number (studentno) of any student who has a PE major in the table named students. Title the studentno column Student Number.

SELECT studentno AS "Student Number" FROM students WHERE major = 'PE';

9. Write a SQL statement that lists all information about all male students in the table named students.

SELECT * FROM students WHERE gender = 'M';

10. Write a SQL statement that will list the titles and years of all the DJs on Demand's CDs that were not produced in 2000.

SELECT title, year_purchased FROM d_cds WHERE year_purchased <> 2000;

11. Write a SQL statement that lists the Global Fast Foods employees who were born before 1980.

SELECT * FROM employees WHERE birth_date < TO_DATE('01-JAN-1980', 'DD-MON-YYYY');

Evaluation Procedure	Marks awarded
Practice Evaluation (5)	
Viva(5)	
Total (10)	
Faculty Signature	

EXERCISE-4

Writing Basic SQL SELECT Statements

OBJECTIVES

After the completion of this exercise, the students will be able to do the following:

- List the capabilities of SQL SELECT Statement
- Execute a basic SELECT statement

Capabilities of SQL SELECT statement

A SELECT statement retrieves information from the database. Using a select statement, we can perform

- ✓ Projection: To choose the columns in a table
- ✓ Selection: To choose the rows in a table
- ✓ Joining: To bring together the data that is stored in different tables

Basic SELECT Statement

Syntax

```
SELECT *|DISTINCT Column_name| alias  
' FROM table_name;
```

NOTE:

DISTINCT—Suppr
ess the duplicates.

Alias—gives selected columns different headings.

Example: 1

```
SELECT * FROM departments;
```

Example: 2

```
SELECT location_id, department_id FROM departments;
```

Writing SQL Statements

- SQL statements are not case sensitive
- SQL statements can be on one or more lines.
- Keywords cannot be abbreviated or split across lines
- Clauses are usually placed on separate lines
- Indents are sued to enhance readability

Using Arithmetic Expressions

Basic Arithmetic operators like *, /, +, -can be used

Example:1

```
SELECT last_name, salary, salary+300 FROM employees;
```

Example:2

```
SELECT last_name, salary, 12*salary+100 FROM employees;
```

The statement is not same as

```
SELECT last_name, salary, 12*(salary+100) FROM employees;
```

Example:3

```
SELECT last_name, job_id, salary, commission_pct FROM employees;
```

Example:4

```
SELECT last_name, job_id, salary, 12*salary*commission_pct FROM employees;
```

Using Column Alias

- To rename a column heading with or without AS keyword.

Example:1

```
SELECT last_name AS Name  
FROM employees;
```

Example: 2

```
SELECT last_name "Name" salary*12 "Annual Salary "  
FROM employees;
```


Concatenation Operator

- Concatenates columns or character strings to other columns
- Represented by two vertical bars (||)
- Creates a resultant column that is a character expression

Example:

SELECT last_name||job_id AS "EMPLOYEES JOB" FROM employees;

Using Literal Character String

- A literal is a character, a number, or a date included in the SELECT list.
- Date and character literal values must be enclosed within single quotation marks.

Example:

SELECT last_name||'is a'||job_id AS "EMPLOYEES JOB" FROM employees;

Eliminating Duplicate Rows

- Using DISTINCT keyword.

Example:

SELECT DISTINCT department_id FROM employees;

Displaying Table Structure

- Using DESC keyword.

Syntax

DESC table_name;

Example:

DESC employees;

Find the Solution for the following:

True OR False

1. The following statement executes successfully.

Identify the Errors

```
SELECT employee_id, last_name  
sal*12 ANNUAL SALARY  
FROM employees;
```

Queries

select id,last_name,salary*12 as "ANNUAL SALARY" from MY_EMPLOYEE;

```

+-----+-----+-----+
| id | last_name | ANNUAL SALARY |
+-----+-----+-----+
| 1 | Patel | 12000.00 |
| 3 | Drexler | 13200.00 |
+-----+-----+-----+
2 rows in set (0.017 sec)

```

2. Show the structure of departments the table. Select all the data from it.

describe DEPT;

Name	Null?	Type
DEPT_ID	NOT NULL	NUMBER(6)
DEPT_NAME	NOT NULL	VARCHAR2(20)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

select * from DEPT;

```

+-----+-----+-----+-----+-----+-----+-----+
| id | last_name | first_name | userid | salary | job_code | hire_date |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Patel | Ralph | rpatel | 1000.00 | 10 | 1990-10-12 |
| 3 | Drexler | Ben | bbiri | 1100.00 | 20 | 1990-12-03 |
| 2 | vishnu | s | svishnu | 7000.00 | 20 | 2002-03-28 |
| 4 | Thani | s | sthani | 8000.00 | 40 | 1995-02-28 |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.011 sec)

```

3. Create a query to display the last name, job code, hire date, and employee number for each employee, with employee number appearing first.

select id,last_name,job_code,hire_date from MY_EMPLOYEE;

```

+-----+-----+-----+-----+
| id | last_name | job_code | hire_date |
+-----+-----+-----+-----+
| 1 | Patel | 10 | 1990-10-12 |
| 3 | Drexler | 20 | 1990-12-03 |
| 2 | vishnu | 20 | 2002-03-28 |
| 4 | Thani | 40 | 1995-02-28 |
+-----+-----+-----+-----+
4 rows in set (0.014 sec)

```

4. Provide an alias STARTDATE for the hire date.

select hire_date as "STARTDATE" from MY_EMPLOYEE;

```
+-----+
| STARTDATE |
+-----+
| 1990-10-12 |
| 1990-12-03 |
| 2002-03-28 |
| 1995-02-28 |
+-----+
4 rows in set (0.012 sec)
```

5. Create a query to display unique job codes from the employee table.

select distinct job_code from MY_EMPLOYEE;

```
+-----+
| job_code |
+-----+
| 10       |
| 20       |
| 40       |
+-----+
3 rows in set (0.220 sec)
```

6. Display the last name concatenated with the job ID , separated by a comma and space, and name the column EMPLOYEE and TITLE.

**SELECT last_name || ', ' || job_code AS "Employee and Title"
FROM MY_EMPLOYEE;**

```

+-----+
| Employee and Title |
+-----+
| Patel, 10          |
| Drexler, 20        |
| vishnu, 20         |
| Thani, 40          |
+-----+
4 rows in set (0.015 sec)

```

7. Create a query to display all the data from the employees table. Separate each column by a comma. Name the column THE_OUTPUT.

```

SELECT employee_id || ',' || first_name || ',' || last_name || ',' || userid || ',' || job_id || ',' || salary || ',' ||
Hire_date AS the_output FROM employees;

```

```

+-----+
| THE_OUTPUT |
+-----+
| 1, Ralph, Patel, rpatel, 1990-10-12, 1000.00, |
| 3, Ben, Drexler, bbiri, 1990-12-03, 1100.00, |
| 2, s, vishnu, svishnu, 2002-03-28, 7000.00, |
| 4, s, Thani, sthani, 1995-02-28, 8000.00, |
+-----+
4 rows in set (0.017 sec)

```

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

Practice Questions

COMPARISON OPERATORS

1. Who are the partners of DJs on Demand who do not get an authorized expense amount?

SELECT * FROM d_partners WHERE authorized_expense IS NULL;

2. Select all the Oracle database employees whose last names end with "s". Change the heading of the column to read Possible Candidates.

SELECT last_name AS "Possible Candidates" FROM employees WHERE last_name LIKE '%s';

3. Which statement(s) are valid?

- a. WHERE quantity <> NULL;
- b. WHERE quantity = NULL;
- c. **WHERE quantity IS NULL;**
- d. WHERE quantity != NULL;

4. Write a SQL statement that lists the songs in the DJs on Demand inventory that are type code 77, 12, or 1.

SELECT * FROM d_songs WHERE type_code IN (77, 12, 1);

Logical Comparisons and Precedence Rules

1. Execute the two queries below. Why do these nearly identical statements produce two different results? Name the difference and explain why.

```
SELECT code, description
FROM d_themes
WHERE code >200 AND description IN('Tropical', 'Football', 'Carnival'); SELECT
code, description
FROM d_themes
WHERE code >200 OR description IN('Tropical', 'Football', 'Carnival');
```

Query 1: AND
SELECT code, description FROM d_themes WHERE code > 200 AND
description IN ('Tropical', 'Football', 'Carnival');

Query 2: OR
SELECT code, description FROM d_themes WHERE code > 200 OR
descriptionIN ('Tropical', 'Football', 'Carnival');

2. Display the last names of all Global Fast Foods employees who have “e” and “i” in their last names.

```
SELECT last_name FROM employees
WHERE last_name LIKE '%e%' AND last_name LIKE '%i%';
```

3. “I need to know who the Global Fast Foods employees are that make more than \$6.50/hour and their position is not order taker.”

```
SELECT * FROM employees WHERE salary > 6.50 AND job_id <> 'ORDER_TAKER';
```

4. Using the employees table, write a query to display all employees whose last names start with “D” and have “a” and “e” anywhere in their last name.

```
SELECT * FROM employees WHERE last_name LIKE 'D%' AND last_name LIKE '%a%' AND last_name LIKE '%e%';
```

5. In which venues did DJs on Demand have events that were not in private homes?

```
SELECT DISTINCT venue FROM d_events WHERE venue <> 'Private Home';
```

6. Which list of operators is in the correct order from highest precedence to lowest precedence?

- a. AND, NOT, OR
- b. NOT, OR, AND
- c. **NOT, AND, OR**

For questions 7 and 8, write SQL statements that will produce the desired output.

7. Who am I?

I was hired by Oracle after May 1998 but before June of 1999. My salary is less than \$8000 per month, and I have an “en” in my last name.

```
SELECT * FROM employees WHERE hire_date > TO_DATE('31-MAY-1998','DD-MON-YYYY')
AND hire_date < TO_DATE('01-JUN-1999','DD-MON-YYYY') AND salary < 8000
```

AND last_name LIKE '%en%';

8. What's my email address?

Because I have been working for Oracle since the beginning of 1996, I make more than \$9000 per month. Because I make so much money, I don't get a commission

SELECT email FROM employees WHERE hire_date >= TO_DATE('01-JAN-1996','DDMON-YYYY') AND salary > 9000 AND commission_pct IS NULL;

Evaluation Procedure	Marks awarded
Practice Evaluation (5)	
Viva(5)	
Total (10)	
Faculty Signature	

EXERCISE-5

Restricting and Sorting data

After the completion of this exercise, the students will be able to do the following:

- Limit the rows retrieved by the queries

- Sort the rows retrieved by the queries
-

Limiting the Rows selected

- Using WHERE clause
- Alias cannot used in WHERE clause

Syntax

SELECT-----
FROM-----
WHERE condition;

Example:

SELECT employee_id,last_name, job_id, deparment_id FROM employees WHERE department_id=90;

Character strings and Dates

Character strings and date values are enclosed in single quotation marks.

Character values are case sensitive and date values are format sensitive.

Example:

SELECT employee_id,last_name, job_id, deparment_id FROM employees WHERE last_name='WHALEN';

Comparison Conditions

All relational operators can be used. (=, >, >=, <, <=, <>, !=)

Example:

SELECT last_name, salary
FROM employees
WHERE salary<=3000;

Other comparison conditions

Operator	Meaning
BETWEEN ...AND...	Between two values
IN	Match any of a list of values
LIKE	Match a character pattern
IS NULL	Is a null values

Example:1

```
SELECT last_name, salary
FROM employees
WHERE salary BETWEEN 2500 AND 3500;
```

Example:2

```
SELECT employee_id, last_name, salary , manager_id
FROM employees
WHERE manager_id IN (101, 100,201);
```

Example:3

- Use the LIKE condition to perform wildcard searches of valid string values.
- Two symbols can be used to construct the search string
 - % denotes zero or more characters
 - _ denotes one character

```
SELECT first_name, salary
FROM employees
WHERE first_name LIKE '%s';
```

Example:4

```
SELECT last_name, salary
FROM employees
WHERE last_name LIKE '_o%';
```

Example:5

ESCAPE option-To have an exact match for the actual % and _ characters
To search for the string that contain 'SA_'

```
SELECT employee_id, first_name, salary, job_id
FROM employees
WHERE job_id LIKE '%sa\__%'ESCAPE'\';
```

Test for NULL

- Using IS NULL operator

Example:

```
SELECT employee_id, last_name, salary , manager_id
FROM employees
WHERE manager_id IS NULL;
```

Logical Conditions

All logical operators can be used.(AND,OR,NOT)

Example:1

```
SELECT employee_id, last_name, salary , job_id
FROM employees
WHERE salary >= 10000
AND job_id LIKE '%MAN%';
```

Example:2

```
SELECT employee_id, last_name, salary , job_id
FROM employees
WHERE salary >= 10000
OR job_id LIKE '%MAN%';
```

Example:3

```
SELECT employee_id, last_name, salary , job_id
FROM employees
WHERE job_id NOT IN ('it_prog', 'st_clerk', 'sa_rep');
```

Rules of Precedence

Order Evaluated	Operator
1	Arithmetic
2	Concatenation
3	Comparison
4	IS [NOT] NULL, LIKE, [NOT] IN
5	[NOT] BETWEEN
6	Logical NOT
7	Logical AND
8	Logical OR

Example:1

```
SELECT employee_id, last_name, salary , job_id
FROM employees
WHERE job_id = 'sa_rep'
OR job_id = 'ad_pres'
AND salary > 15000;
```

Example:2

```
SELECT employee_id, last_name, salary , job_id
FROM employees
WHERE (job_id ='sa_rep'
OR job_id='ad_pres')
AND salary>15000;
```

Sorting the rows

Using ORDER BY Clause

ASC-Ascending Order,Default

DESC-Descending order

Example:1

```
SELECT last_name, salary , job_id,department_id,hire_date
FROM employees
ORDER BY hire_date;
```

Example:2

```
SELECT last_name, salary , job_id,department_id,hire_date
FROM employees
ORDER BY hire_date DESC;
```

Example:3

Sorting by column alias

```
SELECT last_name, salary*12 annsal , job_id,department_id,hire_date
FROM employees
ORDER BY annsal;
```

Example:4

Sorting by Multiple columns

```
SELECT last_name, salary , job_id,department_id,hire_date
FROM employees
ORDER BY department_id, salary DESC;
```

Find the Solution for the following:

1. Create a query to display the last name and salary of employees earning more than 12000.

select last_name,salary from MY_EMPLOYEE where salary>12000;

```
+-----+-----+
| last_name | salary |
+-----+-----+
| Drexler   | 15000.00 |
+-----+-----+
1 row in set (0.009 sec)
```

2. Create a query to display the employee last name and department number for employee number 176.

select last_name,job_code from MY_EMPLOYEE where job_code=176;

```
+-----+-----+
| last_name | job_code |
+-----+-----+
| Drexler   | 176      |
+-----+-----+
1 row in set (0.209 sec)
```

3. Create a query to display the last name and salary of employees whose salary is not in the range of 5000 and 12000. (hints: not between)

select last_name, salary from MY_EMPLOYEE where salary not between 5000 and 12000;

```
+-----+-----+
| last_name | salary |
+-----+-----+
| Patel     | 1000.00 |
| Drexler   | 15000.00 |
+-----+-----+
2 rows in set (0.195 sec)
```

4. Display the employee last name, job ID, and start date of employees hired between February 20,1998 and May 1,1998.order the query in ascending order by start date.(hints: between)

select last_name,job_code,hire_date from MY_EMPLOYEE where hire_date between to_date('1998-02-20','YYYY-MM-DD') and to_date('1998-05-01','YYYY-MM-DD');

```

+-----+-----+-----+
| last_name | job_code | hire_date |
+-----+-----+-----+
| Drexler   | 176      | 1998-02-20 |
+-----+-----+-----+
1 row in set (0.014 sec)

```

5. Display the last name and department number of all employees in departments 20 and 50 in alphabetical order by name.(hints: in, orderby)

select last_name,id from MY_EMPLOYEE where job_code in(20,50) order by last_name;

```

+-----+-----+
| last_name | id |
+-----+-----+
| vishnu    | 2 |
+-----+-----+
1 row in set (0.220 sec)

```

6. Display the last name and salary of all employees who earn between 5000 and 12000 and are in departments 20 and 50 in alphabetical order by name. Label the columns EMPLOYEE, MONTHLY SALARY respectively.(hints: between, in)

select last_name as "EMPLOYEE",salary as "MONTHLY SALARY" from MY_EMPLOYEE where salary between 5000 and 12000 and job_code in(20,50) order by last_name;

```

+-----+-----+
| EMPLOYEE | MONTHLY SALARY |
+-----+-----+
| Thanu    | 6000.00        |
| vishnu    | 7000.00        |
+-----+-----+
2 rows in set (0.016 sec)

```

7. Display the last name and hire date of every employee who was hired in 1994.(hints: like)
select last_name,hire_date from MY_EMPLOYEE where hire_date like '1994%';

```
+-----+-----+
| last_name | hire_date |
+-----+-----+
| Drexler   | 1994-02-20 |
+-----+-----+
1 row in set (0.013 sec)
```

8. Display the last name and job title of all employees who do not have a manager.(hints: is null)
select last_name,job_code from MY_EMPLOYEE where job_code is null;

```
+-----+-----+
| last_name | job_code |
+-----+-----+
| Drexler   | NULL    |
+-----+-----+
1 row in set (0.012 sec)
```

9. Display the last name, salary, and commission for all employees who earn commissions. Sort data in descending order of salary and commissions.(hints: is not nul,orderby)
SELECT last_name, salary, commission FROM MY_EMPLOYEE WHERE commission IS NOT NULL ORDER BY commission DESC;

```
+-----+-----+-----+
| last_name | salary | commission |
+-----+-----+-----+
| Thanu     | 6000.00 | 500        |
| Thani     | 8000.00 | 400        |
| Drexler    | 15000.00 | 300        |
| vishnu    | 7000.00 | 200        |
| Patel     | 1000.00 | 100        |
+-----+-----+-----+
5 rows in set (0.010 sec)
```

10. Display the last name of all employees where the third letter of the name is **a**.(hints:like)

select last_name from MY_EMPLOYEE where last_name like '__a%';

```
+-----+
| last_name |
+-----+
| Thani     |
| Thanu     |
+-----+
2 rows in set (0.221 sec)
```

11. Display the last name of all employees who have an a and an e in their last name.(hints: like)

select last_name from MY_EMPLOYEE where last_name like '%a%' and last_name like '%e%';

```
+-----+
| last_name |
+-----+
| Patel     |
+-----+
1 row in set (0.013 sec)
```

12. Display the last name and job and salary for all employees whose job is sales representative or stock clerk and whose salary is not equal to 2500 ,3500 or 7000.(hints:in,not in)

select last_name,job,salary from MY_EMPLOYEE where job in('sales representative','clerk') and salary in(2500,3500,7000);

```
+-----+-----+-----+
| last_name | job           | salary |
+-----+-----+-----+
| Drexler   | sales representative | 3500.00 |
+-----+-----+-----+
1 row in set (0.010 sec)
```

13. Display the last name, salary, and commission for all employees whose commission amount is 20%.(hints:use predicate logic)

select last_name,salary,commission from MY_EMPLOYEE where commission=20;


```

+-----+-----+-----+
| last_name | salary | commission |
+-----+-----+-----+
| Thanu     | 6000.00 |          20 |
+-----+-----+-----+
1 row in set (0.014 sec)

```

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

Pracice Questions

Sorting Rows

1. In the example below, assign the employee_id column the alias of "Number." Complete the SQL statement to order the result set by the column alias.

```
SELECT employee_id, first_name, last_name FROM employees;
```

```
SELECT employee_id AS "Number", first_name, last_name FROM employees ORDER BY  
"Number";
```

2. Create a query that will return all the DJs on Demand CD titles ordered by year with titles in alphabetical order by year.

```
SELECT title, year_purchased FROM d_cds ORDER BY year_purchased, title;
```

3. Order the DJs on Demand songs by descending title. Use the alias "Our Collection" for the song title.

```
SELECT song_title AS "Our Collection" FROM d_songs ORDER BY song_title DESC;
```

4. Write a SQL statement using the ORDER BY clause that could retrieve the information needed.

```
SELECT column1, column2, column3 FROM table_name ORDER BY column1 ASC, column2 DESC;
```

Evaluation Procedure	Marks awarded
Practice Evaluation (5)	
Viva(5)	
Total (10)	
Faculty Signature	

EXERCISE-6

Single Row Functions

Objective

After the completion of this exercise, the students will be able to do the following:

- Describe various types of functions available in SQL.
- Use character, number and date functions in SELECT statement.
- Describe the use of conversion functions.

Single row functions:

Manipulate data items.

Accept arguments and return one value.

Act on each row returned.

Return one result per row.

May modify the data type.

Can be nested.

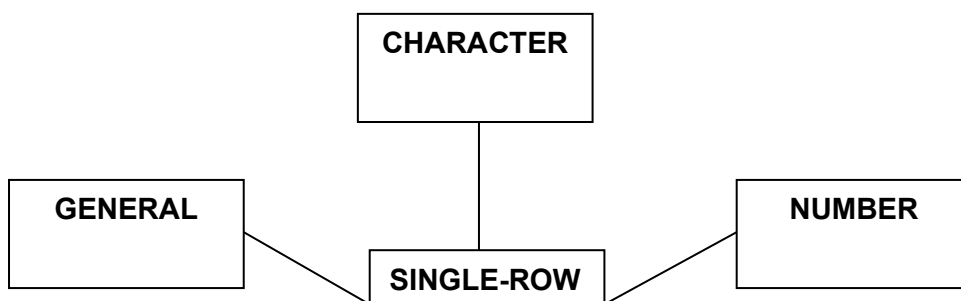
Accept arguments which can be a column or an expression

Syntax

Function_name(arg1,...argn)

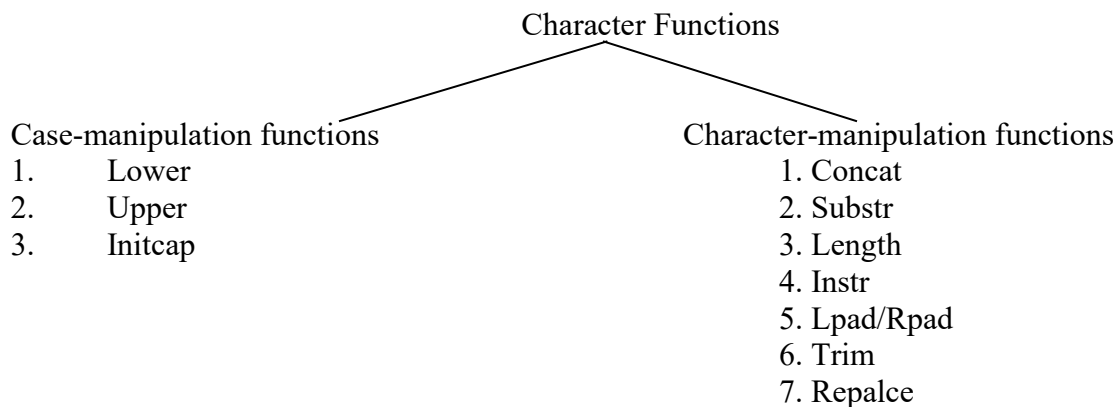
An argument can be one of the following

- ✓ User-supplied constant
- ✓ Variable value
- ✓ Column name
- ✓ Expression



- Character Functions: Accept character input and can return both character and number values.
- Number functions: Accept numeric input and return numeric values.
- Date Functions: Operate on values of the DATE data type.
- Conversion Functions: Convert a value from one type to another.

Character Functions



Function	Purpose
lower(column/expr)	Converts alpha character values to lowercase
upper(column/expr)	Converts alpha character values to uppercase
initcap(column/expr)	Converts alpha character values the to uppercase for the first letter of each word, all other letters in lowercase
concat(column1/expr1, column2/expr2)	Concatenates the first character to the second character
substr(column/expr,m,n)	Returns specified characters from character value starting at character position m, n characters long
length(column/expr)	Returns the number of characters in the expression
instr(column/expr,'string',m,n)	Returns the numeric position of a named string

lpad(column/expr, n, 'string')	Pads the character value right-justified to a total width of n character positions
rpadd(column/expr, 'string', m, n)	Pads the character value left-justified to a total width of n character positions
trim(leading/trailing/both, trim_character FROM trim_source)	Enables you to trim heading or string. trailing or both from a character
replace(text, search_string, replacement_string)	

Example:

lower('SQL Course') □ sql course

upper('SQL Course') □ SQL COURSE

initcap('SQL Course') □ Sql Course

SELECT 'The job id for' || upper(last_name || 'is') || lower(job_id) AS "EMPLOYEE DETAILS"
FROM employees;

SELECT employee_id, last_name, department_id
FROM employees
WHERE LOWER(last_name)='higgins';

Function	Result
CONCAT('hello', 'world')	helloworld
Substr('helloworld', 1, 5)	Hello
Length('helloworld')	10
Instr('helloworld', 'w')	6
Lpad(salary, 10, '*')	*****24000
Rpad(salary, 10, '*')	24000*****
Trim('h' FROM 'helloworld')	elloworld

Command	Query	Output
initcap(char);	<i>select initcap("hello") from dual;</i>	Hello
lower (char); upper (char);	<i>select lower ('HELLO') from dual;</i> <i>select upper ('hello') from dual;</i>	Hello HELLO
ltrim (char,[set]);	<i>select ltrim ('cseit', 'cse') from dual;</i>	IT
rtrim (char,[set]);	<i>select rtrim ('cseit', 'it') from dual;</i>	CSE
replace (char,search string, replace string);	<i>select replace ('jack and jue', 'j', 'bl') from dual;</i>	black and blue

substr (char,m,n);	<i>select substr ('information', 3, 4) from dual;</i>	form

Example:

SELECT employee_id, CONCAT (first_name,last_name) NAME , job_id,LENGTH(last_name),
INSTR(last_name,'a') "contains'a?"
FROM employees WHERE SUBSTR(job_id,4)='ERP';

NUMBER FUNCTIONS

Function	Purpose
round(column/expr, n)	Rounds the value to specified decimal
trunc(column/expr,n)	Truncates value to specified decimal
mod(m,n)	Returns remainder of division

Example

Function	Result
round(45.926,2)	45.93
trunc(45.926,2)	45.92
mod(1600,300)	100

SELECT ROUND(45.923,2), ROUND(45.923,0), ROUND(45.923,-1) FROM dual;

NOTE: Dual is a dummy table you can use to view results from functions and calculations.

SELECT TRUNC(45.923,2), TRUNC(45.923), TRUNC(45.923,-2) FROM dual;

SELECT last_name,salary,MOD(salary,5000) FROM employees WHERE job_id='sa_rep';

Working with Dates

The Oracle database stores dates in an internal numeric format: century, year, month, day, hours, minutes, and seconds.

- The default date display format is DD-MON-RR.
- Enables you to store 21st-century dates in the 20th century by specifying only the last two digits of the year
- Enables you to store 20th-century dates in the 21st century in the same way

Example

SELECT last_name, hire_date FROM employees WHERE hire_date < '01-FEB-88;

Working with Dates

SYSDATE is a function that returns:

- Date
- Time

Example

Display the current date using the DUAL table.

SELECT SYSDATE FROM DUAL;

Arithmetic with Dates

- Add or subtract a number to or from a date for a resultant date value.
- Subtract two dates to find the number of days between those dates.
- Add hours to a date by dividing the number of hours by 24.

Arithmetic with Dates

Because the database stores dates as numbers, you can perform calculations using arithmetic Operators such as addition and subtraction. You can add and subtract number constants as well as dates.

You can perform the following operations:

Operation	Result	Description
date + number	Date	Adds a number of days to a date
date – number	Date	Subtracts a number of days from a date
date – date	Number of days	Subtracts one date from another
date + number/24	Date	Adds a number of hours to a date

Example

```
SELECT last_name, (SYSDATE-hire_date)/7 AS WEEKS
FROM employees
WHERE department_id = 90;
```

Date Functions

Function	Result
MONTHS_BETWEEN	Number of months between two dates
ADD_MONTHS	Add calendar months to date
NEXT_DAY	Next day of the date specified
LAST_DAY	Last day of the month
ROUND	Round date
TRUNC	Truncate date

Date Functions

Date functions operate on Oracle dates. All date functions return a value of DATE data type except MONTHS_BETWEEN, which returns a numeric value.

- MONTHS_BETWEEN(date1, date2)::: Finds the number of months between date1 and date2. The result can be positive or negative. If date1 is later than date2, the result is positive; if date1 is earlier than date2, the result is negative. The noninteger part of the result represents a portion of the month.

- **ADD_MONTHS(date, n):::** Adds n number of calendar months to date. The value of n must be an integer and can be negative.
- **NEXT_DAY(date, 'char'):::** Finds the date of the next specified day of the week ('char') following date. The value of char may be a number representing a day or a character string.
- **LAST_DAY(date):::** Finds the date of the last day of the month that contains date
- **ROUND(date[, 'fmt']:::** Returns date rounded to the unit that is specified by the format model fmt. If the format model fmt is omitted, date is rounded to the nearest day.
- **TRUNC(date[, 'fmt']:::** Returns date with the time portion of the day truncated to the unit that is specified by the format model fmt. If the format model fmt is omitted, date is truncated to the nearest day.

Using Date Functions

Function	Result
MONTHS_BETWEEN ('01-SEP-95 ' , '11-JAN-94 ')	19.6774194
ADD_MONTHS ('11-JAN-94 ' , 6)	'11-JUL-94 '
NEXT_DAY ('01-SEP-95 ' , 'FRIDAY')	'08-SEP-95 '
LAST_DAY ('01-FEB-95 ')	'28-FEB-95 '

Example

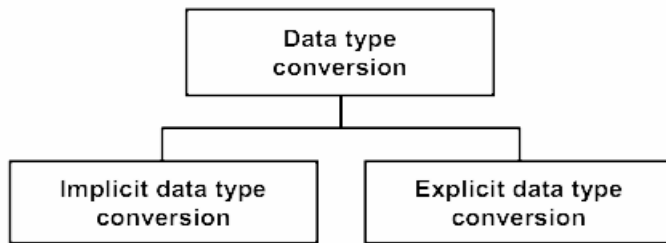
Display the employee number, hire date, number of months employed, sixmonth review date, first Friday after hire date, and last day of the hire month for all employees who have been employed for fewer than 70 months.

```
SELECT employee_id, hire_date, MONTHS_BETWEEN (SYSDATE, hire_date)
TENURE, ADD_MONTHS (hire_date, 6) REVIEW, NEXT_DAY (hire_date, 'FRIDAY'),
LAST_DAY(hire_date)
FROM employees
WHERE MONTHS_BETWEEN (SYSDATE, hire_date) < 70;
```

Conversion Functions

This covers the following topics:

- Writing a query that displays the current date
- Creating queries that require the use of numeric, character, and date functions
- Performing calculations of years and months of service for an employee



Implicit Data Type Conversion

For assignments, the Oracle server can automatically convert the following:

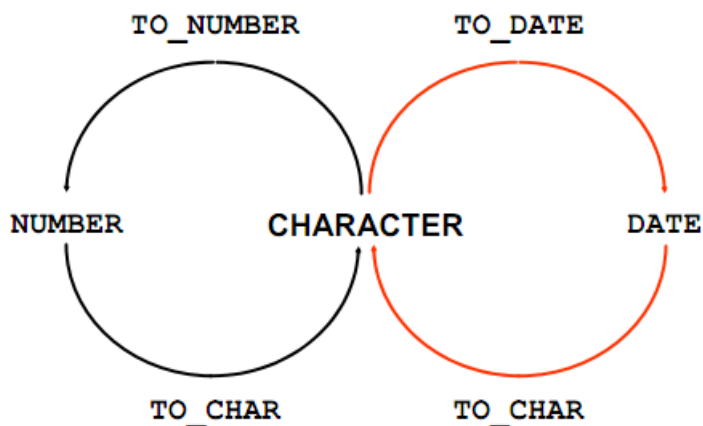
From	To
VARCHAR2 or CHAR	NUMBER
VARCHAR2 or CHAR	DATE
NUMBER	VARCHAR2
DATE	VARCHAR2

For example, the expression `hire_date > '01-JAN-90'` results in the implicit conversion from the string '01-JAN-90' to a date.

For expression evaluation, the Oracle Server can automatically convert the following:

From	To
VARCHAR2 or CHAR	NUMBER
VARCHAR2 or CHAR	DATE

Explicit Data Type Conversion



SQL provides three functions to convert a value from one data type to another:

Example:

Using the TO_CHAR Function with Dates

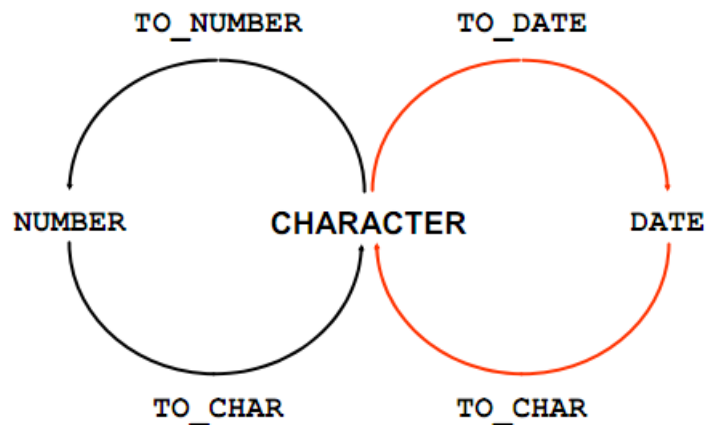
TO_CHAR(date, 'format_model')

The format model:

- Must be enclosed by single quotation marks
- Is case-sensitive
- Can include any valid date format element
- Has an fm element to remove padded blanks or suppress leading zeros
- Is separated from the date value by a comma

```
SELECT employee_id, TO_CHAR(hire_date, 'MM/YY') Month_Hired  
FROM employees WHERE last_name = 'Higgins';
```

Elements of the Date Format Model



Sample Format Elements of Valid Date

Element	Description
SCC or CC	Century; server prefixes B.C. date with -
Years in dates YYYY or SYYYY	Year; server prefixes B.C. date with -
YYY or YY or Y	Last three, two, or one digits of year
Y,YYY	Year with comma in this position
IYYY, IYY, IY, I	Four-, three-, two-, or one-digit year based on the ISO standard
SYEAR or YEAR	Year spelled out; server prefixes B.C. date with -
BC or AD	Indicates B.C. or A.D. year
B.C. or A.D.	Indicates B.C. or A.D. year using periods
Q	Quarter of year
MM	Month: two-digit value
MONTH	Name of month padded with blanks to length of nine characters
MON	Name of month, three-letter abbreviation
RM	Roman numeral month
WW or W	Week of year or month
DDD or DD or D	Day of year, month, or week
DAY	Name of day padded with blanks to a length of nine characters
DY	Name of day; three-letter abbreviation
J	Julian day; the number of days since December 31, 4713 B.C.

Date Format Elements: Time Formats

Use the formats that are listed in the following tables to display time information and literals and to change numerals to spelled numbers.

Element	Description
AM or PM	Meridian indicator
A.M. or P.M.	Meridian indicator with periods
HH or HH12 or HH24	Hour of day, or hour (1–12), or hour (0–23)
MI	Minute (0–59)
SS	Second (0–59)
SSSSS	Seconds past midnight (0–86399)

Other Formats

Element	Description
/ . ,	Punctuation is reproduced in the result.
"of the"	Quoted string is reproduced in the result.

Specifying Suffixes to Influence Number Display

Element	Description
TH	Ordinal number (for example, DDTH for 4TH)
SP	Spelled-out number (for example, DDSP for FOUR)
SPTH or THSP	Spelled-out ordinal numbers (for example, DDSPTH for FOURTH)

Example

```
SELECT last_name,  
       TO_CHAR(hire_date, 'fmDD Month YYYY') AS HIREDATE  
FROM   employees;
```

Modify example to display the dates in a format that appears as "Seventeenth of June 1987 12:00:00 AM."

```
SELECT last_name,  
       TO_CHAR(hire_date, 'fmDdspt "of" Month YYYY fmHH:MI:SS AM') HIREDATE  
FROM   employees;
```

Using the TO_CHAR Function with Numbers

TO_CHAR(number, 'format_model')

These are some of the format elements that you can use with the TO_CHAR function to display a number value as a character:

Element	Result
9	Represents a number
0	Forces a zero to be displayed
\$	Places a floating dollar sign
L	Uses the floating local currency symbol
.	Prints a decimal point
,	Prints a comma as thousands indicator

Number Format Elements

If you are converting a number to the character data type, you can use the following format elements:

Element	Description	Example	Result
9	Numeric position (number of 9s determine display width)	999999	1234
0	Display leading zeros	099999	001234
\$	Floating dollar sign	\$999999	\$1234
L	Floating local currency symbol	L999999	FF1234
D	Returns in the specified position the decimal character. The default is a period (.).	99D99	99.99
.	Decimal point in position specified	999999.99	1234.00
G	Returns the group separator in the specified position. You can specify multiple group separators in a number format model.	9,999	9G999
,	Comma in position specified	999,999	1,234
MI	Minus signs to right (negative values)	999999MI	1234-
PR	Parenthesize negative numbers	999999PR	<1234>
EEEE	Scientific notation (format must specify four Es)	99.999EEEE	1.234E+03
U	Returns in the specified position the "Euro" (or other) dual currency	U9999	€1234
V	Multiply by 10 <i>n</i> times (<i>n</i> = number of 9s after V)	9999V99	123400
S	Returns the negative or positive value	S9999	-1234 or +1234
B	Display zero values as blank, not 0	B9999.99	1234.00

```
SELECT TO_CHAR(salary, '$99,999.00') SALARY
FROM employees
WHERE last_name = 'Ernst';
```

Using the TO_NUMBER and TO_DATE Functions

- Convert a character string to a number format using the TO_NUMBER function:
TO_NUMBER(char[, 'format_model']
- Convert a character string to a date format using the TO_DATE function:
TO_DATE(char[, 'format_model']
- These functions have an fx modifier. This modifier specifies the exact matching for the character argument and date format model of a TO_DATE function.

The fx modifier specifies exact matching for the character argument and date format model of a TO_DATE function:

- Punctuation and quoted text in the character argument must exactly match (except for case) the corresponding parts of the format model.
- The character argument cannot have extra blanks. Without fx, Oracle ignores extra blanks.

- Numeric data in the character argument must have the same number of digits as the corresponding element in the format model. Without fx, numbers in the character argument can omit leading zeros.

```
SELECT last_name, hire_date
FROM employees
WHERE hire_date = TO_DATE('May 24, 1999', 'fxMonth DD, YYYY');
```

Find the Solution for the following:

1. Write a query to display the current date. Label the column Date.
select sysdate as "Date" from dual;

```
Date
-----
24-AUG-25
```

2. The HR department needs a report to display the employee number, last name, salary, and increased by 15.5% (expressed as a whole number) for each employee. Label the column New Salary.

select id,last_name,salary,round(salary*1.155,2) as "New salary" from my_employee;

ID	LAST_NAME	SALARY	New salary
1	Patel	895	1033.73
2	Dancs	860	993.3
3	Biri	1100	1270.5
4	Newman	750	866.25
5	Ropebur	1550	1790.25

3. Modify your query lab_03_02.sql to add a column that subtracts the old salary from the new salary. Label the column Increase.

select id,last_name,salary,round(salary*1.555,2) as "NEW SALARY",round((salary*1.555)-salary,2) as "Increase" from my_employee;

ID	LAST_NAME	SALARY	NEW SALARY	Increase
1	Patel	895	1391.73	496.73
2	Dancs	860	1337.3	477.3
3	Biri	1100	1710.5	610.5
4	Newman	750	1166.25	416.25
5	Ropebur	1550	2410.25	860.25

4. Write a query that displays the last name (with the first letter uppercase and all other letters lowercase) and the length of the last name for all employees whose name starts with the letters J, A, or M. Give each column an appropriate label. Sort the results by the employees' last names.

```
select initcap(last_name) as "Name",length(last_name) as "Length" from my_employee
where last_name like 'J%' or last_name like 'A%' or last_name like 'M%' order by
last_name;
```

Name	Length
Antony	6

5. Rewrite the query so that the user is prompted to enter a letter that starts the last name. For example, if the user enters H when prompted for a letter, then the output should show all employees whose last name starts with the letter H.

```
select initcap(last_name) as "Name",length(last_name) as "Length" from my_employee
where last_name like '&letter%' order by last_name;
```

Name	Length
Antony	6

6. The HR department wants to find the length of employment for each employee. For each employee, display the last name and calculate the number of months between today and the date on which the employee was hired. Label the column MONTHS_WORKED. Order your results by the number of months employed. Round the number of months up to the closest whole number.

```
select last_name,ceil(months_between(sysdate,hire_date)) as "Months worked" from
my_employee order by "Months worked" desc;
```

LAST_NAME	Months worked
Biri	322
Antony	311
Ropebur	310
Patel	302
Newman	296
Dancs	275

Note: Your results will differ.

7. Create a report that produces the following for each employee:

<employee last name> earns <salary> monthly but wants <3 times salary>. Label the column Dream Salaries.

select last_name||' earns '||salary||' monthly but wants '||(salary*3) as " Dream salaries " from my_employee;

```
Dream salaries
-----
Antony earns 7000.5 monthly but wants 21001.5
Patel earns 895 monthly but wants 2685
Dancs earns 860 monthly but wants 2580
Biri earns 1100 monthly but wants 3300
Newman earns 750 monthly but wants 2250
Ropebur earns 1550 monthly but wants 4650
```

8. Create a query to display the last name and salary for all employees. Format the salary to be 15 characters long, left-padded with the \$ symbol. Label the column SALARY.

select last_name,lpad(to_char(salary),15,'\$') as "Salary" from My_employee;


```

LAST_NAME
-----
Salary
-----
Antony
$$$$$$$$$7000.5

Patel
$$$$$$$$$$$$$895

Dancs
$$$$$$$$$$$$$860

```

9. Display each employee's last name, hire date, and salary review date, which is the first Monday after six months of service. Label the column REVIEW. Format the dates to appear in the format similar to "Monday, the Thirty-First of July, 2000."

```

select last_name,to_char(hire_date,'Day,"the" DDSPTH "of" Month,YYYY') as
"hire_date",to_char(next_day(add_months(hire_date,6),'Monday'),'Day,"the" DDSPTH
"of" Month,YYYY') as "Review" from My_employee;

```

```

LAST_NAME
-----
hire_date
-----
Review
-----
Antony
Sunday      ,the TENTH of October  ,1999
Monday      ,the SEVENTEENTH of April  ,2000

Patel
Friday      ,the SEVENTH of July  ,2000
Monday      ,the EIGHTH of January ,2001

```

10. Display the last name, hire date, and day of the week on which the employee started. Label the column DAY. Order the results by the day of the week, starting with Monday.

```

select last_name,hire_Date,to_char(hire_date,'Day') as "Day" from My_employee order by
to_char(hire_date,'D');

```

LAST_NAME	HIRE_DATE	Day
Antony	10-OCT-99	Sunday
Newman	01-JAN-01	Monday
Ropebur	27-OCT-99	Wednesday
Biri	30-OCT-98	Friday
Patel	07-JUL-00	Friday
Dancs	27-SEP-02	Friday

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

Practice Questions

Introduction to Functions

1. For each task, choose whether a single-row or multiple row function would be most appropriate:
 - a. Showing all of the email addresses in upper case letters – single row function
 - b. Determining the average salary for the employees in the sales department – multiple row function

- c. Showing hire dates with the month spelled out (*September 1, 2004*)- *Single row function*
- d. Finding out the employees in each department that had the most seniority (the earliest hire date) – multiple row function
- e. Displaying the employees' salaries rounded to the hundreds place – single row function
- f. Substituting zeros for null values when displaying employee commissions. – single row function

2. The most common multiple-row functions are: AVG, COUNT, MAX, MIN, and SUM. Give your own definition for each of these functions.

Function	Definition
AVG(column)	Returns the average of all values in the column.
COUNT(column)	Returns the number of non-null rows in the column.
MAX(column)	Returns the maximum value in the column.
MIN(column)	Returns the minimum value in the column.
SUM(column)	Returns the total sum of all values in the column.

3. Test your definitions by substituting each of the multiple-row functions in this query.

SELECT FUNCTION(salary)

FROM employees

Write out each query and its results.

a. Average salary

SELECT AVG(salary) AS "Average Salary" FROM employees;

b. Count of employees

SELECT COUNT(employee_id) AS "Employee Count" FROM employees;

c. Maximum salary

SELECT MAX(salary) AS "Maximum Salary" FROM employees;

d. Minimum salary

SELECT MIN(salary) AS "Minimum Salary" FROM employees;

e. Sum of salaries

SELECT SUM(salary) AS "Total Salaries" FROM employees;

Case and Character Manipulation

1. Using the three separate words “Oracle,” “Internet,” and “Academy,” use one command to produce the following output:
The Best Class Oracle Internet Academy

```
SELECT 'The Best Class ' || 'Oracle' || ' ' || 'Internet' || ' ' || 'Academy' AS result
FROM dual;
```

2. Use the string “Oracle Internet Academy” to produce the following output:
The Net net

```
SELECT 'The ' || SUBSTR('Oracle Internet Academy', 8, 3) || ' ' || LOWER(SUBSTR('Oracle Internet
Academy', 8, 3)) AS result
FROM dual;
```

3. What is the length of the string “Oracle Internet Academy”?

```
SELECT LENGTH('Oracle Internet Academy') AS string_length FROM dual;
```

4. What’s the position of “I” in “Oracle Internet Academy”?

```
SELECT INSTR('Oracle Internet Academy', 'I') AS position
FROM dual;
```

5. Starting with the string “Oracle Internet Academy”, pad the string to create
****Oracle****Internet****Academy****

```
SELECT '*****' || REPLACE('Oracle Internet Academy', ' ', '*****') || '*****' AS result FROM dual;
```

Number Functions

1. Display Oracle database employee last_name and salary for employee_ids between 100 and 102. Include a third column that divides each salary by 1.55 and rounds the result to two decimal places.

```
SELECT last_name,  
       salary,  
       ROUND(salary / 1.55, 2) AS adjusted_salary  
FROM employees  
WHERE employee_id BETWEEN 100 AND 102;
```

2. Display employee last_name and salary for those employees who work in department 80. Give each of them a raise of 5.333% and truncate the result to two decimal places.

```
SELECT last_name,  
       salary,  
       TRUNC(salary * 1.05333, 2) AS new_salary  
FROM employees  
WHERE department_id = 80;
```

3. Use a MOD number function to determine whether 38873 is an even number or an odd number.

```
SELECT MOD(38873, 2) AS remainder FROM dual;
```

4. Use the DUAL table to process the following numbers:

845.553 - round to one decimal place

30695.348 - round to two decimal places

30695.348 - round to -2 decimal Places

2.3454 - truncate the 454 from the decimal place

```
SELECT
  ROUND(845.553, 1) AS round_1_decimal,
  ROUND(30695.348, 2) AS
round_2_decimals,
  ROUND(30695.348, -2) AS
round_negative_2,
  TRUNC(2.3454, 2) AS trunc_2_decimals
FROM dual;
```

5. Divide each employee's salary by 3. Display only those employees' last names and salaries who earn a salary that is a multiple of 3.

```
SELECT last_name, salary FROM employees WHERE MOD(salary, 3) = 0;
```

6. Divide 34 by 8. Show only the remainder of the division. Name the output as EXAMPLE.

```
SELECT MOD(34, 8) AS EXAMPLE FROM dual
```

7. How would you like your paycheck – rounded or truncated? What if your paycheck was calculated to be \$565.784 for the week, but you noticed that it was issued for \$565.78. The loss of .004 cent would probably make very little difference to you. However, what if this was done to a thousand people, a 100,000 people, or a million people! Would it make a difference then? How much difference?

If the paycheck was **\$565.784** but it was **truncated** to **\$565.78**, you lost **\$0.004**.

Rounding is fairer than truncating because truncation always drops the remaining value, which, when applied to thousands or millions of transactions, results in a significant total monetary loss.

Evaluation Procedure	Marks awarded
Practice Evaluation (5)	
Viva(5)	
Total (10)	
Faculty Signature	

EXERCISE-7

Displaying data from multiple tables

Objective

After the completion of this exercise, the students will be able to do the following:

- Write SELECT statements to access data from more than one table using equality and nonequality joins
- View data that generally does not meet a join condition by using outer joins
- Join a table to itself by using a self join

Sometimes you need to use data from more than one table.

Cartesian Products

- A Cartesian product is formed when:
 - A join condition is omitted
 - A join condition is invalid
 - All rows in the first table are joined to all rows in the second table
- To avoid a Cartesian product, always include a valid join condition in a WHERE clause.

A Cartesian product tends to generate a large number of rows, and the result is rarely useful. You should always include a valid join condition in a WHERE clause, unless you have a specific need to combine all rows from all tables.

Cartesian products are useful for some tests when you need to generate a large number of rows to simulate a reasonable amount of data.

Example:

To displays employee last name and department name from the EMPLOYEES and DEPARTMENTS tables.

```
SELECT last_name, department_name dept_name
FROM employees, departments;
```

Types of Joins

- Equijoin
- Non-equijoin
- Outer join
- Self join
- Cross joins
- Natural joins
- Using clause
- Full or two sided outer joins
- Arbitrary join conditions for outer joins

Joining Tables Using Oracle Syntax

```
SELECT table1.column, table2.column
FROM table1, table2
WHERE table1.column1 = table2.column2;
```

Write the join condition in the WHERE clause.

- Prefix the column name with the table name when the same column name appears in more than one table.

Guidelines

- When writing a SELECT statement that joins tables, precede the column name with the table name for clarity and to enhance database access.
- If the same column name appears in more than one table, the column name must be prefixed with the table name.
- To join n tables together, you need a minimum of n-1 join conditions. For example, to join four tables, a minimum of three joins is required. This rule may not apply if your table has a concatenated primary key, in which case more than one column is required to uniquely identify each row

What is an Equijoin?

To determine an employee's department name, you compare the value in the DEPARTMENT_ID column in the EMPLOYEES table with the DEPARTMENT_ID values in the DEPARTMENTS table.

The relationship between the EMPLOYEES and DEPARTMENTS tables is an equijoin—that is, values

in the DEPARTMENT_ID column on both tables must be equal. Frequently, this type of join involves

primary and foreign key complements.

Note: Equijoins are also called simple joins or inner joins

```
SELECT employees.employee_id, employees.last_name, employees.department_id,
       departments.department_id, departments.location_id
```

```
FROM   employees, departments
```

```
WHERE  employees.department_id = departments.department_id;
```

Additional Search Conditions

Using the AND Operator

Example:

To display employee Matos's department number and department name, you need an additional condition in the WHERE clause.

```
SELECT last_name, employees.department_id,
       department_name
```

```
FROM   employees, departments
```

```
WHERE  employees.department_id = departments.department_id AND last_name = 'Matos';
```

Qualifying Ambiguous

Column Names

- Use table prefixes to qualify column names that are in multiple tables.
- Improve performance by using table prefixes.
- Distinguish columns that have identical names but reside in different tables by using column aliases.

Using Table Aliases

- Simplify queries by using table aliases.
- Improve performance by using table prefixes

Example:

```
SELECT e.employee_id, e.last_name, e.department_id,
d.department_id, d.location_id
FROM employees e, departments d
WHERE e.department_id = d.department_id;
```

Joining More than Two Tables

To join n tables together, you need a minimum of n-1 join conditions. For example, to join three tables, a minimum of two joins is required.

Example:

To display the last name, the department name, and the city for each employee, you have to join the EMPLOYEES, DEPARTMENTS, and LOCATIONS tables.

```
SELECT e.last_name, d.department_name, l.city
FROM employees e, departments d, locations l
WHERE e.department_id = d.department_id
AND d.location_id = l.location_id;
```

Non-Equijoins

A non-equijoin is a join condition containing something other than an equality operator. The relationship between the EMPLOYEES table and the JOB_GRADES table has an example of a non-equijoin. A relationship between the two tables is that the SALARY column in the EMPLOYEES table must be between the values in the LOWEST_SALARY and HIGHEST_SALARY columns of the JOB_GRADES table. The relationship is obtained using an operator other than equals (=).

Example:

```
SELECT e.last_name, e.salary, j.grade_level
FROM employees e, job_grades j
WHERE e.salary
BETWEEN j.lowest_sal AND j.highest_sal;
```

Outer Joins

Syntax

- You use an outer join to also see rows that do not meet the join condition.
- The Outer join operator is the plus sign (+).

```
SELECT table1.column, table2.column
FROM table1, table2
WHERE table1.column(+) = table2.column;
SELECT table1.column, table2.column
FROM table1, table2
WHERE table1.column = table2.column(+);
```

The missing rows can be returned if an outer join operator is used in the join condition. The operator is a plus sign enclosed in parentheses (+), and it is placed on the “side” of the join that is deficient in information. This operator has the effect of creating one or more null rows, to which one or more

rows from the nondeficient table can be joined.

Example:

```
SELECT e.last_name, e.department_id, d.department_name
FROM   employees e, departments d
WHERE  e.department_id(+) = d.department_id ;
```

Outer Join Restrictions

- The outer join operator can appear on only one side of the expression—the side that has information missing. It returns those rows from one table that have no direct match in the other table.
- A condition involving an outer join cannot use the IN operator or be linked to another condition by the OR operator

Self Join

Sometimes you need to join a table to itself.

Example:

To find the name of each employee’s manager, you need to join the EMPLOYEES table to itself, or perform a self join.

```
SELECT worker.last_name || ' works for '
|| manager.last_name
FROM   employees worker, employees manager
WHERE  worker.manager_id = manager.employee_id ;
```

Use a join to query data from more than one table.

```
SELECT table1.column, table2.column
FROM table1
[CROSS JOIN table2] |
[NATURAL JOIN table2] |
[JOIN table2 USING (column_name)] |
[JOIN table2
ON(table1.column_name = table2.column_name)] |
[LEFT|RIGHT|FULL OUTER JOIN table2
ON (table1.column_name = table2.column_name)];
```

In the syntax:

table1.column Denotes the table and column from which data is retrieved

CROSS JOIN Returns a Cartesian product from the two tables

NATURAL JOIN Joins two tables based on the same column name

JOIN table USING column_name Performs an equijoin based on the column name

JOIN table ON table1.column_name Performs an equijoin based on the condition in the ON clause
= table2.column_name

LEFT/RIGHT/FULL OUTER

Creating Cross Joins

- The CROSS JOIN clause produces the crossproduct of two tables.
- This is the same as a Cartesian product between the two tables.

Example:

```
SELECT last_name, department_name
FROM employees
CROSS JOIN departments ;
SELECT last_name, department_name
FROM employees, departments;
```

Creating Natural Joins

- The NATURAL JOIN clause is based on all columns in the two tables that have the same name.
- It selects rows from the two tables that have equal values in all matched columns.
- If the columns having the same names have different data types, an error is returned.

Example:

```
SELECT department_id, department_name,
location_id, city
FROM departments
NATURAL JOIN locations ;
```

LOCATIONS table is joined to the DEPARTMENT table by the LOCATION_ID column, which is the only column of the same name in both tables. If other common columns were present, the join would have used them all.

Example:

```
SELECT department_id, department_name,
location_id, city
FROM departments
NATURAL JOIN locations
WHERE department_id IN (20, 50);
```

Creating Joins with the USING Clause

- If several columns have the same names but the data types do not match, the NATURAL JOIN clause can be modified with the USING clause to specify the columns that should be used for an

equijoin.

- Use the USING clause to match only one column when more than one column matches.
- Do not use a table name or alias in the referenced columns.
- The NATURAL JOIN and USING clauses are mutually exclusive.

Example:

```
SELECT l.city, d.department_name
FROM   locations l JOIN departments d USING (location_id)
WHERE  location_id = 1400;
EXAMPLE:
```

```
SELECT e.employee_id, e.last_name, d.location_id
FROM   employees e JOIN departments d
USING (department_id);
```

Creating Joins with the ON Clause

- The join condition for the natural join is basically an equijoin of all columns with the same name.
- To specify arbitrary conditions or specify columns to join, the ON clause is used.
- The join condition is separated from other searchconditions.
- The ON clause makes code easy to understand.

Example:

```
SELECT e.employee_id, e.last_name, e.department_id,
d.department_id, d.location_id
FROM   employees e JOIN departments d
ON     (e.department_id = d.department_id);
EXAMPLE:
```

```
SELECT e.last_name emp, m.last_name mgr
FROM   employees e JOIN employees m
ON     (e.manager_id = m.employee_id);
INNER Versus OUTER Joins
```

- A join between two tables that returns the results of the inner join as well as unmatched rows left (or right) tables is a left (or right) outer join.
- A join between two tables that returns the results of an inner join as well as the results of a left and right join is a full outer join.

LEFT OUTER JOIN

Example:

```
SELECT e.last_name, e.department_id, d.department_name
```

```
FROM employees e
LEFT OUTER JOIN departments d
ON (e.department_id = d.department_id) ;
```

Example of LEFT OUTER JOIN

This query retrieves all rows in the EMPLOYEES table, which is the left table even if there is no match in the DEPARTMENTS table.

This query was completed in earlier releases as follows:

```
SELECT e.last_name, e.department_id, d.department_name
FROM employees e, departments d
WHERE d.department_id (+) = e.department_id;
```

RIGHT OUTER JOIN

Example:

```
SELECT e.last_name, e.department_id, d.department_name
FROM employees e
RIGHT OUTER JOIN departments d
ON (e.department_id = d.department_id) ;
```

This query retrieves all rows in the DEPARTMENTS table, which is the right table even if there is no match in the EMPLOYEES table.

This query was completed in earlier releases as follows:

```
SELECT e.last_name, e.department_id, d.department_name
FROM employees e, departments d
WHERE d.department_id = e.department_id (+);
```

FULL OUTER JOIN

Example:

```
SELECT e.last_name, e.department_id, d.department_name
FROM employees e
FULL OUTER JOIN departments d
ON (e.department_id = d.department_id) ;
```

This query retrieves all rows in the EMPLOYEES table, even if there is no match in the DEPARTMENTS table. It also retrieves all rows in the DEPARTMENTS table, even if there is no match in the EMPLOYEES table.

Find the Solution for the following:

1. Write a query to display the last name, department number, and department name for all employees.

select last_name,id,dept from my_employee;

LAST_NAME	ID	DEPT
Antony	6	cse
Patel	1	cse
Dancs	2	cse
Biri	3	cse
Newman	4	cse
Ropebur	5	cse

2. Create a unique listing of all jobs that are in department 80. Include the location of the department in the output.

select distinct e.job_id,d.department_name,l.city as location from employees e join department d on e.department_id = d.department_id join locations l on d.location_id=l.location_id where e.department_id =80;

JOB_ID	DEPARTMENT_NAME	LOCATION
SA_MAN	Sales	Oxford
SA_REP	Sales	Oxford

3. Write a query to display the employee last name, department name, location ID, and city of all employees who earn a commission

select e.last_name,d.department_name,l.location_id,l.city from employees e join departments d on e.department_id=d.department_id join locations l on d.location_id =l.location_id where e.commission_pct is not null;

LAST_NAME	DEPARTMENT_NAME	LOCATION_ID	CITY
Russell	Sales	2500	Oxford
Partners	Sales	2500	Oxford
Errazuriz	Sales	2500	Oxford
Cambrault	Sales	2500	Oxford
Zlotkey	Sales	2500	Oxford

2. Display the employee last name and department name for all employees who have an a(lowercase) in their last names. P

SELECT e.last_name, d.department_name FROM employees e JOIN departments d ON e.department_id = d.department_id WHERE e.last_name LIKE '%a%';

LAST_NAME	DEPARTMENT_NAME
De Haan	Executive
Hartstein	Marketing
Raphaely	Purchasing
Baida	Purchasing
Tobias	Purchasing

1. Write a query to display the last name, job, department number, and department name for all employees who work in Toronto.

select e.last_name,e.job_id,e.department_name from employees e join departments d on e.department_id=d.department_id join locations l on d.location_id = l.location_id where l.city='toronto';

LAST_NAME	JOB_ID	DEPARTMENT_NAME
Fay	MK_REP	Marketing
Hartstein	MK_MAN	Marketing

6. Display the employee last name and employee number along with their manager's last name and manager number. Label the columns Employee, Emp#, Manager, and Mgr#, Respectively


```
select e.last_name as "Employee",e.employee_id as "EMP#",m.last_name as
"Manager",m.employee_id as "Mgr#" from employees e join employees m on
e.manager_id=m.employee_id;
```

Employee	EMP#	Manager	Mgr#
-----	----	-----	----
Whalen	200	King	100
Hartstein	201	King	100
Fay	202	Hartstein	201
Higgins	205	Kochhar	101
Gietz	206	Higgins	205

2. Modify lab4_6.sql to display all employees including King, who has no manager. Order the results by the employee number.

```
select e.last_name as "Employee",e.employee_id as "Emp#",m.last_name as "Manager" ,
m.employee_id as "Mgr#",from employees e left join employees m on e.manager_id
=m.employee id order by e.employee id;
```

Employee	EMP#	Manager	MGR#
-----	----	-----	----
Kochhar	101	King	100
De Haan	102	King	100
Hunold	103	De Haan	102
Ernst	104	Hunold	103
Austin	105	Hunold	103

8. Create a query that displays employee last names, department numbers, and all the employees who work in the same department as a given employee. Give each column an appropriate label

```
SELECT e.employee_id AS emp_id, e.last_name AS Employee, e.department_id AS Dept_no,
c.last_name AS Colleague FROM employees e JOIN employees c
ON e.department_id = c.department_id AND e.employee_id < > c.employee_id ORDER BY
e.department_id, e.last_name;
```

EMP_ID	Employee	DEPT_NO	Colleague
103	Hunold	60	Ernst
103	Hunold	60	Austin
103	Hunold	60	Pataballa
103	Hunold	60	Lorentz
104	Ernst	60	Hunold
104	Ernst	60	Austin
104	Ernst	60	Pataballa
104	Ernst	60	Lorentz

9. Show the structure of the JOB_GRADES table. Create a query that displays the name, job, department name, salary, and grade for all employees

DESC job_grades;

**SELECT e.last_name AS "Employee",e.job_id AS "Job",d.department_name AS "Dept",
e.salary AS "Salary",j.grade_level AS "Grade" FROM employees e JOIN departments d
ON e.department_id = d.department_id JOIN job_grades j ON e.salary BETWEEN
j.lowest_sal AND j.highest_sal ORDER BY e.last_name;**

Name	Null?	Type
GRADE_LEVEL		VARCHAR2(3)
LOWEST_SAL		NUMBER
HIGHEST_SAL		NUMBER

Employee	Job	Dept	Salary	Grade
Abel	SA_REP	Sales	11000	E
Ande	SA_REP	Sales	6400	C
Atkinson	SA_REP	Sales	4800	B
Baida	ST_CLERK	Purchasing	2900	A
Banda	ST_CLERK	Purchasing	6200	C
Bell	SA_REP	Sales	9000	D
Bloom	SA_REP	Sales	10000	D

10. Create a query to display the name and hire date of any employee hired after employee Davies.
select last_name,hire_date from employees where hire_date >(select hire_date from employees where last_name ="Davis");

LAST_NAME	HIRE_DATE
Matos	07-JUN-05
Taylor	24-MAR-06
Grant	13-JAN-07
King	09-JUN-07
Smith	23-OCT-07

11. Display the names and hire dates for all employees who were hired before their managers, along with their manager's names and hire dates. Label the columns Employee, Emp Hired, Manager, and Mgr Hired, respectively.

SELECT e.last_name AS "Emp", e.hire_date AS "Emp hired", m.last_name AS "Manager",

```
m.hire_date AS "Manager hired" FROM employees e JOIN employees m ON e.manager_id =
m.employee_id WHERE e.hire_date < m.hire_date ORDER BY e.hire_date;
```

Emp	Emp hired	Manager	Manager hired
Abel	11-MAY-04	Hartstein	17-FEB-04
Baida	24-DEC-05	Raphaely	07-DEC-04
Kochhar	21-SEP-05	King	17-JUN-03
Chen	28-SEP-05	Raphaely	07-DEC-04
Faviet	16-AUG-05	Raphaely	07-DEC-04

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

EXERCISE-8

Aggregating Data Using Group Functions

Objectives

After the completion of this exercise, the students be will be able to do the following:

- Identify the available group functions
- Describe the use of group functions
- Group data by using the GROUP BY clause
- Include or exclude grouped rows by using the HAVING clause

What Are Group Functions?

Group functions operate on sets of rows to give one result per group

Types of Group Functions

- AVG
- COUNT
- MAX
- MIN
- STDDEV
- SUM
- VARIANCE

Each of the functions accepts an argument. The following table identifies the options that you can use in the syntax:

Function	Description
AVG ([DISTINCT <u>ALL</u>] <i>n</i>)	Average value of <i>n</i> , ignoring null values
COUNT ({ * [DISTINCT <u>ALL</u>] <i>expr</i> })	Number of rows, where <i>expr</i> evaluates to something other than null (count all selected rows using *, including duplicates and rows with nulls)
MAX ([DISTINCT <u>ALL</u>] <i>expr</i>)	Maximum value of <i>expr</i> , ignoring null values
MIN ([DISTINCT <u>ALL</u>] <i>expr</i>)	Minimum value of <i>expr</i> , ignoring null values
STDDEV ([DISTINCT <u>ALL</u>] <i>x</i>)	Standard deviation of <i>n</i> , ignoring null values
SUM ([DISTINCT <u>ALL</u>] <i>n</i>)	Sum values of <i>n</i> , ignoring null values
VARIANCE ([DISTINCT <u>ALL</u>] <i>x</i>)	Variance of <i>n</i> , ignoring null values

Group Functions: Syntax

```
SELECT [column,] group_function(column), ...  
FROM table  
[WHERE condition]  
[GROUP BY column]  
[ORDER BY column];
```

Guidelines for Using Group Functions

- DISTINCT makes the function consider only nonduplicate values; ALL makes it consider every value, including duplicates. The default is ALL and therefore does not need to be specified.
- The data types for the functions with an expr argument may be CHAR, VARCHAR2, NUMBER, or DATE.
- All group functions ignore null values.

Using the AVG and SUM Functions

You can use AVG and SUM for numeric data.

```
SELECT AVG(salary), MAX(salary),  
MIN(salary), SUM(salary)  
FROM employees  
WHERE job_id LIKE '%REP%';
```

Using the MIN and MAX Functions

You can use MIN and MAX for numeric, character, and date data types.

```
SELECT MIN(hire_date), MAX(hire_date)  
FROM employees;
```

You can use the MAX and MIN functions for numeric, character, and date data types. example displays the most junior and most senior employees.

The following example displays the employee last name that is first and the employee last name that is last in an alphabetized list of all employees:

```
SELECT MIN(last_name), MAX(last_name)  
FROM employees;
```

Note: The AVG, SUM, VARIANCE, and STDDEV functions can be used only with numeric data types. MAX and MIN cannot be used with LOB or LONG data types.

Using the COUNT Function

COUNT(*) returns the number of rows in a table:

```
SELECT COUNT(*)
```

```
FROM employees
```

```
WHERE department_id = 50;
```

COUNT(*expr*) returns the number of rows with nonnull values for the *expr*:

```
SELECT COUNT(commission_pct)
```

```
FROM employees
```

```
WHERE department_id = 80;
```

Using the DISTINCT Keyword

- COUNT(DISTINCT *expr*) returns the number of distinct non-null values of the *expr*.

- To display the number of distinct department values in the EMPLOYEES table:

```
SELECT COUNT(DISTINCT department_id) FROM employees;
```

Use the DISTINCT keyword to suppress the counting of any duplicate values in a column.

Group Functions and Null Values

Group functions ignore null values in the column:

```
SELECT AVG(commission_pct)
```

```
FROM employees;
```

The NVL function forces group functions to include null values:

```
SELECT AVG(NVL(commission_pct, 0))
```

```
FROM employees;
```

Creating Groups of Data

To divide the table of information into smaller groups. This can be done by using the GROUP BY clause.

GROUP BY Clause Syntax

```
SELECT column, group_function(column)
```

```
FROM table
```

```
[WHERE condition]
```

```
[GROUP BY group_by_expression]
```

```
[ORDER BY column];
```

In the syntax:

group_by_expression specifies columns whose values determine the basis for grouping rows

Guidelines

- If you include a group function in a SELECT clause, you cannot select individual results as well, *unless* the individual column appears in the GROUP BY clause. You receive an error message if you fail to include the column list in the GROUP BY clause.
- Using a WHERE clause, you can exclude rows before dividing them into groups.
- You must include the *columns* in the GROUP BY clause.
- You cannot use a column alias in the GROUP BY clause.

Using the GROUP BY Clause

All columns in the SELECT list that are not in group functions must be in the GROUP BY clause.

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id ;
```

The GROUP BY column does not have to be in the SELECT list.

```
SELECT AVG(salary) FROM employees GROUP BY department_id ;
```

You can use the group function in the ORDER BY clause:

```
SELECT department_id, AVG(salary) FROM employees GROUP BY department_id ORDER BY
AVG(salary);
```

Grouping by More Than One Column

```
SELECT department_id dept_id, job_id, SUM(salary) FROM employees
GROUP BY department_id, job_id ;
```

Illegal Queries Using Group Functions

Any column or expression in the SELECT list that is not an aggregate function must be in the GROUP

BY clause:

```
SELECT department_id, COUNT(last_name) FROM employees;
```


You can correct the error by adding the GROUP BY clause:

```
SELECT department_id, count(last_name) FROM employees GROUP BY department_id;
```

You cannot use the WHERE clause to restrict groups.

- You use the HAVING clause to restrict groups.
- You cannot use group functions in the WHERE clause.

```
SELECT department_id, AVG(salary) FROM employees WHERE AVG(salary) > 8000  
GROUP BY department_id;
```

You can correct the error in the example by using the HAVING clause to restrict groups:

```
SELECT department_id, AVG(salary) FROM employees  
HAVING AVG(salary) > 8000 GROUP BY department_id;
```

Restricting Group Results

With the HAVING Clause .When you use the HAVING clause, the Oracle server restricts groups as follows:

1. Rows are grouped.
2. The group function is applied.
3. Groups matching the HAVING clause are displayed.

Using the HAVING Clause

```
SELECT department_id, MAX(salary) FROM employees  
GROUP BY department_id HAVING MAX(salary) > 10000 ;
```

The following example displays the department numbers and average salaries for those departments with a maximum salary that is greater than \$10,000:

```
SELECT department_id, AVG(salary) FROM employees GROUP BY department_id  
HAVING max(salary) > 10000;
```

Example displays the job ID and total monthly salary for each job that has a total payroll exceeding \$13,000. The example excludes sales representatives and sorts the list by the total monthly salary.

```
SELECT job_id, SUM(salary) PAYROLL FROM employees WHERE job_id NOT LIKE  
'%REP%'  
GROUP BY job_id HAVING SUM(salary) > 13000 ORDER BY SUM(salary);
```

Nesting Group Functions

Display the maximum average salary:

Group functions can be nested to a depth of two. The slide example displays the maximum average salary.

```
SELECT MAX(AVG(salary)) FROM employees GROUP BY department_id;
```

Summary

In this exercise, students should have learned how to:

- Use the group functions COUNT, MAX, MIN, and AVG
- Write queries that use the GROUP BY clause
- Write queries that use the HAVING clause

```
SELECT column, group_function
FROM table
[WHERE condition]
[GROUP BY group_by_expression]
[HAVING group_condition]
[ORDER BY column];
```

Find the Solution for the following:

Determine the validity of the following three statements. Circle either True or False.

1. Group functions work across many rows to produce one result per group.
True/False
2. Group functions include nulls in calculations.
True/False
3. The WHERE clause restricts rows prior to inclusion in a group calculation.
True/False

The HR department needs the following reports:

4. Find the highest, lowest, sum, and average salary of all employees. Label the columns Maximum, Minimum, Sum, and Average, respectively. Round your results to the nearest whole number

```
select max(salary) as "Maximum",min(salary) as "Minimum",sum(salary) as "Sum",avg(salary) as "Average" from my_employee;
```

Maximum	Minimum	Sum	Average
7000.5	750	12155.5	2025.91667

5. Modify the above query to display the minimum, maximum, sum, and average salary for each job type.

select max(salary) as "Maximum",min(salary) as "Minimum",sum(salary) as "Sum",avg(salary) as "Average" from my_employee group by job_id order by job_id;

JOB_ID	Minimum	Maximum	Sum	Average
AC_ACCOUNT	8300	8300	8300	8300.00
AC_MGR	12000	12000	12000	12000.00
AD_ASST	4400	4400	4400	4400.00
AD_PRES	24000	24000	24000	24000.00
AD_VP	17000	17000	34000	17000.00
FI_ACCOUNT	6900	12008	39600	9900.00
FI_MGR	12000	12000	12000	12000.00

6. Write a query to display the number of people with the same job. Generalize the query so that the user in the HR department is prompted for a job title.

SELECT job_id, COUNT(*) AS "Number of Employees" FROM employees WHERE job_id = UPPER('&job_title') GROUP BY job_id;

JOB_ID	Number of Employees
IT_PROG	5

7. Determine the number of managers without listing them. Label the column Number of Managers. *Hint: Use the MANAGER_ID column to determine the number of managers.*

SELECT COUNT(DISTINCT manager_id) AS "Number of Managers" FROM employees WHERE manager_id IS NOT NULL;

Number of Managers
18

8. Find the difference between the highest and lowest salaries. Label the column DIFFERENCE.

```
select (max(salary)-min(salary)) as "Difference" from my_employee;
```

Difference
6250.5

9. Create a report to display the manager number and the salary of the lowest-paid employee for that manager. Exclude anyone whose manager is not known. Exclude any groups where the minimum salary is \$6,000 or less. Sort the output in descending order of salary.

```
SELECT manager_id, MIN(salary) AS "Lowest_Salary" FROM employees WHERE  
manager_id IS NOT NULL GROUP BY manager_id HAVING MIN(salary) > 6000 ORDER  
BY MIN(salary) DESC;
```

ID	Lowest_Salary
6	7000.5

10. Create a query to display the total number of employees and, of that total, the number of employees hired in 1995, 1996, 1997, and 1998. Create appropriate column headings.

```
SELECT COUNT(*) AS "Total_Employees", SUM(CASE WHEN  
TO_CHAR(hire_date,'YYYY') = '1995' THEN 1 ELSE 0 END) AS  
"Hired_1995",SUM(CASE WHEN TO_CHAR(hire_date,'YYYY') = '1996' THEN 1 ELSE 0  
END) AS "Hired_1996",SUM(CASE WHEN TO_CHAR(hire_date,'YYYY') = '1997' THEN  
1 ELSE 0 END) AS "Hired_1997", SUM(CASE WHEN TO_CHAR(hire_date,'YYYY') =  
'1998' THEN 1 ELSE 0 END) AS "Hired_1998" FROM employees;
```

Total_Employees	Hired_1995	Hired_1996	Hired_1997	Hired_1998
6	0	0	0	1

11. Create a matrix query to display the job, the salary for that job based on department number, and the total salary for that job, for departments 20, 50, 80, and 90, giving each column an appropriate heading.

```
SELECT job_id AS "Job",SUM(CASE WHEN department_id = 20 THEN salary ELSE 0
END) AS "Dept_20", SUM(CASE WHEN department_id = 50 THEN salary ELSE 0 END)
AS "Dept_50", SUM(CASE WHEN department_id = 80 THEN salary ELSE 0 END) AS
"Dept_80", SUM(CASE WHEN department_id = 90 THEN salary ELSE 0 END) AS
"Dept_90", SUM(salary) AS "Total_Salary" FROM employees WHERE department_id
IN (20, 50, 80, 90) GROUP BY job_id ORDER BY job_id;
```

Job	Dept_20	Dept_50	Dept_80	Dept_90	Total_Salary
AD_ASST	4400	0	0	0	4400
SA_REP	0	156400	3060000	0	3216400
SA_MAN	0	61000	610000	0	671000
FI_ACCOUNT	39600	0	0	0	39600
AD_PRES	0	0	0	24000	24000

12. Write a query to display each department's name, location, number of employees, and the average salary for all the employees in that department. Label the column name-Location, Number of people, and salary respectively. Round the average salary to two decimal places.

```
SELECT d.department_name AS "Name", l.city AS "Location", COUNT(e.employee_id) AS
"Number of People", ROUND(AVG(e.salary), 2) AS "Salary"
FROM employees e JOIN departments d ON e.department_id = d.department_id
JOIN locations l ON d.location_id = l.location_id GROUP BY d.department_name, l.city
ORDER BY d.department_name;
```

Name	Location	Number of People	Salary
-----	-----	-----	-----
Administration	Seattle	1	4400.00
Executive	Seattle	3	19333.33
Finance	Seattle	6	8600.00
Human Resources	Toronto	1	6500.00
IT	Southlake	5	5760.00
Sales	Oxford	34	8955.88

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

Practice Questions

Date Functions

1. For DJs on Demand, display the number of months between the event_date of the Vigil wedding and today's date. Round to the nearest month.

```
SELECT ROUND(MONTHS_BETWEEN(SYSDATE, event_date)) AS "Months_Between" FROM  
events WHERE event_name='Vigil wedding';
```

2. Display the days between the start of last summer's school vacation break and the day school started this year. Assume 30.5 days per month. Name the output "Days."

```
SELECT ROUND((school_start - summer_break_start) * 30.5, 0) AS "Days" FROM dual;
```

3. Display the days between January 1 and December 31.

```
SELECT (TO_DATE('31-DEC-2025','DD-MON-YYYY') - TO_DATE('01-JAN-2025','DD-MON-YYYY'))  
AS "Days_Between" FROM dual;
```

4. Using one statement, round today's date to the nearest month and nearest year and truncate it to the nearest month and nearest year. Use an alias for each column.

```
SELECT ROUND(SYSDATE,'MONTH') AS "Rounded_Month",  
ROUND(SYSDATE,'YEAR') AS "Rounded_Year", TRUNC(SYSDATE,'MONTH') AS  
"Truncated_Month", TRUNC(SYSDATE,'YEAR') AS "Truncated_Year" FROM dual;
```

5. What is the last day of the month for June 2005? Use an alias for the output.

```
SELECT LAST_DAY(TO_DATE('01-JUN-2005','DD-MON-YYYY')) AS  
"Last_Day_June_2005" FROM dual;
```

6. Display the number of years between the Global Fast Foods employee Bob Miller's birthday and today. Round to the nearest year.

```
SELECT ROUND(MONTHS_BETWEEN(SYSDATE, birth_date)/12) AS "Years" FROM employees WHERE  
first_name='Bob' AND last_name='Miller';
```

7. Your next appointment with the dentist is six months from today. On what day will you go to the dentist? Name the output, "Appointment."

```
SELECT ADD_MONTHS(SYSDATE,6) AS "Appointment" FROM dual;
```

8. The teacher said you have until the last day of this month to turn in your research paper. What day will this be? Name the output, "Deadline."

```
SELECT LAST_DAY(SYSDATE) AS "Deadline" FROM dual;
```

9. How many months between your birthday this year and January 1 next year?

```
SELECT MONTHS_BETWEEN(TO_DATE('01-JAN-2026','DD-MON-YYYY'),TO_DATE('15-SEP-  
2025','DD-MON-YYYY')) AS "Months_Between" FROM dual;
```

10. What's the date of the next Friday after your birthday this year? Name the output, "First Friday."

```
SELECT NEXT_DAY(TO_DATE('15-SEP-2025','DD-MON-YYYY'),'FRIDAY') AS "First Friday" FROM  
dual;
```

1. Name a date function that will return a number.

MONTHS_BETWEEN(date1, date2)

12. Name a date function that will return a date.

ADD_MONTHS(date, n)

13. Give one example of why it is important for businesses to be able to manipulate date data?

Businesses use date functions to calculate billing cycles, employee age, due dates, and project deadlines accurately.

Conversion Functions

In each of the following exercises, feel free to use labels for the converted column to make the output more readable.

1. List the last names and birthdays of Global Fast Food Employees. Convert the birth dates to character data in the Month DD, YYYY format. Suppress any leading zeros.

```
SELECT last_name, TO_CHAR(birth_date, 'Month DD, YYYY') AS "Birthday" FROM f_employees;
```

2. Convert January 3, 04, to the default date format 03-Jan-2004.

```
SELECT TO_DATE('January 3, 04', 'Month DD, YY') AS "Formatted_Date" FROM dual;
```

3. Format a query from the Global Fast Foods f_promotional_menus table to print out the start_date of promotional code 110 as: The promotion began on the tenth of February 2004.

SELECT TO_DATE('January 3, 04', 'Month DD, YY') AS "Formatted_Date" FROM dual;

4. Convert today's date to a format such as: "Today is the Twentieth of March, Two Thousand Four"

SELECT 'Today is the ' || TO_CHAR(SYSDATE, 'fmDdspth "of" Month, Year') AS "Today" FROM dual;

5. List the ID, name and salary for all Global Fast Foods employees. Display salary with a \$ sign and two decimal places.

SELECT employee_id, first_name || ' ' || last_name AS "Name", TO_CHAR(salary, '\$9999.99') AS "Salary" FROM f_employees;

Evaluation Procedure	Marks awarded
Practice Evaluation (5)	
Viva(5)	
Total (10)	
Faculty Signature	

EXERCISE-9

Sub queries

Objectives

After completing this lesson, you should be able to do the following:

- Define subqueries
- Describe the types of problems that subqueries can solve
- List the types of subqueries
- Write single-row and multiple-row subqueries

Using a Subquery to Solve a Problem

Who has a salary greater than Abel's?

Main query:

Which employees have salaries greater than Abel's salary?

Subquery:

What is Abel's salary?

Subquery Syntax

`SELECT select_list FROM table WHERE expr operator (SELECT select_list FROM table);`

- The subquery (inner query) executes once before the main query (outer query).
- The result of the subquery is used by the main query.

A subquery is a SELECT statement that is embedded in a clause of another SELECT statement. You can build powerful statements out of simple ones by using subqueries. They can be very useful when you need to select rows from a table with a condition that depends on the data in the table itself.

You can place the subquery in a number of SQL clauses, including the following:

- WHERE clause
- HAVING clause
- FROM clause

In the syntax:

operator includes a comparison condition such as >, =, or IN

Note: Comparison conditions fall into two classes: single-row operators (>, =, >=, <, <=, <>) and multiple-row operators (IN, ANY, ALL). statement. The subquery generally executes first, and its output is used to complete the query condition for the main (or outer) query

Using a Subquery

`SELECT last_name FROM employees WHERE salary > (SELECT salary FROM employees WHERE last_name = 'Abel');`

The inner query determines the salary of employee Abel. The outer query takes the result of the inner query and uses this result to display all the employees who earn more than this amount.

Guidelines for Using Subqueries

- Enclose subqueries in parentheses.
- Place subqueries on the right side of the comparison condition.
- The ORDER BY clause in the subquery is not needed unless you are performing Top-N analysis.
- Use single-row operators with single-row

subqueries, and use multiple-row operators with multiple-row subqueries.

Types of Subqueries

- Single-row subqueries: Queries that return only one row from the inner SELECT statement.
- Multiple-row subqueries: Queries that return more than one row from the inner SELECT statement.

Single-Row Subqueries

- Return only one row
- Use single-row comparison operators

Example

Display the employees whose job ID is the same as that of employee 141:

```
SELECT last_name, job_id FROM employees WHERE job_id = (SELECT job_id FROM
employees
WHERE employee_id = 141);
```

Displays employees whose job ID is the same as that of employee 141 and whose salary is greater than that of employee 143.

```
SELECT last_name, job_id, salary FROM employees WHERE job_id =(SELECT job_id FROM
employees WHERE employee_id = 141) AND salary > (SELECT salary FROM employees
WHERE employee_id = 143);
```

Using Group Functions in a Subquery

Displays the employee last name, job ID, and salary of all employees whose salary is equal to the minimum salary. The MIN group function returns a single value (2500) to the outer query.

```
SELECT last_name, job_id, salary FROM employees WHERE salary = (SELECT MIN(salary)
FROM employees);
```

The HAVING Clause with Subqueries

- The Oracle server executes subqueries first.
- The Oracle server returns results into the HAVING clause of the main query.

Displays all the departments that have a minimum salary greater than that of department 50.

```
SELECT department_id, MIN(salary)
FROM employees
GROUP BY department_id
HAVING MIN(salary) >
(SELECT MIN(salary)
FROM employees
WHERE department_id = 50);
```

Example

Find the job with the lowest average salary.

```
SELECT job_id, AVG(salary)
FROM employees
GROUP BY job_id
HAVING AVG(salary) = (SELECT MIN(AVG(salary))
FROM employees
GROUP BY job_id);
```

What Is Wrong in this Statements?

```
SELECT employee_id, last_name
FROM employees
WHERE salary =(SELECT MIN(salary) FROM employees GROUP BY department_id);
Will This Statement Return Rows?
SELECT last_name, job_id
FROM employees
WHERE job_id =(SELECT job_id FROM employees WHERE last_name = 'Haas');
```

Multiple-Row Subqueries

- Return more than one row
- Use multiple-row comparison operators

Example

Find the employees who earn the same salary as the minimum salary for each department.

```
SELECT last_name, salary, department_id FROM employees WHERE salary IN (SELECT  
MIN(salary)  
FROM employees GROUP BY department_id);
```

Using the ANY Operator in Multiple-Row Subqueries

```
SELECT employee_id, last_name, job_id, salary FROM employees WHERE salary < ANY  
(SELECT salary FROM employees WHERE job_id = 'IT_PROG') AND job_id <> 'IT_PROG';
```

Displays employees who are not IT programmers and whose salary is less than that of any IT programmer. The maximum salary that a programmer earns is \$9,000.

< ANY means less than the maximum. >ANY means more than the minimum. =ANY is equivalent to IN.

Using the ALL Operator in Multiple-Row Subqueries

```
SELECT employee_id, last_name, job_id, salary  
FROM employees  
WHERE salary < ALL (SELECT salary FROM employees WHERE job_id = 'IT_PROG')  
AND job_id <> 'IT_PROG';
```

Displays employees whose salary is less than the salary of all employees with a job ID of IT_PROG and whose job is not IT_PROG.

➤ ALL means more than the maximum, and <ALL means less than the minimum.

The NOT operator can be used with IN, ANY, and ALL operators.

Null Values in a Subquery

```
SELECT emp.last_name FROM employees emp  
WHERE emp.employee_id NOT IN (SELECT mgr.manager_id FROM employees mgr);
```

Notice that the null value as part of the results set of a subquery is not a problem if you use the IN operator. The IN operator is equivalent to =ANY. For example, to display the employees who have subordinates, use the following SQL statement:

```
SELECT emp.last_name  
FROM employees emp  
WHERE emp.employee_id IN (SELECT mgr.manager_id FROM employees mgr);
```

Display all employees who do not have any subordinates:

```
SELECT last_name FROM employees
```

WHERE employee_id NOT IN (SELECT manager_id FROM employees WHERE manager_id IS NOT NULL);

Find the Solution for the following:

1. The HR department needs a query that prompts the user for an employee last name. The query then displays the last name and hire date of any employee in the same department as the employee whose name they supply (excluding that employee). For example, if the user enters Zlotkey, find all employees who work with Zlotkey (excluding Zlotkey).

SQL> SELECT last_name, hire_date FROM employees WHERE department_id = (SELECT department_id FROM employees WHERE last_name = 'Last_Name') AND last_name <> '&Last_Name';

LAST_NAME	HIRE_DATE
Abel	11-JUN-96
Taylor	24-MAR-97
King	09-JUN-94

2. Create a report that displays the employee number, last name, and salary of all employees who earn more than the average salary. Sort the results in order of ascending salary.

SELECT employee_id, last_name, salary FROM employees WHERE salary > (SELECT AVG(salary) FROM employees) ORDER BY salary ASC;

EMPLOYEE_ID	LAST_NAME	SALARY
101	Akash	42000
102	Sathya	45000
103	Thanesh	48000
104	Antony	52000
105	Tina	57000
106	Kiran	61000
107	Meena	67000
108	Nk	74000
109	Priya	83000
110	Rohit	91000

3. Write a query that displays the employee number and last name of all employees who work in a department with any employee whose last name contains a *u*.

```
SELECT employee_id, last_name FROM employees WHERE department_id IN ( SELECT
department_id FROM employees WHERE last_name LIKE '%u%' );
```

EMPLOYEE_ID	LAST_NAME
101	Akash
102	Sathya
103	Thanesh
104	Antony
105	Tina
106	Kiran
107	Meena
108	Nk
109	Priya
110	Rohit

4. The HR department needs a report that displays the last name, department number, and job ID of all employees whose department location ID is 1700.

```
SQL> SELECT last_name, department_id, job_id FROM employees WHERE department_id IN  
(SELECT department_id FROM departments WHERE location_id = 1700);
```

LAST_NAME	DEPARTMENT_ID	JOB_ID
Akash	10	IT_PROG
Sathya	10	IT_PROG
Thanesh	20	ST_CLERK
Antony	20	ST_CLERK
Tina	30	SA_REP
Kiran	30	SA_REP
Meena	40	HR_REP
Nk	40	HR_REP
Priya	50	FI_MGR
Rohit	50	FI_ACCOUNT

5. Create a report for HR that displays the last name and salary of every employee who reports to King.

```
SELECT last_name, salary FROM employees WHERE manager_id = (SELECT employee_id FROM  
employees WHERE last_name = 'King');
```

LAST_NAME	SALARY
-----	-----
Akash	42000
Sathya	45000
Thanesh	48000
Antony	52000
Tina	57000
Kiran	61000
Meena	67000
Nk	74000
Priya	83000
Rohit	91000

6. Create a report for HR that displays the department number, last name, and job ID for every employee in the Executive department.

```
SELECT department_id, last_name, job_id FROM employees WHERE department_id =
(SELECT department_id FROM departments WHERE department_name = 'Executive');
```

DEPARTMENT_ID	LAST_NAME	JOB_ID
-----	-----	-----
10	Akash	IT_PROG
10	Sathya	IT_PROG
10	Thanesh	IT_PROG
10	Antony	IT_PROG

7. Modify the query 3 to display the employee number, last name, and salary of all employees who earn more than the average salary and who work in a department with any employee whose last name contains a *u*.

```
SELECT employee_id, last_name, salary FROM employees WHERE salary > (SELECT AVG(salary)
FROM employees) AND department_id IN (SELECT department_id FROM employees WHERE
last_name LIKE '%u%') ORDER BY salary ASC;
```

EMPLOYEE_ID	LAST_NAME	SALARY
103	Thanesh	48000
106	Kiran	61000
108	Nk	74000
109	Priya	83000
110	Rohit	91000

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

Practice Questions

1. Ellen Abel is an employee who has received a \$2,000 raise. Display her first name and last name, her current salary, and her new salary. Display both salaries with a \$ and two decimal places. Label her new salary column AS New Salary.

```
SELECT first_name, last_name, TO_CHAR(salary, '$9999.99') AS "Current Salary", TO_CHAR(salary + 2000, '$9999.99') AS "New Salary" FROM employees WHERE first_name='Ellen' AND last_name='Abel';
```

2. On what day of the week and date did Global Fast Foods' promotional code 110 Valentine's Special begin?

```
SELECT TO_CHAR(start_date, 'Day, DD-Month-YYYY') AS "Valentine Start" FROM f_promotional_menus WHERE promo_code=110;
```

3. Create one query that will convert 25-Dec-2004 into each of the following (you will have to convert 25-Dec-2004 to a date and then to character data):

December 25th, 2004
DECEMBER 25TH, 2004
25th december, 2004

```
SELECT TO_CHAR(TO_DATE('25-Dec-2004','DD-Mon-YYYY'),'Month DDth, YYYY') AS "Mixed Case", TO_CHAR(TO_DATE('25-Dec-2004','DD-Mon-YYYY'),'MONTH DDTH, YYYY') AS "Upper Case", TO_CHAR(TO_DATE('25-Dec-2004','DD-Mon-YYYY'),'DDth month, YYYY') AS "Lower Case" FROM dual;
```

4. Create a query that will format the DJs on Demand d_packages columns, low-range and high-range package costs, in the format \$2500.00.

```
SELECT package_id, TO_CHAR(low_range, '$9999.00') AS "Low Range", TO_CHAR(high_range, '$9999.00') AS "High Range" FROM d_packages;
```

5. Convert JUNE192004 to a date using the fx format model.

```
SELECT TO_DATE('JUNE192004', 'fxMonthDDYYYY') AS "Converted Date" FROM dual;
```

6. What is the distinction between implicit and explicit datatype conversion? Give an example of each.

Implicit Conversion: Automatically done by Oracle
e.g., WHERE emp_id = '101' when emp_id is numeric.

Explicit Conversion: Manually done using functions
e.g., TO_NUMBER('101') or TO_CHAR(SYSDATE, 'DD-Mon-YYYY').

7. Why is it important from a business perspective to have datatype conversions?

Datatype conversions ensure **accurate calculations, proper data comparisons, and correct report formatting**, preventing data mismatch errors in business operations.

Evaluation Procedure	Marks awarded
Practice Evaluation (5)	
Viva(5)	
Total (10)	
Faculty Signature	

EXERCISE-10

USING THE SET OPERATORS

Objectives

After the completion this exercise, the students should be able to do the following:

- Describe set operators
- Use a set operator to combine multiple queries into a single query
- Control the order of rows returned

The set operators combine the results of two or more component queries into one result.

Queries containing set operators are called *compound queries*.

Operator	Returns
UNION	All distinct rows selected by either query
UNION ALL	All rows selected by either query, including all duplicates
INTERSECT	All distinct rows selected by both queries
MINUS	All distinct rows that are selected by the first SELECT statement and not selected in the second SELECT statement

The tables used in this lesson are:

- EMPLOYEES: Provides details regarding all current employees
- JOB_HISTORY: Records the details of the start date and end date of the former job, and the job identification number and department when an employee switches jobs

UNION Operator

Guidelines

- The number of columns and the data types of the columns being selected must be identical in all the SELECT statements used in the query. The names of the columns need not be identical.
- UNION operates over all of the columns being selected.
- NULL values are not ignored during duplicate checking.
- The IN operator has a higher precedence than the UNION operator.
- By default, the output is sorted in ascending order of the first column of the SELECT clause.

Example:

Display the current and previous job details of all employees. Display each employee only once.

```
SELECT employee_id, job_id FROM employees UNION SELECT employee_id, job_id  
FROM job_history;
```

Example:

```
SELECT employee_id, job_id, department_id  
FROM employees
```

```
UNION
SELECT employee_id, job_id, department_id
FROM job_history;
```

UNION ALL Operator

Guidelines

The guidelines for UNION and UNION ALL are the same, with the following two exceptions that pertain to UNION ALL:

- Unlike UNION, duplicate rows are not eliminated and the output is not sorted by default.
- The DISTINCT keyword cannot be used.

Example:

Display the current and previous departments of all employees.

```
SELECT employee_id, job_id, department_id
FROM employees
UNION ALL
SELECT employee_id, job_id, department_id
FROM job_history
ORDER BY employee_id;
```

INTERSECT Operator

Guidelines

- The number of columns and the data types of the columns being selected by the SELECT statements in the queries must be identical in all the SELECT statements used in the query. The names of the columns need not be identical.
- Reversing the order of the intersected tables does not alter the result.
- INTERSECT does not ignore NULL values.

Example:

Display the employee IDs and job IDs of those employees who currently have a job title that is the same as their job title when they were initially hired (that is, they changed jobs but have now gone back to doing their original job).

```
SELECT employee_id, job_id FROM employees
INTERSECT
SELECT employee_id, job_id
FROM job_history;
```

Example

```
SELECT employee_id, job_id, department_id
FROM employees
```

INTERSECT

```
SELECT employee_id, job_id, department_id  
FROM job_history;
```

MINUS Operator

Guidelines

- The number of columns and the data types of the columns being selected by the SELECT statements in the queries must be identical in all the SELECT statements used in the query. The names of the columns need not be identical.
- All of the columns in the WHERE clause must be in the SELECT clause for the MINUS operator to work.

Example:

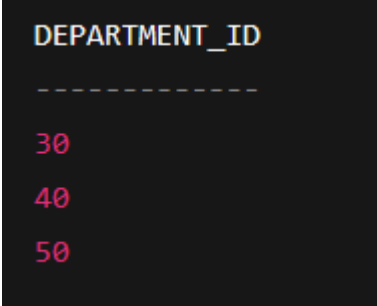
Display the employee IDs of those employees who have not changed their jobs even once.

```
SELECT employee_id, job_id  
FROM employees  
MINUS  
SELECT employee_id, job_id  
FROM job_history;
```

Find the Solution for the following:

1. The HR department needs a list of department IDs for departments that do not contain the job ID ST_CLERK. Use set operators to create this report.

```
SELECT department_id FROM departments MINUS SELECT DISTINCT department_id  
FROM employees WHERE job_id = 'ST_CLERK';
```



DEPARTMENT_ID
30
40
50

2. The HR department needs a list of countries that have no departments located in them. Display the country ID and the name of the countries. Use set operators to create this report.

```
SELECT country_id, country_name FROM countries MINUS SELECT DISTINCT  
c.country_id, c.country_name FROM countries c JOIN locations l ON c.country_id =  
l.country_id JOIN departments d ON l.location_id = d.location_id;
```


COUNTRY_ID	COUNTRY_NAME
CA	Canada
FR	France
JP	Japan

3. Produce a list of jobs for departments 10, 50, and 20, in that order. Display job ID and department ID using set operators.

```
SELECT job_id, department_id FROM employees WHERE department_id = 10 UNION ALL
SELECT job_id, department_id FROM employees WHERE department_id = 50 UNION ALL
SELECT job_id, department_id FROM employees WHERE department_id = 20;
```

JOB_ID	DEPARTMENT_ID
IT_PROG	10
IT_PROG	10
FI_ACCOUNT	50
FI_MGR	50
ST_CLERK	20
ST_CLERK	20

4. Create a report that lists the employee IDs and job IDs of those employees who currently have a job title that is the same as their job title when they were initially hired by the company (that is, they changed jobs but have now gone back to doing their original job).

```
SELECT e.employee_id, e.job_id FROM employees e JOIN job_history jh ON e.employee_id
= jh.employee_id WHERE e.job_id = jh.job_id;
```

EMPLOYEE_ID	JOB_ID
101	IT_PROG
104	ST_CLERK
108	HR_REP
110	FI_ACCOUNT

6. The HR department needs a report with the following specifications:

- Last name and department ID of all the employees from the EMPLOYEES table, regardless of whether or not they belong to a department.
- Department ID and department name of all the departments from the DEPARTMENTS table, regardless of whether or not they have employees working in them Write a compound query to accomplish this.

```
SELECT last_name AS name_or_employee, department_id FROM employees UNION  
SELECT department_name AS name_or_employee, department_id FROM departments;
```

NAME_OR_EMPLOYEE	DEPARTMENT_ID
-----	-----
Akash	10
Sathya	10
Thanesh	20
Antony	20
Tina	30
Kiran	30
Meena	40
Nk	40
Priya	50
Rohit	50
IT	10
Finance	50
Sales	30
HR	40
Executive	20

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

Practice Exercise

NULL Functions

1. Create a report that shows the Global Fast Foods promotional name, start date, and end date from the f_promotional_menus table. If there is an end date, temporarily replace it with “end in two weeks”. If there is no end date, replace it with today’s date.

```
SELECT promo_name, start_date, NVL(TO_CHAR(end_date, "end in two weeks"), TO_CHAR(SYSDATE, 'DD-Mon-YYYY')) AS "End Info" FROM f_promotional_menus;
```

2. Not all Global Fast Foods staff members receive overtime pay. Instead of displaying a null value for these employees, replace null with zero. Include the employee’s last name and overtime rate in the output. Label the overtime rate as “Overtime Status”.

```
SELECT last_name, NVL(overtime_rate, 0) AS "Overtime Status" FROM f_staffs;
```

3. The manager of Global Fast Foods has decided to give all staff who currently do not earn overtime an overtime rate of \$5.00. Construct a query that displays the last names and the overtime rate for each staff member, substituting \$5.00 for each null overtime value.

```
SELECT last_name, NVL(overtime_rate, 5.00) AS "Overtime Rate" FROM f_staffs;
```

4. Not all Global Fast Foods staff members have a manager. Create a query that displays the employee last name and 9999 in the manager ID column for these employees.

```
SELECT last_name, NVL(manager_id, 9999) AS "Manager ID" FROM f_staffs;
```

5. Which statement(s) below will return null if the value of v_sal is 50?

- a. SELECT nvl(v_sal, 50) FROM emp;
- b. SELECT nvl2(v_sal, 50) FROM emp;
- c. **SELECT nullif(v_sal, 50) FROM emp;**
- d. SELECT coalesce(v_sal, Null, 50) FROM emp;

6. What does this query on the Global Fast Foods table return?

```
SELECT COALESCE(last_name, to_char(manager_id)) as NAME  
FROM f_staffs;
```

```
SELECT COALESCE(last_name, to_char(manager_id)) as NAME FROM f_staffs;
```

Returns the last_name if it exists; otherwise returns manager_id converted to text.

7a. Create a report listing the first and last names and month of hire for all employees in the EMPLOYEES table (use TO_CHAR to convert hire_date to display the month).

```
SELECT first_name, last_name, TO_CHAR(hire_date, 'Month') AS "Month of Hire" FROM employees;
```

b. Modify the report to display null if the month of hire is September. Use the NULLIF function.

```
SELECT first_name, last_name, NULLIF(TO_CHAR(hire_date, 'Month'), 'September') AS "Month of Hire"  
FROM employees;
```

8. For all null values in the specialty column in the DJs on Demand d_partners table, substitute “No Specialty.” Show the first name and specialty columns only.

```
SELECT first_name, NVL(specialty, 'No Specialty') AS "Specialty" FROM d_partners;
```

Conditional Expressions

1. From the DJs on Demand d_songs table, create a query that replaces the 2-minute songs with “shortest” and the 10-minute songs with “longest”. Label the output column “Play Times”.

```
SELECT title, CASE duration WHEN 2 THEN 'shortest' WHEN 10 THEN 'longest' ELSE  
TO_CHAR(duration) END AS "Play Times" FROM d_songs;
```

2. Use the Oracle database employees table and CASE expression to decode the department id. Display the department id, last name, salary and a column called “New Salary” whose value is based on the following conditions:

If the department id is 10 then 1.25 * salary
If the department id is 90 then 1.5 * salary
If the department id is 130 then 1.75 * salary
Otherwise, display the old salary.

```
SELECT department_id, last_name, salary, CASE WHEN department_id = 10 THEN 1.25 * salary WHEN  
department_id = 90 THEN 1.5 * salary WHEN department_id = 130 THEN 1.75 * salary ELSE salary  
END AS "New Salary" FROM employees;
```

3. Display the first name, last name, manager ID, and commission percentage of all employees in departments 80 and 90. In a 5th column called “Review”, again display the manager ID. If they don’t have a manager, display the commission percentage. If they don’t have a commission, display 99999.

```
SELECT first_name, last_name, manager_id, commission_pct, CASE WHEN manager_id  
IS NULL THEN NVL(commission_pct, 99999) ELSE manager_id END AS "Review"  
FROM employees WHERE department_id IN (80, 90);
```

Cross Joins and Natural Joins

Use the Oracle database for problems 1-4.

1. Create a cross-join that displays the last name and department name from the employees and departments tables.

```
SELECT e.last_name, d.department_name FROM employees e CROSS JOIN departments d;
```

2. Create a query that uses a natural join to join the departments table and the locations table. Display the department id, department name, location id, and city.

```
SELECT department_id, department_name, location_id, city FROM departments NATURAL JOIN locations;
```

3. Create a query that uses a natural join to join the departments table and the locations table. Restrict the output to only department IDs of 20 and 50. Display the department id, department name, location id, and city.

```
SELECT department_id, department_name, location_id, city FROM departments NATURAL JOIN locations WHERE department_id IN (20, 50);
```

Evaluation Procedure	Marks awarded
Practice Evaluation (5)	
Viva(5)	
Total (10)	
Faculty Signature	

EXERCISE-11

CREATING VIEWS

After the completion of this exercise, students will be able to do the following:

- Describe a view
- Create, alter the definition of, and drop a view
- Retrieve data through a view
- Insert, update, and delete data through a view
- Create and use an inline view

View

A view is a logical table based on a table or another view. A view contains no data but is like a window through which data from tables can be viewed or changed. The tables on which a view is based are called base tables.

Advantages of Views

- To restrict data access
- To make complex queries easy
- To provide data independence
- To present different views of the same data

Classification of views

1. Simple view
2. Complex view

Feature	Simple	Complex
No. of tables	One	One or more
Contains functions	No	Yes
Contains groups of data	No	Yes
DML operations thr' view	Yes	Not always

Creating a view

Syntax

CREATE OR REPLACE FORCE/NOFORCE VIEW view_name AS Subquery WITH CHECK
OPTION CONSTRAINT constraint WITH READ ONLY CONSTRAINT constraint;

FORCE - Creates the view regardless of whether or not the base tables exist.

NOFORCE - Creates the view only if the base table exist.

WITH CHECK OPTION CONSTRAINT-specifies that only rows accessible to the view can be inserted or updated.

WITH READ ONLY CONSTRAINT-ensures that no DML operations can be performed on the view.

Example: 1 (Without using Column aliases)

Create a view EMPVU80 that contains details of employees in department80.

Example 2:

```
CREATE VIEW empvu80 AS SELECT employee_id, last_name, salary FROM employees
WHERE department_id=80;
```

Example:1 (Using column aliases)

```
CREATE VIEW salvu50
AS SELECT employee_id, id_number, last_name NAME, salary *12 ANN_SALARY
FROM employees
WHERE department_id=50;
```

Retrieving data from a view

Example:

```
SELECT * from salvu50;
```

Modifying a view

A view can be altered without dropping, re-creating.

Example: (Simple view)

Modify the EMPVU80 view by using CREATE OR REPLACE.

```
CREATE OR REPLACE VIEW empvu80 (id_number, name, sal, department_id)
AS SELECT employee_id, first_name, last_name, salary, department_id
FROM employees
WHERE department_id=80;
```

Example: (complex view)

```
CREATE VIEW dept_sum_vu (name, minsal, maxsal,avgsal)
AS   SELECT d.department_name, MIN(e.salary), MAX(e.salary), AVG(e.salary)
FROM employees e, department d
WHERE e.deparment_id=d.deparment_id
GROUP BY d.department_name;
```

Rules for performing DML operations on view

- Can perform operations on simple views
- Cannot remove a row if the view contains the following:
 - Group functions
 - Group By clause
 - Distinct keyword
- Cannot modify data in a view if it contains
 - Group functions
 - Group By clause
 - Distinct keyword
 - Columns contain by expressions
 -
- Cannot add data thr' a view if it contains
 - Group functions
 - Group By clause
 - Distinct keyword
 - Columns contain by expressions
 - NOT NULL columns in the base table that are not selected by the view

Example: (Using the WITH CHECK OPTION clause)

```
CREATE OR REPLACE VIEW empvu20
AS   SELECT *
FROM employees
WHERE department_id=20
WITH CHECK OPTION CONSTRAINT empvu20_ck;
```

Note:Any attempt to change the department number for any row in the view fails because it violates the WITH CHECK OPTION constraint.

Example – (Execute this and note the error)

```
UPDATE empvu20 SET department_id=10 WHERE employee_id=201;
```

Denying DML operations

Use of WITH READ ONLY option.

Any attempt to perform a DML on any row in the view results in an oracle server error.

Try this code:

```
CREATE OR REPLACE VIEW empvu10(employee_number, employee_name, job_title)
AS SELECT employee_id, last_name, job_id
FROM employees
WHERE department_id=10
WITH READ ONLY;
```

Find the Solution for the following:

1. Create a view called EMPLOYEE_VU based on the employee numbers, employee names and department numbers from the EMPLOYEES table. Change the heading for the employee name to EMPLOYEE.

```
CREATE OR REPLACE VIEW EMPLOYEE_VU AS SELECT employee_id, last_name AS
EMPLOYEE, department_id FROM employees;
```

EMPLOYEE_ID	EMPLOYEE	DEPARTMENT_ID
101	Akash	10
102	Sathya	10
103	Thanesh	20
104	Antony	20
105	Tina	30
106	Kiran	30
107	Meena	40
108	Nk	40
109	Priya	50
110	Rohit	50

2. Display the contents of the EMPLOYEES_VU view.

SELECT * FROM EMPLOYEE_VU;

EMPLOYEE_ID	EMPLOYEE	DEPARTMENT_ID
101	Akash	10
102	Sathya	10
103	Thanesh	20
104	Antony	20
105	Tina	30
106	Kiran	30
107	Meena	40
108	Nk	40
109	Priya	50
110	Rohit	50

3. Select the view name and text from the USER_VIEWS data dictionary views.

SELECT view_name, text FROM user_views;

VIEW_NAME	TEXT
EMPLOYEE_VU	CREATE VIEW EMPLOYEE_VU AS SELECT employee_id, last_name AS EMPLOYEE, department_id FROM employees
ANOTHER_VIEW	CREATE VIEW ANOTHER_VIEW AS SELECT ...

4. Using your EMPLOYEES_VU view, enter a query to display all employees names and department.

SELECT EMPLOYEE, department_id FROM EMPLOYEE_VU;

EMPLOYEE	DEPARTMENT_ID
Akash	10
Sathya	10
Thanesh	20
Antony	20
Tina	30
Kiran	30
Meena	40
Nk	40
Priya	50
Rohit	50

5. Create a view named DEPT50 that contains the employee number, employee last names and department numbers for all employees in department 50. Label the view columns EMPNO, EMPLOYEE and DEPTNO. Do not allow an employee to be reassigned to another department through the view.

**CREATE OR REPLACE VIEW DEPT50 (EMPNO, EMPLOYEE, DEPTNO) AS SELECT
employee_id, last_name, department_id FROM employees WHERE department_id = 50
WITH READ ONLY;**

EMPNO	EMPLOYEE	DEPTNO
109	Priya	50
110	Rohit	50

6. Display the structure and contents of the DEPT50 view.

DESCRIBE DEPT50;

Name	Null?	Type
EMPNO		NUMBER
EMPLOYEE		VARCHAR2(50)
DEPTNO		NUMBER

7. Attempt to reassign Matos to department 80.

UPDATE DEPT50 SET DEPTNO = 80 WHERE EMPLOYEE = 'Matos';

UPDATE DEPT50

8. Create a view called SALARY_VU based on the employee last names, department names, salaries, and salary grades for all employees. Use the Employees, DEPARTMENTS and JOB_GRADE tables. Label the column Employee, Department, salary, and Grade respectively.

CREATE OR REPLACE VIEW SALARY_VU AS SELECT e.last_name AS Employee, d.department_name AS Department, e.salary AS salary, j.grade AS Grade FROM employees e JOIN departments d ON e.department_id = d.department_id JOIN job_grades j ON e.salary BETWEEN j.lowest_sal AND j.highest_sal;

EMPLOYEE	DEPARTMENT	SALARY	GRADE
Akash	IT	42000	3
Sathya	IT	45000	3
Thanesh	Executive	48000	4
Antony	Executive	52000	4
Tina	Sales	57000	4
Kiran	Sales	61000	5
Meena	HR	67000	5
Nk	HR	74000	5
Priya	Finance	83000	5
Rohit	Finance	91000	6

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

EXERCISE 12

Intro to Constraints: NOT NULL and UNIQUE Constraints

Global Fast Foods has been very successful this past year and has opened several new stores. They need to add a table to their database to store information about each of their store's locations. The owners want to make sure that all entries have an identification number, date opened, address, and city and that no other entry in the table can have the same email address. Based on this information, answer the following questions about the global_locations table. Use the table for your answers.

Global Fast Foods global_locations Table						
NAME	TYPE	LENGTH	PRECISION	SCALE	NULLABLE	DEFAULT
Id	NUMBER		10	0		
name	VARCHAR2	100			NULLABLE	
date_opened	DATE					
address	VARCHAR2	200				
city	VARCHAR2	50				
zip/postal code	VARCHAR2	10			NULLABLE	
phone	VARCHAR2	15			NULLABLE	
email	VARCHAR2	100			NULLABLE	
manager_id	NUMBER		10	0	NULLABLE	
Emergency contact	VARCHAR2	100			NULLABLE	

1. What is a "constraint" as it relates to data integrity?

Constrain restricts the database to ensure the given data is restricted or accurate. It maintains the data quality that is stored in daatabase.

2. What are the limitations of constraints that may be applied at the column level and at the table level?

Column level constraints applied to one column.

Table level constraints applied to multiple column and flexible but can be more complex.

3. Why is it important to give meaningful names to constraints?

Because it will be easier to debug simplifies communication, easier to understand and better readability.

4. Based on the information provided by the owners, choose a datatype for each column. Indicate the length, precision, and scale for each NUMBER datatype.

5. Use "(nullable)" to indicate those columns that can have null values.

6. Write the CREATE TABLE statement for the Global Fast Foods locations table to define the constraints at the column level.

```
CREATE TABLE global_locations (Id NUMBER(10) CONSTRAINT pk_global_locations  
PRIMARY KEY, name VARCHAR2(100) NOT NULL, date_opened DATE DEFAULT SYSDATE  
NOT NULL, address VARCHAR2(200), city VARCHAR2(50) NOT NULL, zip_postal_code  
VARCHAR2(10) NOT NULL, phone VARCHAR2(15), email VARCHAR2(100), manager_id  
NUMBER(10), emergency_contact VARCHAR2(100));
```

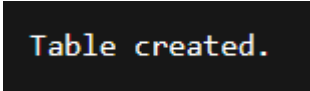


Table created.

7. Execute the CREATE TABLE statement in Oracle Application Express.

```
CREATE TABLE global_locations (Id NUMBER(10) CONSTRAINT pk_global_locations  
PRIMARY KEY, name VARCHAR2(100) NOT NULL, date_opened DATE DEFAULT SYSDATE  
NOT NULL, address VARCHAR2(200), city VARCHAR2(50) NOT NULL, zip_postal_code  
VARCHAR2(10) NOT NULL, phone VARCHAR2(15), email VARCHAR2(100), manager_id  
NUMBER(10), emergency_contact VARCHAR2(100));
```

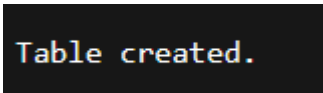


Table created.

8. Execute a DESCRIBE command to view the Table Summary information.

DESCRIBE global_locations;

Name	Null?	Type
-----	-----	-----
ID	NOT NULL	NUMBER(10)
NAME	NOT NULL	VARCHAR2(100)
DATE_OPENED	NOT NULL	DATE
ADDRESS		VARCHAR2(200)
CITY	NOT NULL	VARCHAR2(50)
ZIP_POSTAL_CODE	NOT NULL	VARCHAR2(10)
PHONE		VARCHAR2(15)
EMAIL		VARCHAR2(100)
MANAGER_ID		NUMBER(10)
EMERGENCY_CONTACT		VARCHAR2(100)

9. Rewrite the CREATE TABLE statement for the Global Fast Foods locations table to define the UNIQUE constraints at the table level. Do not execute this statement.

NAME	TYPE	LENGTH	PRECISION	SCALE	NULLABLE	DEFAULT
id	number	4				
loc_name	varchar2	20			X	
	date					
address	varchar2	30				
city	varchar2	20				
zip_postal	varchar2	20			X	
phone	varchar2	15			X	
email	varchar2	80			X	
manager_id	number	4			X	
contact	varchar2	40			X	

```
CREATE TABLE global_locations (  
  id      NUMBER(4)    NOT NULL,  
  loc_name VARCHAR2(20),  
  date_opened DATE,  
  address  VARCHAR2(30),  
  city     VARCHAR2(20),  
  zip_postal VARCHAR2(20),  
  phone    VARCHAR2(15),  
  email     VARCHAR2(80),  
  manager_id NUMBER(4),  
  contact   VARCHAR2(40),  
  CONSTRAINT pk_global_locations PRIMARY KEY (id),  
  CONSTRAINT uq_loc_name UNIQUE (loc_name),  
  CONSTRAINT uq_zip_postal UNIQUE (zip_postal),  
  CONSTRAINT uq_phone UNIQUE (phone),  
  CONSTRAINT uq_email UNIQUE (email),  
  CONSTRAINT uq_manager_id UNIQUE (manager_id),  
  CONSTRAINT uq_contact UNIQUE (contact)  
);
```

PRACTICE PROBLEM

Managing Constraints

Using Oracle Application Express, click the SQL Workshop tab in the menu bar. Click the Object Browser and verify that you have a table named copy_d_clients and a table named copy_d_events. If you don't have these tables in your schema, create them before completing the exercises below. Here is how the original tables are related. The d_clients table has a primary key client_number. This has a primary-key constraint and it is referenced in the foreign-key constraint on the d_events table.

NOTE: The practice exercises use the d_clients and d_events tables in the DJs on Demand database. Students will work with copies of these two tables named copy_d_clients and copy_d_events. Make sure they have new copies of the tables (without changes made from previous exercises). Remember, tables copied using a subquery do not have the integrity constraints as established in the original tables. When using the SELECT statement to view the constraint name, the tablename must be all capital letters.

1. What are four functions that an ALTER statement can perform on constraints?

- Add constrain
- Enable constrain
- Drop constrain
- Disable constrain

2. Since the tables are copies of the original tables, the integrity rules are not passed onto the new tables; only the column datatype definitions remain. You will need to add a PRIMARY KEY constraint to the copy_d_clients table. Name the primary key copy_d_clients_pk . What is the syntax you used to create the PRIMARY KEY constraint to the copy_d_clients.table?

ALTER TABLE copy_d_clients ADD CONSTRAINT copy_d_clients_pk PRIMARY KEY (client_id);

3. Create a FOREIGN KEY constraint in the copy_d_events table. Name the foreign key copy_d_events_fk. This key references the copy_d_clients table client_number column. What is the syntax you used to create the FOREIGN KEY constraint in the copy_d_events table?

**ALTER TABLE copy_d_events ADD CONSTRAINT copy_d_events_fk FOREIGN KEY (client_number)
REFERENCES copy_d_clients (client_number);**

4. Use a SELECT statement to verify the constraint names for each of the tables. Note that the tablenames must be capitalized.

a. The constraint name for the primary key in the copy_d_clients table is **COPY_D_CLIENTS_PK**

5. Drop the PRIMARY KEY constraint on the copy_d_clients table. Explain your results.

ALTER TABLE copy_d_clients DROP CONSTRAINT copy_d_clients_pk;

6. Add the following event to the copy_d_events table. Explain your results.

ID	NAME	EVENT_DATE	DESCRIPTION	COST	VENUE_ID	PACKAGE_CODE	THEME_CODE	CLIENT_NUMBER
140	Cline Bas Mitzvah	15-Jul-2004	Church and Private Home formal	4500	105	87	77	7125

**INSERT INTO copy_d_events
(id, name, event_date, description, cost, venue_id, package_code, theme_code, client_number)
VALUES (140, 'Cline Bas Mitzvah', TO_DATE('15-Jul-2004', 'DD-Mon-YYYY'), 'Church and Private Home formal', 4500, 105, 87, 77, 7125);**

7. Create an ALTER TABLE query to disable the primary key in the copy_d_clients table. Then add the values from #6 to the copy_d_events table. Explain your results.

```
ALTER TABLE copy_d_clients DISABLE CONSTRAINT copy_d_clients_pk;
```

8. Repeat question 6: Insert the new values in the copy_d_events table. Explain your results.

```
INSERT INTO copy_d_events  
(id, name, event_date, description, cost, venue_id, package_code, theme_code, client_number)  
VALUES (140, 'Cline Bas Mitzvah', TO_DATE('15-Jul-2004', 'DD-Mon-YYYY'),  
'Church and Private Home formal', 4500, 105, 87, 77, 7125);
```

9. Enable the primary-key constraint in the copy_d_clients table. Explain your results.

```
ALTER TABLE copy_d_clients ENABLE CONSTRAINT copy_d_clients_pk;
```

10. If you wanted to enable the foreign-key column and reestablish the referential integrity between these two tables, what must be done?

```
ALTER TABLE copy_d_clients ENABLE CONSTRAINT copy_d_clients_pk;
```

11. Why might you want to disable and then re-enable a constraint?

You might disable a constraint to **load or modify data quickly** or **fix data errors**, and then re-enable it to **restore data integrity**.

A constraint may be **disabled** temporarily to allow **bulk data insertion, updates, or corrections** without triggering errors. This is useful when existing data violates the rules or when performing maintenance on

the table. Once the data is corrected or the operation is complete, the constraint is **re-enabled** to **restore referential integrity and enforce data rules**.

12. Query the data dictionary for some of the constraints that you have created. How does the data dictionary identify each constraint type?

```
SELECT table_name, constraint_name, constraint_type FROM user_constraints  
WHERE table_name IN ('COPY_D_CLIENTS', 'COPY_D_EVENTS');
```

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

EXERCISE 13

Creating Views

1. What are three uses for a view from a DBA's perspective?

- **TO restrict access to sensitive data.**
- **To simplify complex queries for users.**
- **To hide table structure changes.**

2. Create a simple view called view_d_songs that contains the ID, title and artist from the DJs on Demand table for each "New Age" type code. In the subquery, use the alias "Song Title" for the title column.

CREATE VIEW view_d_songs AS SELECT ID, title AS "Song Title", artist FROM DJs_on_Demand

WHERE type_code = 'New Age';

3. SELECT * FROM view_d_songs. What was returned?

It returns ID, song title and artist of all songs from the new age type code.

4. REPLACE view_d_songs. Add type_code to the column list. Use aliases for all columns. Or use alias after the CREATE statement as shown.

This replace s the old view ams adds the type code column with aliases for all columns.

5. Jason Tsang, the disk jockey for DJs on Demand, needs a list of the past events and those planned for the coming months so he can make arrangements for each event's equipment setup. As the company manager, you do not want him to have access to the price that clients paid for their events. Create a view for Jason to use that displays the name of the event, the event date, and the theme description. Use aliases for each column name.

```
CREATE VIEW view_jason_event_schedule AS SELECT event_name AS "Event Name",  
event_date AS "Date of Event", theme_description AS "Event Theme" FROM EVENTS;
```

6. It is company policy that only upper-level management be allowed access to individual employee salaries. The department managers, however, need to know the minimum, maximum, and average salaries, grouped by department. Use the Oracle database to prepare a view that displays the needed information for department managers.

```
CREATE VIEW view_dept_salary_summary AS SELECT DEPARTMENT_ID, MIN(SALARY) AS "Min Salary",  
MAX(SALARY) AS "Max Salary", ROUND(AVG(SALARY)) AS "Avg Salary" FROM EMPLOYEES GROUP BY  
DEPARTMENT_ID;
```

DML Operations and Views

Use the DESCRIBE statement to verify that you have tables named copy_d_songs, copy_d_events, copy_d_cds, and copy_d_clients in your schema. If you don't, write a query to create a copy of each.

1. Query the data dictionary USER_UPDATABLE_COLUMNS to make sure the columns in the base tables will allow UPDATE, INSERT, or DELETE. All table names in the data dictionary are stored in uppercase.

```
SELECT TABLE_NAME, COLUMN_NAME, UPDATABLE, INSERTABLE, DELETABLE  
FROM USER_UPDATABLE_COLUMNS  
WHERE TABLE_NAME = 'YOUR_TABLE_NAME';
```

Use the same syntax but change table_name of the other tables.

```
SELECT TABLE_NAME, COLUMN_NAME, UPDATABLE, INSERTABLE, DELETABLE  
FROM USER_UPDATABLE_COLUMNS  
WHERE TABLE_NAME = 'EMPLOYEES';
```

2. Use the CREATE or REPLACE option to create a view of *all* the columns in the copy_d_songs table called view_copy_d_songs.

```
CREATE OR REPLACE VIEW view_copy_d_songs AS  
SELECT * FROM copy_d_songs;
```

3. Use view_copy_d_songs to INSERT the following data into the underlying copy_d_songs table. Execute a SELECT * from copy_d_songs to verify your DML command. See the graphic.

ID	TITLE	DURATION	ARTIST	TYPE_CODE
88	Mello Jello	2	The What	4

CREATE OR REPLACE VIEW view_copy_d_songs AS

SELECT * FROM copy_d_songs;

4. Create a view based on the DJs on Demand COPY_D_CDS table. Name the view read_copy_d_cds. Select all columns to be included in the view. Add a WHERE clause to restrict the year to 2000. Add the WITH READ ONLY option.

CREATE OR REPLACE VIEW read_copy_d_cds AS SELECT * FROM copy_d_cds WHERE year = 2000 WITH READ ONLY;

5. Using the read_copy_d_cds view, execute a DELETE FROM read_copy_d_cds WHERE cd_number = 90;

DELETE FROM copy_d_cds WHERE cd_number = 90;

6. Use REPLACE to modify read_copy_d_cds. Replace the READ ONLY option with WITH CHECK OPTION CONSTRAINT ck_read_copy_d_cds. Execute a SELECT * statement to verify that the view exists.

CREATE OR REPLACE VIEW read_copy_d_cds AS SELECT * FROM copy_d_cds WHERE year = 2000 WITH CHECK OPTION CONSTRAINT ck_read_copy_d_cds;

7. Use the read_copy_d_cds view to delete any CD of year 2000 from the underlying copy_d_cds.

DELETE FROM read_copy_d_cds WHERE year = 2000;

8. Use the read_copy_d_cds view to delete cd_number 90 from the underlying copy_d_cds table.

DELETE FROM read_copy_d_cds WHERE cd_number = 90;

9. Use the read_copy_d_cds view to delete year 2001 records.

DELETE FROM read_copy_d_cds WHERE year = 2001;

10. Execute a SELECT * statement for the base table copy_d_cds. What rows were deleted?

SELECT * FROM copy_d_cds;

11. What are the restrictions on modifying data through a view?

Data can only be modified through a view if it is simple—based on a single table with no joins, aggregations, or calculated columns.

Views with joins, GROUP BY, DISTINCT, aggregates, or read-only restrictions cannot be updated

12. What is Moore's Law? Do you consider that it will continue to apply indefinitely? Support your opinion with research from the internet.

Moore's Law is the observation that the number of transistors on a microchip doubles approximately every two years, leading to an exponential increase in computing power and a decrease in relative cost.
No, it will not continue to apply indefinitely in its traditional form.

13. What is the "singularity" in terms of computing?

The **Technological Singularity** is the theoretical future point where **Artificial Intelligence surpasses human intelligence** and enters a cycle of **rapid, self-driven improvement**. This event would lead to **unpredictable and irreversible** technological and societal change.

Managing Views

1. Create a view from the copy_d_songs table called view_copy_d_songs that includes only the title and artist. Execute a SELECT * statement to verify that the view exists.

CREATE OR REPLACE VIEW view_copy_d_songs AS SELECT title, artist FROM copy_d_songs;

2. Issue a DROP view_copy_d_songs. Execute a SELECT * statement to verify that the view has been deleted.

```
DROP VIEW view_copy_d_songs;  
SELECT * FROM view_copy_d_songs;
```

3. Create a query that selects the last name and salary from the Oracle database. Rank the salaries from highest to lowest for the top three employees.

```
SELECT last_name, salary, RANK() OVER (ORDER BY salary DESC) AS salary_rank  
FROM employees  
WHERE RANK() OVER (ORDER BY salary DESC) <= 3;
```

4. Construct an inline view from the Oracle database that lists the last name, salary, department ID, and maximum salary for each department. Hint: One query will need to calculate maximum salary by

department ID.

```
SELECT e.last_name, e.salary, e.department_id, d.max_salary  
FROM employees e JOIN (SELECT department_id, MAX(salary) AS max_salary FROM employees  
GROUP BY department_id) d ON e.department_id = d.department_id;
```

5. Create a query that will return the staff members of Global Fast Foods ranked by salary from lowest to highest.

```
SELECT last_name, salary, RANK() OVER (ORDER BY salary ASC) AS salary_rank  
FROM employees;
```

Indexes and Synonyms

1. What is an index and what is it used for?

- **An index is a database object that improves the speed of data retrieval from a table.**
- **It works like a book's index, allowing Oracle to quickly locate rows without scanning the entire table.**
- **Indexes do not store all table data, only pointers to rows.**

Usage:

- **Speeds up SELECT queries with WHERE, JOIN, or ORDER BY clauses.**
- **Can enforce uniqueness if defined as a unique index.**

2. What is a ROWID, and how is it used?

- **ROWID is a unique address for each row in an Oracle table.**
- **It identifies the physical location of a row in the database.**

Usage:

- **Quickly retrieve or delete a specific row.**
- **Optimize performance for row-level operations.**

3. When will an index be created automatically?

An index is automatically created in Oracle:

- **When a PRIMARY KEY constraint is created.**
- **When a UNIQUE constraint is created.**

4. Create a nonunique index (foreign key) for the DJs on Demand column (cd_number) in the D_TRACK_LISTINGS table. Use the Oracle Application Express SQL Workshop Data Browser to confirm that the index was created.

```
CREATE INDEX idx_cd_number ON d_track_listings(cd_number);  
SELECT index_name, table_name, uniqueness  
FROM user_indexes  
WHERE table_name = 'D_TRACK_LISTINGS';
```

5. Use the join statement to display the indexes and uniqueness that exist in the data dictionary for the DJs on Demand D_SONGS table.

```
SELECT ui.index_name, ui.uniqueness, uic.column_name  
FROM user_indexes ui  
JOIN user_ind_columns uic  
ON ui.index_name = uic.index_name  
WHERE ui.table_name = 'D_SONGS';
```

6. Use a SELECT statement to display the index_name, table_name, and uniqueness from the data dictionary USER_INDEXES for the DJs on Demand D_EVENTS table.

```
SELECT index_name, table_name, uniqueness  
FROM user_indexes  
WHERE table_name = 'D_EVENTS';
```

7. Write a query to create a synonym called dj_tracks for the DJs on Demand d_track_listings table.

```
CREATE SYNONYM dj_tracks FOR d_track_listings;
```

8. Create a function-based index for the last_name column in DJs on Demand D_PARTNERS table that makes it possible not to have to capitalize the table name for searches. Write a SELECT statement that would use this index.

CREATE INDEX idx_last_name_lower ON d_partners(LOWER(last_name));

9. Create a synonym for the D_TRACK_LISTINGS table. Confirm that it has been created by querying the data dictionary.

CREATE SYNONYM track_list FOR d_track_listings;

10. Drop the synonym that you created in question

DROP SYNONYM track_list;

Evaluation Procedure	Marks awarded
Query(5)	
Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

EXERCISE-14

OTHER DATABASE OBJECTS

Objectives

After the completion of this exercise, the students will be able to do the following:

- Create, maintain, and use sequences
- Create and maintain indexes

Database Objects

Many applications require the use of unique numbers as primary key values. You can either build code into the application to handle this requirement or use a sequence to generate unique numbers. If you want to improve the performance of some queries, you should consider creating an index. You can also use indexes to enforce uniqueness on a column or a collection of columns. You can provide alternative names for objects by using synonyms.

What Is a Sequence?

A sequence:

- Automatically generates unique numbers
- Is a sharable object
- Is typically used to create a primary key value
- Replaces application code
- Speeds up the efficiency of accessing sequence values when cached in memory

The CREATE SEQUENCE Statement Syntax

Define a sequence to generate sequential numbers automatically:

```
CREATE SEQUENCE sequence
[INCREMENT BY n]
[START WITH n]
[{MAXVALUE n | NOMAXVALUE}]
[{MINVALUE n | NOMINVALUE}]
[{CYCLE | NOCYCLE}]
```

[{CACHE *n* | NOCACHE}];

In the syntax:

sequence is the name of the sequence generator

INCREMENT BY *n* specifies the interval between sequence numbers where *n* is an integer (If this clause is omitted, the sequence increments by 1.)

START WITH *n* specifies the first sequence number to be generated (If this clause is omitted, the sequence starts with 1.)

MAXVALUE *n* specifies the maximum value the sequence can generate

NOMAXVALUE specifies a maximum value of 10^{27} for an ascending sequence and -1 for a descending sequence (This is the default option.)

MINVALUE *n* specifies the minimum sequence value

NOMINVALUE specifies a minimum value of 1 for an ascending sequence and $-(10^{26})$ for a descending sequence (This is the default option.)

CYCLE | NOCYCLE specifies whether the sequence continues to generate values after reaching its maximum or minimum value (NOCYCLE is the default option.)

CACHE *n* | NOCACHE specifies how many values the Oracle server preallocates and keep in memory (By default, the Oracle server caches 20 values.)

Creating a Sequence

- Create a sequence named DEPT_DEPTID_SEQ to be used for the primary key of the DEPARTMENTS table.
- Do not use the CYCLE option.

EXAMPLE:

```
CREATE SEQUENCE dept_deptid_seq  
INCREMENT BY 10  
START WITH 120  
MAXVALUE 9999  
NOCACHE  
NOCYCLE;
```

Confirming Sequences

- Verify your sequence values in the USER_SEQUENCES data dictionary table.
- The LAST_NUMBER column displays the next available sequence number if NOCACHE is specified.

EXAMPLE:

SELECT sequence_name, min_value, max_value, increment_by, last_number

NEXTVAL and CURRVAL Pseudocolumns

- NEXTVAL returns the next available sequence value. It returns a unique value every time it is referenced, even for different users.
- CURRVAL obtains the current sequence value.
- NEXTVAL must be issued for that sequence before CURRVAL contains a value.

Rules for Using NEXTVAL and CURRVAL

You can use NEXTVAL and CURRVAL in the following contexts:

- The SELECT list of a SELECT statement that is not part of a subquery
- The SELECT list of a subquery in an INSERT statement
- The VALUES clause of an INSERT statement
- The SET clause of an UPDATE statement

You cannot use NEXTVAL and CURRVAL in the following contexts:

- The SELECT list of a view
- A SELECT statement with the DISTINCT keyword
- A SELECT statement with GROUP BY, HAVING, or ORDER BY clauses
- A subquery in a SELECT, DELETE, or UPDATE statement
- The DEFAULT expression in a CREATE TABLE or ALTER TABLE statement

Using a Sequence

- Insert a new department named “Support” in location ID 2500.
- View the current value for the DEPT_DEPTID_SEQ sequence.

EXAMPLE:

```
INSERT INTO departments(department_id, department_name, location_id)
VALUES (dept_deptid_seq.NEXTVAL, 'Support', 2500);
```

```
SELECT dept_deptid_seq.CURRVAL FROM dual;
```

The example inserts a new department in the DEPARTMENTS table. It uses the DEPT_DEPTID_SEQ sequence for generating a new department number as follows:

You can view the current value of the sequence:

```
SELECT dept_deptid_seq.CURRVAL FROM dual;
```

Removing a Sequence

- Remove a sequence from the data dictionary by using the DROP SEQUENCE statement.
- Once removed, the sequence can no longer be referenced.

EXAMPLE:

DROP SEQUENCE dept_deptid_seq;

What is an Index?

An index:

- Is a schema object
- Is used by the Oracle server to speed up the retrieval of rows by using a pointer
- Can reduce disk I/O by using a rapid path access method to locate data quickly
- Is independent of the table it indexes
- Is used and maintained automatically by the Oracle server

How Are Indexes Created?

- Automatically: A unique index is created automatically when you define a PRIMARY KEY or UNIQUE constraint in a table definition.
- Manually: Users can create nonunique indexes on columns to speed up access to the rows.

Types of Indexes

Two types of indexes can be created. One type is a unique index: the Oracle server automatically creates this index when you define a column in a table to have a PRIMARY KEY or a UNIQUE key constraint. The name of the index is the name given to the constraint.

The other type of index is a nonunique index, which a user can create. For example, you can create a FOREIGN KEY column index for a join in a query to improve retrieval speed.

Creating an Index

- Create an index on one or more columns.
- Improve the speed of query access to the LAST_NAME column in the EMPLOYEES table.

CREATE INDEX *index*
ON *table* (*column*[, *column*]...);

EXAMPLE:

CREATE INDEX emp_last_name_idx
ON employees(last_name);

In the syntax:

index is the name of the index

table is the name of the table

column is the name of the column in the table to be indexed

When to Create an Index

You should create an index if:

- A column contains a wide range of values
- A column contains a large number of null values
- One or more columns are frequently used together in a WHERE clause or a join condition
- The table is large and most queries are expected to retrieve less than 2 to 4 percent of the rows

When Not to Create an Index

It is usually not worth creating an index if:

- The table is small
- The columns are not often used as a condition in the query
- Most queries are expected to retrieve more than 2 to 4 percent of the rows in the table
- The table is updated frequently
- The indexed columns are referenced as part of an Expression

Confirming Indexes

- The USER_INDEXES data dictionary view contains the name of the index and its uniqueness.
- The USER_IND_COLUMNS view contains the index name, the table name, and the column name.

EXAMPLE:

```
SELECT ic.index_name, ic.column_name, ic.column_position col_pos, ix.uniqueness
FROM user_indexes ix, user_ind_columns ic
WHERE ic.index_name = ix.index_name
AND ic.table_name = 'EMPLOYEES';
```

Removing an Index

- Remove an index from the data dictionary by using the DROP INDEX command.
- Remove the UPPER_LAST_NAME_IDX index from the data dictionary.
- To drop an index, you must be the owner of the index or have the DROP ANY INDEX privilege.

```
DROP INDEX upper_last_name_idx;
```

```
DROP INDEX index;
```

Find the Solution for the following:

1. Create a sequence to be used with the primary key column of the DEPT table. The sequence should start at 200 and have a maximum value of 1000. Have your sequence increment by ten numbers. Name the sequence DEPT_ID_SEQ.

```
CREATE SEQUENCE DEPT_ID_SEQ START WITH 200 INCREMENT BY 10
MAXVALUE 1000 NOCACHE NOCYCLE;
```


2. Write a query in a script to display the following information about your sequences: sequence name, maximum value, increment size, and last number.

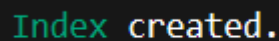
```
SELECT SEQUENCE_NAME AS "Sequence Name", MAX_VALUE AS "Maximum Value",  
INCREMENT_BY AS "Increment Size", LAST_NUMBER AS "Last Number" FROM  
USER_SEQUENCES;
```

3. Write a script to insert two rows into the DEPT table. Name your script lab12_3.sql. Be sure to use the sequence that you created for the ID column. Add two departments named Education and Administration. Confirm your additions. Run the commands in your script.

```
INSERT INTO DEPT (DEPT_ID, DEPT_NAME) VALUES (DEPT_ID_SEQ.NEXTVAL,  
'Education'); INSERT INTO DEPT (DEPT_ID, DEPT_NAME) VALUES  
(DEPT_ID_SEQ.NEXTVAL, 'Administration'); SELECT * FROM DEPT;
```

4. Create a nonunique index on the foreign key column (DEPT_ID) in the EMP table.

```
CREATE INDEX emp_dept_id_idx ON emp(dept_id);
```

A screenshot of a terminal window with a dark background. The text "Index created." is displayed in a green monospace font.

5. Display the indexes and uniqueness that exist in the data dictionary for the EMP table.

```
SELECT INDEX_NAME, TABLE_NAME, UNIQUENESS  
FROM USER_INDEXES  
WHERE TABLE_NAME = 'EMP';
```

EXERCISE-15

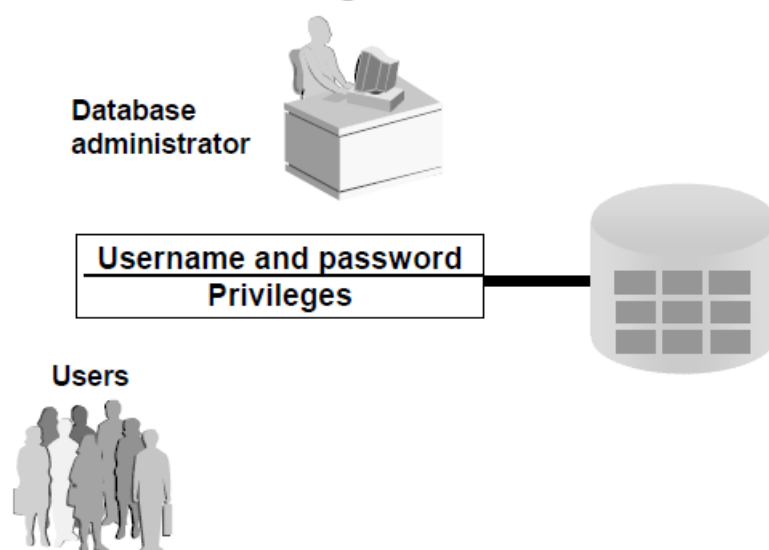
Controlling User Access

Objectives

After the completion of this exercise, the students will be able to do the following:

- Create users
- Create roles to ease setup and maintenance of the security model
- Use the GRANT and REVOKE statements to grant and revoke object privileges
- Create and access database links

Controlling User Access



Controlling User Access

In a multiple-user environment, you want to maintain security of the database access and use. With Oracle server database security, you can do the following:

- Control database access
- Give access to specific objects in the database
- Confirm given and received *privileges* with the Oracle data dictionary
- Create synonyms for database objects

Privileges

- Database security:
 - System security
 - Data security
- System privileges: Gaining access to the database
- Object privileges: Manipulating the content of the database objects
- Schemas: Collections of objects, such as tables, views, and sequences

System Privileges

- More than 100 privileges are available.
- The database administrator has high-level system privileges for tasks such as:
 - Creating new users
 - Removing users
 - Removing tables
 - Backing up tables

Typical DBA Privileges

System Privilege	Operations Authorized
CREATE USER	Grantee can create other Oracle users (a privilege required for a DBA role).
DROP USER	Grantee can drop another user.
DROP ANY TABLE	Grantee can drop a table in any schema.
BACKUP ANY TABLE	Grantee can back up any table in any schema with the export utility.
SELECT ANY TABLE	Grantee can query tables, views, or snapshots in any schema.
CREATE ANY TABLE	Grantee can create tables in any schema.

Creating Users

The DBA creates users by using the CREATE USER statement.

EXAMPLE:

```
CREATE USER scott IDENTIFIED BY tiger;
```

User System Privileges

- Once a user is created, the DBA can grant specific system privileges to a user.
- An application developer, for example, may have the following system privileges:

- CREATE SESSION
- CREATE TABLE
- CREATE SEQUENCE
- CREATE VIEW
- CREATE PROCEDURE

GRANT *privilege* [, *privilege*...]

TO *user* [, *user*| *role*, PUBLIC...];

Typical User Privileges

System Privilege	Operations Authorized
CREATE SESSION	Connect to the database
CREATE TABLE	Create tables in the user's schema
CREATE SEQUENCE	Create a sequence in the user's schema
CREATE VIEW	Create a view in the user's schema
CREATE PROCEDURE	Create a stored procedure, function, or package in the user's schema

In the syntax:

privilege is the system privilege to be granted

user |*role*|PUBLIC is the name of the user, the name of the role, or PUBLIC designates that every user is granted the privilege

Note: Current system privileges can be found in the dictionary view SESSION_PRIVS.

Granting System Privileges

The DBA can grant a user specific system privileges.

GRANT create session, create table, create sequence, create view TO scott;

What is a Role?

A role is a named group of related privileges that can be granted to the user. This method makes it easier to revoke and maintain privileges.

A user can have access to several roles, and several users can be assigned the same role. Roles are typically created for a database application.

Creating and Assigning a Role

First, the DBA must create the role. Then the DBA can assign privileges to the role and users to the role.

Syntax

CREATE ROLE *role*;

In the syntax:

role is the name of the role to be created

Now that the role is created, the DBA can use the GRANT statement to assign users to the role as well as

assign privileges to the role.

Creating and Granting Privileges to a Role

CREATE ROLE manager;

Role created.

GRANT create table, create view TO manager;

Grant succeeded.

GRANT manager TO DEHAAN, KOCHHAR;

Grant succeeded.

- Create a role
- Grant privileges to a role
- Grant a role to users

Changing Your Password

- The DBA creates your user account and initializes your password.
- You can change your password by using the

ALTER USER statement.

ALTER USER scott

IDENTIFIED BY lion;

User altered.

Object Privileges

Object Privilege	Table	View	Sequence	Procedure
ALTER	√		√	
DELETE	√	√		
EXECUTE				√
INDEX	√			
INSERT	√	√		
REFERENCES	√	√		
SELECT	√	√	√	
UPDATE	√	√		

Object Privileges

- Object privileges vary from object to object.
- An owner has all the privileges on the object.
- An owner can give specific privileges on that owner's object.

GRANT *object_priv* [(*columns*)]

ON *object*

TO {*user*|*role*|PUBLIC}

[WITH GRANT OPTION];

In the syntax:

object_priv is an object privilege to be granted

ALL specifies all object privileges

columns specifies the column from a table or view on which privileges are granted

ON *object* is the object on which the privileges are granted

TO identifies to whom the privilege is granted

PUBLIC grants object privileges to all users

WITH GRANT OPTION allows the grantee to grant the object privileges to other users and roles

Granting Object Privileges

- Grant query privileges on the EMPLOYEES table.
- Grant privileges to update specific columns to users and roles.

```
GRANT select
ON employees
TO sue, rich;
```

```
GRANT update (department_name, location_id)
ON departments
TO scott, manager;
```

Using the WITH GRANT OPTION and PUBLIC Keywords

- Give a user authority to pass along privileges.
- Allow all users on the system to query data from Alice's DEPARTMENTS table.

```
GRANT select, insert
ON departments
TO scott
WITH GRANT OPTION;
```

```
.
GRANT select
ON alice.departments
TO PUBLIC;
```

How to Revoke Object Privileges

- You use the REVOKE statement to revoke privileges granted to other users.
- Privileges granted to others through the WITH GRANT OPTION clause are also revoked.

```
REVOKE {privilege [, privilege...]|ALL}  
ON object  
FROM {user[, user...]|role|PUBLIC}  
[CASCADE CONSTRAINTS];
```

In the syntax:

CASCADE is required to remove any referential integrity constraints made to the CONSTRAINTS object by means of the REFERENCES privilege

Revoking Object Privileges

As user Alice, revoke the SELECT and INSERT privileges given to user Scott on the DEPARTMENTS table.

```
REVOKE select, insert  
ON departments  
FROM scott;
```

Find the Solution for the following:

1. What privilege should a user be given to log on to the Oracle Server? Is this a system or an object privilege?

A user should be given the **CREATE SESSION** privilege to log on to the Oracle server.

2. What privilege should a user be given to create tables?

A user should be given the **CREATE TABLE** privilege to create tables.

3. If you create a table, who can pass along privileges to other users on your table?

If you create a table, **you (the table owner)** can pass privileges on your table to other users.

4. You are the DBA. You are creating many users who require the same system privileges. What should you use to make your job easier?

As the DBA, to make granting the same system privileges easier for many users, you should create a **role**.

- Assign the required **system privileges** to the role once.
- Then **grant the role** to multiple users.

This avoids granting privileges individually to each user.

5. What command do you use to change your password?

You use the **ALTER USER** command to change your password.

Example:

ALTER USER your_username IDENTIFIED BY new_password;

6. Grant another user access to your DEPARTMENTS table. Have the user grant you query access to his or her DEPARTMENTS table.

GRANT SELECT ON DEPARTMENTS TO SCOTT; GRANT SELECT ON SCOTT.DEPARTMENTS TO your_username WITH GRANT OPTION;

7. Query all the rows in your DEPARTMENTS table.

SELECT * FROM DEPARTMENTS;

8. Add a new row to your DEPARTMENTS table. Team 1 should add Education as department number 500. Team 2 should add Human Resources department number 510. Query the other team's table.

**INSERT INTO DEPARTMENTS (DEPT_ID, DEPT_NAME)
VALUES (500, 'Education');**

**INSERT INTO DEPARTMENTS (DEPT_ID, DEPT_NAME)
VALUES (510, 'Human Resources');**

SELECT * FROM team2.DEPARTMENT

SELECT * FROM team1.DEPARTMENTS;

9. Query the USER_TABLES data dictionary to see information about the tables that you own.

**SELECT TABLE_NAME, TABLESPACE_NAME, NUM_ROWS, LAST_ANALYZED
FROM USER_TABLES;**

10. Revoke the SELECT privilege on your table from the other team.

REVOKE SELECT ON DEPARTMENTS FROM team2;

11. Remove the row you inserted into the DEPARTMENTS table in step 8 and save the changes.

**DELETE FROM DEPARTMENTS
WHERE DEPT_ID = 500;**

COMMIT;

<u>Evaluation Procedure</u>	<u>Marks awarded</u>
<u>Practice Evaluation (5)</u>	
<u>Viva(5)</u>	
<u>Total (10)</u>	
<u>Faculty Signature</u>	

PL/SQL

PL/SQL

Control Structures

In addition to SQL commands, PL/SQL can also process data using flow of statements. The flow of control statements are classified into the following categories.

- Conditional control - Branching
- Iterative control - looping
- Sequential control

BRANCHING in PL/SQL:

Sequence of statements can be executed on satisfying certain condition.

If statements are being used and different forms of if are:

1. Simple IF

2. ELSIF

3. ELSE IF

SIMPLE IF:

Syntax:

IF condition THEN

 statement1;

 statement2;

END IF;

IF-THEN-ELSE STATEMENT:

Syntax:

IF condition THEN

 statement1;

ELSE

 statement2;

END IF;

ELSIF STATEMENTS:

Syntax:

IF condition1 THEN

 statement1;

ELSIF condition2 THEN

 statement2;

ELSIF condition3 THEN

 statement3;

ELSE

 statementn;

END IF;

NESTED IF :

Syntax:

IF condition THEN

 statement1;

ELSE

 IF condition THEN

 statement2;

 ELSE

 statement3;

 END IF;

END IF;

ELSE

 statement3;

END IF;

SELECTION IN PL/SQL(Sequential Controls)

SIMPLE CASE

Syntax:

CASE SELECTOR

 WHEN Expr1 THEN statement1;

 WHEN Expr2 THEN statement2;

:

ELSE

 Statement n;

END CASE;

SEARCHED CASE:

CASE

 WHEN searchcondition1 THEN statement1;

 WHEN searchcondition2 THEN statement2;

:

:

ELSE

statementn;

END CASE;

ITERATIONS IN PL/SQL

Sequence of statements can be executed any number of times using loop construct.

It is broadly classified into:

- Simple Loop

- For Loop
- While Loop

SIMPLE LOOP

Syntax:

LOOP

statement1;

EXIT [WHEN Condition];

END LOOP;

WHILE LOOP

Syntax:

WHILE condition LOOP

statement1;

statement2;

END LOOP;

FOR LOOP

Syntax:

FOR counter IN [REVERSE]

LowerBound..UpperBound

LOOP

statement1;

statement2;

END LOOP;

PROGRAM 1

Write a PL/SQL block to calculate the incentive of an employee whose ID is 110.

DECLARE

```
v_employee_id NUMBER := 110;
```

```
v_salary      NUMBER;
```

```
v_incentive   NUMBER;
```

```
c_incentive_rate CONSTANT NUMBER := 0.10; -- 10% incentive rate
```

BEGIN

```
-- Retrieve the employee's salary
```

```
SELECT salary
```

```
INTO v_salary
```

```
FROM employees -- Replace 'employees' with your actual employee table name
```

```
WHERE employee_id = v_employee_id;
```

```
-- Calculate the incentive
```

```
v_incentive := v_salary * c_incentive_rate;
```

```
-- Display the incentive
```



```
    DBMS_OUTPUT.PUT_LINE('The incentive for employee ' || v_employee_id || ' is: ' ||  
v_incentive);
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Employee with ID ' || v_employee_id || ' not found.');
```

```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
```

```
END;
```

```
/
```

PROGRAM 2

Write a PL/SQL block to show an invalid case-insensitive reference to a quoted and without quoted user-defined identifier.

```
DECLARE
```

```
    "myVariable" VARCHAR2(20) := 'Hello'; -- Quoted identifier, case-sensitive
```

```
BEGIN
```

```
    -- Attempting to reference with different casing and quotes, which is invalid
```

```
    DBMS_OUTPUT.PUT_LINE("MyVariable");
```

```
END;
```

```
/
```

PROGRAM 3

Write a PL/SQL block to adjust the salary of the employee whose ID 122.

Sample table: employees

DECLARE

 v_employee_id NUMBER := 122;

 v_salary_increase_percentage NUMBER := 10; -- 10% increase

BEGIN

 -- Update the salary for the specified employee

 UPDATE employees

 SET salary = salary * (1 + (v_salary_increase_percentage / 100))

 WHERE employee_id = v_employee_id;

 -- Check if the update was successful and commit the transaction

 IF SQL%ROWCOUNT > 0 THEN

 DBMS_OUTPUT.PUT_LINE('Salary for employee ' || v_employee_id || ' updated successfully.');

```

        COMMIT;

    ELSE

        DBMS_OUTPUT.PUT_LINE('Employee ' || v_employee_id || ' not found or salary not
updated.');
```

```

        ROLLBACK;

    END IF;

EXCEPTION

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);

        ROLLBACK;

END;

/

```

PROGRAM 4

Write a PL/SQL block to create a procedure using the "IS [NOT] NULL Operator" and show AND operator returns TRUE if and only if both operands are TRUE.

```
SET SERVEROUTPUT ON;
```

```
-- Creating the procedure
```

```
CREATE OR REPLACE PROCEDURE check_values IS
```

```
    num1 NUMBER := 10;
```

```
    num2 NUMBER := NULL;
```

```
BEGIN

    -- Using IS [NOT] NULL operator

    IF num1 IS NOT NULL THEN

        DBMS_OUTPUT.PUT_LINE('num1 is NOT NULL');

    END IF;


    IF num2 IS NULL THEN

        DBMS_OUTPUT.PUT_LINE('num2 is NULL');

    END IF;


    -- Demonstrating AND operator

    IF (num1 IS NOT NULL) AND (num2 IS NOT NULL) THEN

        DBMS_OUTPUT.PUT_LINE('Both values are NOT NULL');

    ELSE

        DBMS_OUTPUT.PUT_LINE('AND condition is FALSE because one operand is FALSE');

    END IF;

END;

/

-- Executing the procedure

BEGIN

    check_values;

END;

/
```

PROGRAM 5

Write a PL/SQL block to describe the usage of LIKE operator including wildcard characters and escape character.

```
SET SERVEROUTPUT ON;
```

```
BEGIN
```

```
-- Demonstrating LIKE operator with wildcards and escape character
```

```
DECLARE
```

```
name1 VARCHAR2(20) := 'Sundar';
```

```
name2 VARCHAR2(20) := 'Sumanth';
```

```
name3 VARCHAR2(20) := 'S%raja';
```

```
BEGIN
```

```
-- Using % wildcard: matches any sequence of characters
```

```
IF name1 LIKE 'S%' THEN
```

```
    DBMS_OUTPUT.PUT_LINE(name1 || ' matches pattern S% (starts with S)');
```

```
END IF;
```

```
-- Using _ wildcard: matches exactly one character
```

```
IF name2 LIKE 'Su_anth' THEN
```

```
    DBMS_OUTPUT.PUT_LINE(name2 || ' matches pattern Su_anth (exactly one char in place  
of _)');
```

```
END IF;
```

```
-- Using ESCAPE character: treating % as a normal symbol
```

```
IF name3 LIKE 'S\%raja' ESCAPE '\' THEN
```

```
    DBMS_OUTPUT.PUT_LINE(name3 || ' matches pattern S\%raja using ESCAPE  
character');
```

```
END IF;
```

```
END;
```

```
END;
```

PROGRAM 6

Write a PL/SQL program to arrange the number of two variable in such a way that the small number will store in num_small variable and large number will store in num_large variable.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    num1 NUMBER := 45;
```

```
    num2 NUMBER := 20;
```

```
    num_small NUMBER;
```

```
    num_large NUMBER;
```

```
BEGIN
```

```
    -- Compare and assign
```

```
IF num1 < num2 THEN
```

```
    num_small := num1;
```

```
    num_large := num2;
```

```
ELSE
```

```
    num_small := num2;
```

```
    num_large := num1;
```

```
END IF;
```

```
-- Display the results
```

```
DBMS_OUTPUT.PUT_LINE('Smaller number: ' || num_small);
```

```
DBMS_OUTPUT.PUT_LINE('Larger number : ' || num_large);
```

```
END;
```

```
/
```


PROGRAM 7

Write a PL/SQL procedure to calculate the incentive on a target achieved and display the message either the record updated or not.

```
SET SERVEROUTPUT ON;
```

```
CREATE OR REPLACE PROCEDURE calc_incentive(p_emp_id IN NUMBER) IS
```

```
    v_target  NUMBER;
```

```
    v_incentive NUMBER;
```

```
    v_count   NUMBER;
```

```
BEGIN
```

```
    -- Retrieve target achieved for the employee
```

```
    SELECT target_achieved
```

```
    INTO v_target
```

```
    FROM employee_target
```

```
WHERE emp_id = p_emp_id;
```

```
-- Calculate incentive based on target
```

```
IF v_target >= 70000 THEN
```

```
    v_incentive := v_target * 0.10; -- 10% incentive
```

```
ELSIF v_target >= 40000 THEN
```

```
    v_incentive := v_target * 0.07; -- 7% incentive
```

```
ELSE
```

```
    v_incentive := v_target * 0.05; -- 5% incentive
```

```
END IF;
```

```
-- Update incentive in the table
```

```
UPDATE employee_target
```

```
SET incentive = v_incentive
```

```
WHERE emp_id = p_emp_id;
```

```
-- Check if the record was updated
```

```
v_count := SQL%ROWCOUNT;
```

```
IF v_count > 0 THEN
```

```
    DBMS_OUTPUT.PUT_LINE('Record updated successfully!');
```

```
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || p_emp_id);
```

```
    DBMS_OUTPUT.PUT_LINE('Incentive Amount: ' || v_incentive);
```

```
ELSE
```

```
    DBMS_OUTPUT.PUT_LINE('No record found for the given Employee ID.');
```

```
END IF;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Employee ID ' || p_emp_id || ' not found.');
```

```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
```

```
END;
```

```
/
```

PROGRAM 8

Write a PL/SQL procedure to calculate incentive achieved according to the specific sale limit.

```
SET SERVEROUTPUT ON;
```

```
CREATE OR REPLACE PROCEDURE calc_incentive(p_sales IN NUMBER) IS  
    incentive NUMBER;
```

```
BEGIN
```

```
    -- Check the sales limit and calculate incentive
```

```
    IF p_sales >= 100000 THEN
```

```
        incentive := p_sales * 0.10; -- 10% incentive for sales >= 1,00,000
```

```
    ELSIF p_sales >= 50000 THEN
```

```
        incentive := p_sales * 0.07; -- 7% incentive for sales >= 50,000
```

```
    ELSIF p_sales >= 20000 THEN
```

```
incentive := p_sales * 0.05; -- 5% incentive for sales >= 20,000
```

```
ELSE
```

```
incentive := 0;          -- No incentive for lower sales
```

```
END IF;
```

```
-- Display result
```

```
DBMS_OUTPUT.PUT_LINE('Total Sales      : ' || p_sales);
```

```
DBMS_OUTPUT.PUT_LINE('Incentive Achieved: ' || incentive);
```

```
END;
```

```
/
```

PROGRAM 9

Write a PL/SQL program to count number of employees in department 50 and check whether this department have any vacancies or not. There are 45 vacancies in this department.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_emp_count NUMBER;
```

```
    v_vacancies NUMBER := 45;
```

```
BEGIN
```

```
    -- Count number of employees in department 50
```

```
    SELECT COUNT(*)
```

```
    INTO v_emp_count
```

```
    FROM employees
```

```
WHERE department_id = 50;
```

```
-- Display number of employees
```

```
DBMS_OUTPUT.PUT_LINE('Number of Employees in Department 50: ' ||  
v_emp_count);
```

```
-- Check if there are vacancies
```

```
IF v_emp_count < v_vacancies THEN
```

```
DBMS_OUTPUT.PUT_LINE('Vacancies available: ' || (v_vacancies -  
v_emp_count));
```

```
ELSIF v_emp_count = v_vacancies THEN
```

```
DBMS_OUTPUT.PUT_LINE('No vacancies available — Department is full.');
```

```
ELSE
```

```
DBMS_OUTPUT.PUT_LINE('Overstaffed Department!');
```

```
END IF;
```

```
EXCEPTION
```

```
WHEN NO_DATA_FOUND THEN
```

```
DBMS_OUTPUT.PUT_LINE('Department 50 not found.');
```

```
WHEN OTHERS THEN
```

```
DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
```

```
END;
```

```
/
```

PROGRAM 10

Write a PL/SQL program to count number of employees in a specific department and check whether this department have any vacancies or not. If any vacancies, how many vacancies are in that department.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
v_dept_id  NUMBER := &dept_id; -- user enters department ID
```

```
v_emp_count NUMBER;
```

```
v_total_posts NUMBER := 45;      -- total available positions
```

```
BEGIN
```

```
-- Count number of employees in the given department
```

```
SELECT COUNT(*)
```



```
INTO v_emp_count
```

```
FROM employees
```

```
WHERE department_id = v_dept_id;
```

```
-- Display number of employees
```

```
DBMS_OUTPUT.PUT_LINE('Department ID: ' || v_dept_id);
```

```
DBMS_OUTPUT.PUT_LINE('Number of Employees: ' || v_emp_count);
```

```
-- Compare with total vacancies
```

```
IF v_emp_count < v_total_posts THEN
```

```
    DBMS_OUTPUT.PUT_LINE('Vacancies Available: ' || (v_total_posts -  
v_emp_count));
```

```
ELSIF v_emp_count = v_total_posts THEN
```

```
    DBMS_OUTPUT.PUT_LINE('No vacancies — Department is full.');
```

```
ELSE
```

```
    DBMS_OUTPUT.PUT_LINE('Overstaffed Department!');
```

```
END IF;
```

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Department ' || v_dept_id || ' not found.');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);

END;

/

PROGRAM 11

Write a PL/SQL program to display the employee IDs, names, job titles, hire dates, and salaries of all employees.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
-- Declare variables to hold employee data
```

```
v_emp_id    employees.employee_id%TYPE;
```

```
v_first_name employees.first_name%TYPE;
```

```
v_last_name  employees.last_name%TYPE;
```

```
v_job_id     employees.job_id%TYPE;
```

```
v_hire_date  employees.hire_date%TYPE;
```

```
v_salary     employees.salary%TYPE;
```

-- Cursor to fetch all employee details

CURSOR emp_cur IS

SELECT employee_id, first_name, last_name, job_id, hire_date, salary

FROM employees;

BEGIN

DBMS_OUTPUT.PUT_LINE('EMP_ID | EMPLOYEE_NAME | JOB_TITLE |
HIRE_DATE | SALARY');

DBMS_OUTPUT.PUT_LINE('-----');

-- Loop through each record

FOR emp_rec IN emp_cur LOOP

DBMS_OUTPUT.PUT_LINE(

emp_rec.employee_id || ' | ' ||

emp_rec.first_name || ' ' || emp_rec.last_name || ' | ' ||

emp_rec.job_id || ' | ' ||

TO_CHAR(emp_rec.hire_date, 'DD-MON-YYYY') || ' | ' ||

emp_rec.salary

```
);  
  
END LOOP;  
  
END;  
  
/
```

PROGRAM 12

Write a PL/SQL program to display the employee IDs, names, and department names of all employees.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
-- Variables to hold employee and department data
```

```
v_emp_id    employees.employee_id%TYPE;
```

```
v_first_name employees.first_name%TYPE;
```

```
v_last_name  employees.last_name%TYPE;
```

```
v_dept_name  departments.department_name%TYPE;
```

```
-- Cursor to retrieve employee and department details
```

CURSOR emp_cur IS

SELECT e.employee_id, e.first_name, e.last_name, d.department_name

FROM employees e

JOIN departments d

ON e.department_id = d.department_id

ORDER BY e.employee_id;

BEGIN

DBMS_OUTPUT.PUT_LINE('EMP_ID | EMPLOYEE_NAME |
DEPARTMENT_NAME');

DBMS_OUTPUT.PUT_LINE('-----');

-- Loop through each record from the cursor

FOR emp_rec IN emp_cur LOOP

DBMS_OUTPUT.PUT_LINE(

emp_rec.employee_id || ' | ' ||

emp_rec.first_name || ' ' || emp_rec.last_name || ' | ' ||

emp_rec.department_name

);

```
END LOOP;  
  
END;  
  
/
```

PROGRAM 13

Write a PL/SQL program to display the job IDs, titles, and minimum salaries of all jobs.

```
SET SERVEROUTPUT ON;  
  
DECLARE  
  
    -- Variables to hold job details  
  
    v_job_id    jobs.job_id%TYPE;  
  
    v_job_title jobs.job_title%TYPE;  
  
    v_min_salary jobs.min_salary%TYPE;  
  
    -- Cursor to fetch all job details  
  
    CURSOR job_cur IS  
  
        SELECT job_id, job_title, min_salary  
  
        FROM jobs
```

```
ORDER BY job_id;
```

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE('JOB_ID | JOB_TITLE | MIN_SALARY');
```

```
DBMS_OUTPUT.PUT_LINE('-----');
```

```
-- Loop through each record in the cursor
```

```
FOR job_rec IN job_cur LOOP
```

```
    DBMS_OUTPUT.PUT_LINE(
```

```
        job_rec.job_id || ' | ' ||
```

```
        job_rec.job_title || ' | ' ||
```

```
        job_rec.min_salary
```

```
    );
```

```
END LOOP;
```

```
END;
```

```
/
```


PROGRAM 14

Write a PL/SQL program to display the employee IDs, names, and job history start dates of all employees.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
-- Cursor to fetch employee and job history details
```

```
CURSOR emp_job_cur IS
```

```
    SELECT e.employee_id, e.first_name, e.last_name, jh.start_date
```

```
    FROM employees e
```

```
    JOIN job_history jh
```

```
    ON e.employee_id = jh.employee_id
```

```
    ORDER BY e.employee_id;
```

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE('EMP_ID | EMPLOYEE_NAME |  
JOB_HISTORY_START_DATE');
```

```
DBMS_OUTPUT.PUT_LINE('-----');
```

```
-- Loop through each record from the cursor
```

```
FOR emp_rec IN emp_job_cur LOOP
```

```
DBMS_OUTPUT.PUT_LINE(  
    emp_rec.employee_id || ' | ' ||  
    emp_rec.first_name || ' ' || emp_rec.last_name || ' | ' ||  
    TO_CHAR(emp_rec.start_date, 'DD-MON-YYYY'));
```

```
END LOOP;
```

```
END;
```

```
/
```

PROGRAM 15

Write a PL/SQL program to display the employee IDs, names, and job history end dates of all employees.

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
-- Cursor to fetch employee and job history end dates
```

```
CURSOR emp_job_cur IS
```

```
    SELECT e.employee_id, e.first_name, e.last_name, jh.end_date
```

```
    FROM employees e
```

```
    JOIN job_history jh
```

```
    ON e.employee_id = jh.employee_id
```

```
    ORDER BY e.employee_id;
```

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE('EMP_ID | EMPLOYEE_NAME |  
JOB_HISTORY_END_DATE');
```

```
DBMS_OUTPUT.PUT_LINE('-----');
```

```
-- Loop through each record from the cursor
```

```
FOR emp_rec IN emp_job_cur LOOP
```

```
DBMS_OUTPUT.PUT_LINE(  
    emp_rec.employee_id || ' | ' ||
```

```
    emp_rec.first_name || ' ' || emp_rec.last_name || ' | ' ||
```

```
    TO_CHAR(emp_rec.end_date, 'DD-MON-YYYY'));
```

```
END LOOP;
```

```
END;
```

```
/
```

Evaluation Procedure	Marks awarded
PL/SQL Procedure(5)	
Program/Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

EXERCISE-16

PROCEDURES AND FUNCTIONS

PROCEDURES

DEFINITION

A procedure or function is a logically grouped set of SQL and PL/SQL statements that perform a specific task. They are essentially sub-programs. Procedures and functions are made up of,

- Declarative part
- Executable part
- Optional exception handling part

These procedures and functions do not show the errors.

KEYWORDS AND THEIR PURPOSES

REPLACE: It recreates the procedure if it already exists.

PROCEDURE: It is the name of the procedure to be created.

ARGUMENT: It is the name of the argument to the procedure. Paranthesis can be omitted if no arguments are present.

IN: Specifies that a value for the argument must be specified when calling the procedure ie. used to pass values to a sub-program. This is the default parameter.

OUT: Specifies that the procedure passes a value for this argument back to it's calling environment after execution ie. used to return values to a caller of the sub-program.

INOUT: Specifies that a value for the argument must be specified when calling the procedure and that procedure passes a value for this argument back to it's calling environment after execution.

RETURN: It is the datatype of the function's return value because every function must return a value, this clause is required.

PROCEDURES – SYNTAX

```
create or replace procedure <procedure name> (argument {in,out,inout} datatype ) {is,as}
variable declaration;
constant declaration;
begin
PL/SQL subprogram body;
exception
exception PL/SQL block;
end;
```

FUNCTIONS – SYNTAX

```
create or replace function <function name> (argument in datatype,.....) return datatype {is,as}
variable declaration;
constant declaration;
begin
PL/SQL subprogram body;
exception
exception PL/SQL block;
end;
```

CREATING THE TABLE 'ITITEMS' AND DISPLAYING THE CONTENTS

```
SQL> create table ititems(itemid number(3), actualprice number(5), ordid number(4), prodid
number(4));
Table created.
```

```
SQL> insert into ititems values(101, 2000, 500, 201);
```

1 row created.

SQL> insert into ititems values(102, 3000, 1600, 202);
1 row created.

SQL> insert into ititems values(103, 4000, 600, 202);
1 row created.

SQL> select * from ititems;

ITEMID	ACTUALPRICE	ORDID	PRODID
101	2000	500	201
102	3000	1600	202
103	4000	600	202

PROGRAM FOR GENERAL PROCEDURE – SELECTED RECORD’S PRICE IS INCREMENTED BY 500 , EXECUTING THE PROCEDURE CREATED AND DISPLAYING THE UPDATED TABLE

SQL> create procedure itsum(identity number, total number) is price number;
2 null_price exception;
3 begin
4 select actualprice into price from ititems where itemid=identity;
5 if price is null then
6 raise null_price;
7 else
8 update ititems set actualprice=actualprice+total where itemid=identity;
9 end if;
10 exception
11 when null_price then
12 dbms_output.put_line('price is null');
13 end;
14 /

Procedure created.

SQL> exec itsum(101, 500);
PL/SQL procedure successfully completed.

SQL> select * from ititems;

ITEMID	ACTUALPRICE	ORDID	PRODID
101	2500	500	201
102	3000	1600	202
103	4000	600	202

PROCEDURE FOR ‘IN’ PARAMETER – CREATION, EXECUTION

SQL> set serveroutput on;

```
SQL> create procedure yyy (a IN number) is price number;
  2 begin
  3 select actualprice into price from ititems where itemid=a;
  4 dbms_output.put_line('Actual price is ' || price);
  5 if price is null then
  6 dbms_output.put_line('price is null');
  7 end if;
  8 end;
  9 /
```

Procedure created.

SQL> exec yyy(103);

Actual price is 4000

PL/SQL procedure successfully completed.

PROCEDURE FOR 'OUT' PARAMETER – CREATION, EXECUTION

SQL> set serveroutput on;

```
SQL> create procedure zzz (a in number, b out number) is identity number;
  2 begin
  3 select ordid into identity from ititems where itemid=a;
  4 if identity<1000 then
  5 b:=100;
  6 end if;
  7 end;
  8 /
```

Procedure created.

SQL> declare

```
  2 a number;
  3 b number;
  4 begin
  5 zzz(101,b);
  6 dbms_output.put_line('The value of b is '|| b);
  7 end;
  8 /
```

The value of b is 100

PL/SQL procedure successfully completed.

PROCEDURE FOR 'INOUT' PARAMETER – CREATION, EXECUTION

SQL> create procedure itit (a in out number) is


```
2 begin
3 a:=a+1;
4 end;
5 /
Procedure created.
```

```
SQL> declare
2 a number:=7;
3 begin
4 itit(a);
5 dbms_output.put_line('The updated value is '||a);
6 end;
7 /
```

The updated value is 8

PL/SQL procedure successfully completed.

CREATE THE TABLE 'ITTRAIN' TO BE USED FOR FUNCTIONS

```
SQL>create table ittrain ( tno number(10), tfare number(10));
Table created.
```

```
SQL>insert into ittrain values (1001, 550);
1 row created.
```

```
SQL>insert into ittrain values (1002, 600);
1 row created.
```

```
SQL>select * from ittrain;
```

TNO	TFARE
-----	-----
1001	550
1002	600

PROGRAM FOR FUNCTION AND IT'S EXECUTION

```
SQL> create function aaa (trainnumber number) return number is
2 trainfunction ittrain.tfare % type;
3 begin
4 select tfare into trainfunction from ittrain where tno=trainnumber;
5 return(trainfunction);
6 end;
7 /
```

Function created.

```
SQL> set serveroutput on;
```

```

SQL> declare
2 total number;
3 begin
4 total:=aaa (1001);
5 dbms_output.put_line('Train fare is Rs. '||total);
6 end;
7 /

```

Train fare is Rs.550

PL/SQL procedure successfully completed.

Program 1

FACTORIAL OF A NUMBER USING FUNCTION

```

CREATE OR REPLACE FUNCTION factorial(n IN PLS_INTEGER) RETURN NUMBER IS r
NUMBER := 1;
BEGIN
  IF n < 0 THEN RAISE_APPLICATION_ERROR(-20001,'n>=0'); END IF;
  FOR i IN 2..n LOOP r := r * i; END LOOP;
  RETURN r;
END;
/
-- Test
SET SERVEROUTPUT ON;
BEGIN DBMS_OUTPUT.PUT_LINE('5! = '|| factorial(5)); END; /

```

Program 2

Write a PL/SQL program using Procedures IN,INOUT,OUT parameters to retrieve the corresponding book information in library

```
-- Assumes BOOKS(BOOK_ID, TITLE, AUTHOR, PRICE, AVAILABLE_COPIES)
CREATE OR REPLACE PROCEDURE get_book_info( p_book_id IN
BOOKS.BOOK_ID%TYPE, p_title OUT BOOKS.TITLE%TYPE, p_author OUT
BOOKS.AUTHOR%TYPE, p_price OUT BOOKS.PRICE%TYPE,
p_avail IN OUT BOOKS.AVAILABLE_COPIES%TYPE
) IS
BEGIN
    SELECT title, author, price, available_copies
    INTO p_title, p_author, p_price, p_avail
    FROM books WHERE book_id = p_book_id; EXCEPTION WHEN NO_DATA_FOUND
THEN
```

```
    p_title:='NOT FOUND'; p_author:=NULL; p_price:=0; p_avail:=0;
END;
/
-- Test
SET SERVEROUTPUT ON; DECLARE
    v_title BOOKS.TITLE%TYPE; v_author BOOKS.AUTHOR%TYPE; v_price
BOOKS.PRICE%TYPE; v_avail NUMBER := -1; BEGIN
    get_book_info(101, v_title, v_author, v_price, v_avail);
    DBMS_OUTPUT.PUT_LINE(v_title||' | '||NVL(v_author,'NA')||' |
'||v_price||' | '||v_avail);
END;
/
```

Evaluation Procedure	Marks awarded
PL/SQL Procedure(5)	
Program/Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

EXERCISE-17

TRIGGER

DEFINITION

A trigger is a statement that is executed automatically by the system as a side effect of a modification to the database. The parts of a trigger are,

- **Trigger statement:** Specifies the DML statements and fires the trigger body. It also specifies the table to which the trigger is associated.
- **Trigger body or trigger action:** It is a PL/SQL block that is executed when the triggering statement is used.
- **Trigger restriction:** Restrictions on the trigger can be achieved

The different uses of triggers are as follows,

- *To generate data automatically*
- *To enforce complex integrity constraints*
- *To customize complex securing authorizations*
- *To maintain the replicate table*
- *To audit data modifications*

TYPES OF TRIGGERS

The various types of triggers are as follows,

- **Before:** It fires the trigger before executing the trigger statement.
- **After:** It fires the trigger after executing the trigger statement
- .
- **For each row:** It specifies that the trigger fires once per row
- .
- **For each statement:** This is the default trigger that is invoked. It specifies that the trigger fires once per statement.

VARIABLES USED IN TRIGGERS

- :new
- :old

These two variables retain the new and old values of the column updated in the database. The values in these variables can be used in the database triggers for data manipulation

SYNTAX

```
create or replace trigger triggername [before/after] {DML statements}
on [tablename] [for each row/statement]
begin
-----
-----
-----
exception
end;
```

USER DEFINED ERROR MESSAGE

The package “raise_application_error” is used to issue the user defined error messages

Syntax: raise_application_error(error number, ‘error message’);

The error number can lie between -20000 and -20999.

The error message should be a character string.

TO CREATE THE TABLE ‘ITEMPLS’

SQL> create table itempls (ename varchar2(10), eid number(5), salary number(10));
Table created.

SQL> insert into itempls values('xxx',11,10000);
1 row created.

SQL> insert into itempls values('yyy',12,10500);
1 row created.

SQL> insert into itempls values('zzz',13,15500);
1 row created.

SQL> select * from itempls;
ENAME EID SALARY

xxx 11 10000
yyy 12 10500
zzz 13 15500

TO CREATE A SIMPLE TRIGGER THAT DOES NOT ALLOW INSERT UPDATE AND DELETE OPERATIONS ON THE TABLE

SQL> create trigger ittrigg before insert or update or delete on itempls for each row
2 begin
3 raise_application_error(-20010,'You cannot do manipulation');
4 end;
5
6 /
Trigger created.

SQL> insert into itempls values('aaa',14,34000);
insert into itempls values('aaa',14,34000)
*

ERROR at line 1:
ORA-20010: You cannot do manipulation
ORA-06512: at "STUDENT.ITTRIGG", line 2
ORA-04088: error during execution of trigger 'STUDENT.ITTRIGG'

SQL> delete from itempls where ename='xxx';
delete from itempls where ename='xxx'
*

ERROR at line 1:
ORA-20010: You cannot do manipulation
ORA-06512: at "STUDENT.ITTRIGG", line 2
ORA-04088: error during execution of trigger 'STUDENT.ITTRIGG'

```
SQL> update itempls set eid=15 where ename='yyy';
update itempls set eid=15 where ename='yyy'
*
```

ERROR at line 1:

ORA-20010: You cannot do manipulation

ORA-06512: at "STUDENT.ITTRIGG", line 2

ORA-04088: error during execution of trigger 'STUDENT.ITTRIGG'

TO DROP THE CREATED TRIGGER

```
SQL> drop trigger ittrigg;
```

Trigger dropped.

TO CREATE A TRIGGER THAT RAISES AN USER DEFINED ERROR MESSAGE AND DOES NOT ALLOW UPDATION AND INSERTION

```
SQL> create trigger ittriggs before insert or update of salary on itempls for each row
```

```
2 declare
3 triggssal itempls.salary%type;
4 begin
5 select salary into triggssal from itempls where eid=12;
6 if(:new.salary>triggssal or :new.salary<triggssal) then
7 raise_application_error(-20100,'Salary has not been changed');
8 end if;
9 end;
10 /
```

Trigger created.

```
SQL> insert into itempls values ('bbb',16,45000);
insert into itempls values ('bbb',16,45000)
*
```

ERROR at line 1:

ORA-04098: trigger 'STUDENT.ITTRIGGS' is invalid and failed re-validation

```
SQL> update itempls set eid=18 where ename='zzz';
update itempls set eid=18 where ename='zzz'
*
```

ERROR at line 1:

ORA-04298: trigger 'STUDENT.ITTRIGGS' is invalid and failed re-validation

Cursor for loop

- ☐ Explicit cursor
- ☐ Implicit cursor

TO CREATE THE TABLE 'SSEMP'

```
SQL> create table ssemp( eid number(10), ename varchar2(20), job varchar2(20), sal number
(10),dnonumber(5));
Table created.
```

```
SQL> insert into ssemp values(1,'nala','lecturer',34000,11);
1 row created.
```

```
SQL> insert into ssemp values(2,'kala',' seniorlecturer',20000,12);
1 row created.
```

```
SQL> insert into ssemp values(5,'ajay','lecturer',30000,11);
1 row created.
```

```
SQL> insert into ssemp values(6,'vijay','lecturer',18000,11);
1 row created.
```

```
SQL> insert into ssemp values(3,'nila','professor',60000,12);
1 row created.
```

```
SQL> select * from ssemp;
```

EID	ENAME	JOB	SAL	DNO
1	nala	lecturer	34000	11
2	kala	seniorlecturer	20000	12
5	ajay	lecturer	30000	11
6	vijay	lecturer	18000	11
3	nila	professor	60000	12

EXTRA PROGRAMS

TO WRITE A PL/SQL BLOCK TO DISPLAY THE EMPLOYEE ID AND EMPLOYEE NAME USING CURSOR FOR LOOP

```
SQL> set serveroutput on;
```

```
SQL> declare
```

```
2 begin
```

```
3 for emy in (select eid,ename from ssemp)
```

```
4 loop
```

```
5 dbms_output.put_line('Employee id and employee name are '|| emy.eid 'and' || emy.ename);
```

```
6 end loop;
```

```
7 end;
```

```
8 /
```

```
Employee id and employee name are 1 and nala
```

Employee id and employee name are 2 and kala
Employee id and employee name are 5 and ajay
Employee id and employee name are 6 and vijay
Employee id and employee name are 3 and nila

PL/SQL procedure successfully completed.

TO WRITE A PL/SQL BLOCK TO UPDATE THE SALARY OF ALL EMPLOYEES WHERE DEPARTMENT NO IS 11 BY 5000 USING CURSOR FOR LOOP AND TO DISPLAY THE UPDATED TABLE

```
SQL> set serveroutput on;
SQL> declare
  2 cursor cem is select eid,ename,sal,dno from ssempp where dno=11;
  3 begin
  4 --open cem;
  5 for rem in cem
  6 loop
  7 update ssempp set sal=rem.sal+5000 where eid=rem.eid;
  8 end loop;
  9 --close cem;
 10 end;
 11 /
```

PL/SQL procedure successfully completed.

```
SQL> select * from ssempp;
```

EID	ENAME	JOB	SAL	DNO
1	nala	lecturer	39000	11
2	kala	seniorlecturer	20000	12
5	ajay	lecturer	35000	11
6	vijay	lecturer	23000	11
3	nila	professor	60000	12

TO WRITE A PL/SQL BLOCK TO DISPLAY THE EMPLOYEE ID AND EMPLOYEE NAME WHERE DEPARTMENT NUMBER IS 11 USING EXPLICIT CURSORS

```
1 declare
2 cursor cenl is select eid,sal from ssempp where dno=11;
3 ecode ssempp.eid%type;
4 esal empp.sal%type;
5 begin
6 open cenl;
7 loop
```

```

8 fetch cenl into ecode,esal;
9 exit when cenl%notfound;
10 dbms_output.put_line(' Employee code and employee salary are' || ecode 'and' || esal);
11 end loop;
12 close cenl;
13* end;

```

SQL> /

Employee code and employee salary are 1 and 39000

Employee code and employee salary are 5 and 35000

Employee code and employee salary are 6 and 23000

PL/SQL procedure successfully completed.

TO WRITE A PL/SQL BLOCK TO UPDATE THE SALARY BY 5000 WHERE THE JOB IS LECTURER , TO CHECK IF UPDATES ARE MADE USING IMPLICIT CURSORS AND TO DISPLAY THE UPDATED TABLE

SQL> declare

```

2 county number;
3 begin
4 update ssempp set sal=sal+10000 where job='lecturer';
5 county:= sql%rowcount;
6 if county > 0 then
7 dbms_output.put_line('The number of rows are '|| county);
8 end if;
9 if sql %found then
10 dbms_output.put_line('Employee record modification successful');
11 else if sql%notfound then
12 dbms_output.put_line('Employee record is not found');
13 end if;
14 end if;
15 end;
16 /

```

The number of rows are 3

Employee record modification successful

PL/SQL procedure successfully completed.

SQL> select * from ssempp;

EID	ENAME	JOB	SAL	DNO
1	nala	lecturer	44000	11
2	kala	seniorlecturer	20000	12

5	ajay	lecturer	40000	11
6	vijay	lecturer	28000	11
3	nila	professor	60000	12

PROGRAMS

TO DISPLAY HELLO MESSAGE

```
SQL> set serveroutput on;
SQL> declare
  2 a varchar2(20);
  3 begin
  4 a:='Hello';
  5 dbms_output.put_line(a);
  6 end;
  7 /
Hello
```

PL/SQL procedure successfully completed.

TO INPUT A VALUE FROM THE USER AND DISPLAY IT

```
SQL> set serveroutput on;
SQL> declare
  2 a varchar2(20);
  3 begin
  4 a:=&a;
  5 dbms_output.put_line(a);
  6 end;
  7 /
Enter value for a: 5
old  4: a:=&a;
new  4: a:=5;
5
```

PL/SQL procedure successfully completed.

GREATEST OF TWO NUMBERS

```
SQL> set serveroutput on;
```

```
SQL> declare
  2 a number(7);
  3 b number(7);
  4 begin
  5 a:=&a;
```

```

6 b:=&b;
7 if(a>b) then
8 dbms_output.put_line (' The greater of the two is'|| a);
9 else
10 dbms_output.put_line (' The greater of the two is'|| b);
11 end if;
12 end;
13 /

```

Enter value for a: 5

old 5: a:=&a;

new 5: a:=5;

Enter value for b: 9

old 6: b:=&b;

new 6: b:=9;

The greater of the two is9

PL/SQL procedure successfully completed.

GREATEST OF THREE NUMBERS

SQL> set serveroutput on;

SQL> declare

```

2 a number(7);
3 b number(7);
4 c number(7);
5 begin
6 a:=&a;
7 b:=&b;
8 c:=&c;
9 if(a>b and a>c) then
10 dbms_output.put_line (' The greatest of the three is ' || a);
11 else if (b>c) then
12 dbms_output.put_line (' The greatest of the three is ' || b);
13 else
14 dbms_output.put_line (' The greatest of the three is ' || c);
15 end if;
16 end if;
17 end;
18 /

```

Enter value for a: 5

old 6: a:=&a;

new 6: a:=5;

Enter value for b: 7

old 7: b:=&b;

new 7: b:=7;

Enter value for c: 1
old 8: c:=&c;
new 8: c:=1;
The greatest of the three is 7

PL/SQL procedure successfully completed.

PRINT NUMBERS FROM 1 TO 5 USING SIMPLE LOOP

SQL> set serveroutput on;

```
SQL> declare
  2 a number:=1;
  3 begin
  4 loop
  5 dbms_output.put_line (a);
  6 a:=a+1;
  7 exit when a>5;
  8 end loop;
  9 end;
 10 /
1
2
3
4
5
```

PL/SQL procedure successfully completed.

PRINT NUMBERS FROM 1 TO 4 USING WHILE LOOP

SQL> set serveroutput on;

```
SQL> declare
  2 a number:=1;
  3 begin
  4 while(a<5)
  5 loop
  6 dbms_output.put_line (a);
  7 a:=a+1;
  8 end loop;
  9 end;
 10 /
1
```

2
3
4

PL/SQL procedure successfully completed.

PRINT NUMBERS FROM 1 TO 5 USING FOR LOOP

SQL> set serveroutput on;

SQL> declare

```
2 a number:=1;
3 begin
4 for a in 1..5
5 loop
6 dbms_output.put_line (a);
7 end loop;
8 end;
9 /
```

1
2
3
4
5

PL/SQL procedure successfully completed.

PRINT NUMBERS FROM 1 TO 5 IN REVERSE ORDER USING FOR LOOP

SQL> set serveroutput on;

SQL> declare

```
2 a number:=1;
3 begin
4 for a in reverse 1..5
5 loop
6 dbms_output.put_line (a);
7 end loop;
8 end;
9 /
```

5
4
3
2
1

PL/SQL procedure successfully completed.

TO CALCULATE AREA OF CIRCLE

SQL> set serveroutput on;

SQL> declare

```
2 pi constant number(4,2):=3.14;
3 a number(20);
4 r number(20);
5 begin
```

```

6 r:=&r;
7 a:= pi* power(r,2);
8 dbms_output.put_line (' The area of circle is ' || a);
9 end;
10 /

```

Enter value for r: 2

old 6: r:=&r;

new 6: r:=2;

The area of circle is 13

PL/SQL procedure successfully completed.

TO CREATE SACCOUNT TABLE

```
SQL> create table saccount ( accno number(5), name varchar2(20), bal number(10));
```

Table created.

```
SQL> insert into saccount values ( 1,'mala',20000);
```

1 row created.

```
SQL> insert into saccount values (2,'kala',30000);
```

1 row created.

```
SQL> select * from saccount;
```

ACCNO	NAME	BAL
1	mala	20000
2	kala	30000

```
SQL> set serveroutput on;
```

```
SQL> declare
```

```
2 a_bal number(7);
```

```
3 a_no varchar2(20);
```

```
4 debit number(7):=2000;
```

```
5 minamt number(7):=500;
```

```
6 begin
```

```
7 a_no:=&a_no;
```

```
8 select bal into a_bal from saccount where accno= a_no;
```

```
9 a_bal:= a_bal-debit;
```

```
10 if (a_bal > minamt) then
```

```
11 update saccount set bal=bal-debit where accno=a_no;
```

```
12 end if;
```

```
13 end;
```

```
14
```

```
15 /
```

Enter value for a_no: 1

old 7: a_no:=&a_no;

new 7: a_no:=1;

PL/SQL procedure successfully completed.

```
SQL> select * from saccount;
```

ACCNO	NAME	BAL
-------	------	-----


```
-----
1 mala          18000
2 kala          30000
```

TO CREATE TABLE SROUTES

```
SQL> create table sroutes ( rno number(5), origin varchar2(20), destination varchar2(20), fare
numbe
```

```
r(10), distance number(10));
```

```
Table created.
```

```
SQL> insert into sroutes values ( 2, 'chennai', 'dindugal', 400,230);
```

```
1 row created.
```

```
SQL> insert into sroutes values ( 3, 'chennai', 'madurai', 250,300);
```

```
1 row created.
```

```
SQL> insert into sroutes values ( 6, 'thanjavur', 'palani', 350,370);
```

```
1 row created.
```

```
SQL> select * from sroutes;
```

RNO	ORIGIN	DESTINATION	FARE	DISTANCE
2	chennai	dindugal	400	230
3	chennai	madurai	250	300
6	thanjavur	palani	350	370

```
SQL> set serveroutput on;
```

```
SQL> declare
```

```
2 route sroutes.rno % type;
```

```
3 fares sroutes.fare % type;
```

```
4 dist sroutes.distance % type;
```

```
5 begin
```

```
6 route:=&route;
```

```
7 select fare, distance into fares , dist from sroutes where rno=route;
```

```
8 if (dist < 250) then
```

```
9 update sroutes set fare=300 where rno=route;
```

```
10 else if dist between 250 and 370 then
```

```
11 update sroutes set fare=400 where rno=route;
```

```
12 else if (dist > 400) then
```

```
13 dbms_output.put_line('Sorry');
```

```
14 end if;
```

```
15 end if;
```

```
16 end if;
```

```
17 end;
```

```
18 /
```

```
Enter value for route: 3
```

```
old 6: route:=&route;
```

```
new 6: route:=3;
```

PL/SQL procedure successfully completed.

SQL> select * from sroutes;

RNO	ORIGIN	DESTINATION	FARE	DISTANCE
2	chennai	dindugal	400	230
3	chennai	madurai	400	300
6	thanjavur	palani	350	370

TO CREATE SCALCULATE TABLE

SQL> create table scalculate (radius number(3), area number(5,2));

Table created.

SQL> desc scalculate;

Name	Null?	Type
RADIUS		NUMBER(3)
AREA		NUMBER(5,2)

SQL> set serveroutput on;

SQL> declare

```
2 pi constant number(4,2):=3.14;
3 area number(5,2);
4 radius number(3);
5 begin
6 radius:=3;
7 while (radius <=7)
8 loop
9 area:= pi* power(radius,2);
10 insert into scalculate values (radius,area);
11 radius:=radius+1;
12 end loop;
13 end;
14 /
```

PL/SQL procedure successfully completed.

SQL> select * from scalculate;

RADIUS	AREA
3	28.26
4	50.24
5	78.5
6	113.04

TO CALCULATE FACTORIAL OF A GIVEN NUMBER

```

SQL> set serveroutput on;
SQL> declare
  2 f number(4):=1;
  3 i number(4);
  4 begin
  5 i:=&i;
  6 while(i>=1)
  7 loop
  8 f:=f*i;
  9 i:=i-1;
 10 end loop;
 11 dbms_output.put_line('The value is ' || f);
 12 end;
 13 /
Enter value for i: 5
old  5: i:=&i;
new  5: i:=5;
The value is 120

```

PL/SQL procedure successfully completed.

Program 1

Write a code in PL/SQL to develop a trigger that enforces referential integrity by preventing the deletion of a parent record if child records exist.

```

-- PARENT(PID PK), CHILD(PID FK->PARENT.PID)

CREATE OR REPLACE TRIGGER trg_parent_no_delete

BEFORE DELETE ON parent FOR EACH ROW

DECLARE v_cnt INTEGER;

BEGIN

```

```
SELECT COUNT(*) INTO v_cnt FROM child WHERE pid = :OLD.pid;  
  
IF v_cnt > 0 THEN RAISE_APPLICATION_ERROR(-20001,'Child rows exist');  
END IF;  
  
END; /
```

Program 2

Write a code in PL/SQL to create a trigger that checks for duplicate values in a specific column and raises an exception if found.

```
-- MEMBERS(MID PK, EMAIL)
```

```
CREATE OR REPLACE TRIGGER trg_members_email_unique
```

```
BEFORE INSERT OR UPDATE OF email ON members FOR EACH ROW
```

```
DECLARE v_cnt INTEGER;
```

```
BEGIN
```

```
IF :NEW.email IS NULL THEN RETURN; END IF;
```

```
SELECT COUNT(*) INTO v_cnt FROM members
```

```
WHERE email = :NEW.email AND NVL(mid,-1) <> NVL(:NEW.mid,-1);
```

```
IF v_cnt > 0 THEN RAISE_APPLICATION_ERROR(-20002,'Duplicate EMAIL');  
END IF;
```

END; /

Program 3

Write a code in PL/SQL to create a trigger that restricts the insertion of new rows if the total of a column's values exceeds a certain threshold.

```
-- SALES(SID PK, AMOUNT) -- limit: 100000
```

```
CREATE OR REPLACE TRIGGER trg_sales_total_cap
```

```
FOR INSERT ON sales COMPOUND TRIGGER
```

```
  g_total NUMBER; g_limit CONSTANT NUMBER := 100000;
```

```
BEFORE STATEMENT IS
```

```
BEGIN SELECT NVL(SUM(amount),0) INTO g_total FROM sales; END;
```

```
BEFORE EACH ROW IS
```

```
BEGIN IF g_total + :NEW.amount > g_limit THEN
```

```
  RAISE_APPLICATION_ERROR(-
```

```
20003,'Limit exceeded'); END IF;
```

```
  g_total := g_total + :NEW.amount; END;
```

```
END; /
```

Program 4

Write a code in PL/SQL to design a trigger that captures changes made to specific columns and logs them in an audit table.

```
-- EMP(EMPID PK, SAL, DEPTNO); EMP_AUDIT(AUD_ID, EMPID, COL_NAME,  
OLD_VAL,
```

```
NEW_VAL, WHO, WHEN_)
```

```
CREATE OR REPLACE TRIGGER trg_emp_col_audit
```

```
BEFORE UPDATE OF sal, deptno ON emp FOR EACH ROW
```

```
BEGIN
```

```
IF NVL(:OLD.sal,-1) <> NVL(:NEW.sal,-1) THEN
```

```
INSERT INTO emp_audit VALUES
```



```
(emp_audit_seq.NEXTVAL,:OLD.empid,'SAL',TO_CHAR(:OLD.sal),TO_CHAR(:NEW.sal)
```

```
,USER,SYSTIMESTAMP);
```

```
END IF;
```

```
IF NVL(:OLD.deptno,-1) <> NVL(:NEW.deptno,-1) THEN
```

```
INSERT INTO emp_audit VALUES
```

```
(emp_audit_seq.NEXTVAL,:OLD.empid,'DEPTNO',TO_CHAR(:OLD.deptno),TO_CHAR(:NEW
```

```
W.deptno),USER,SYSTIMESTAMP);
```

```
END IF;
```

```
END; /
```

Program 5

Write a code in PL/SQL to implement a trigger that records user activity (inserts, updates, deletes) in an audit log for a given set of tables.

```
-- PRODUCTS(PID PK,...); DML_AUDIT(AUD_ID, TAB_NAME, OP, PK_VAL,  
WHO,
```

```
WHEN_)
```

```
CREATE OR REPLACE TRIGGER trg_products_dml_audit
```

```
AFTER INSERT OR UPDATE OR DELETE ON products FOR EACH ROW
```

```
BEGIN
```

```
IF INSERTING THEN
```

```
INSERT INTO dml_audit VALUES
```

```
(dml_audit_seq.NEXTVAL,'PRODUCTS','INSERT',TO_CHAR(:NEW.pid),USER,SY  
STIMEST
```

```
AMP);
```

```
ELSIF UPDATING THEN
```

```
INSERT INTO dml_audit VALUES
```

```
(dml_audit_seq.NEXTVAL,'PRODUCTS','UPDATE',TO_CHAR(:OLD.pid),USER,SY  
YSTIMEST
```

```
AMP);
```

```
ELSE
```

```
INSERT INTO dml_audit VALUES
```

```
(dml_audit_seq.NEXTVAL,'PRODUCTS','DELETE',TO_CHAR(:OLD.pid),USER,SY  
STIMEST AMP);
```

```
END IF;
```

```
END; /
```

Program 7

Write a code in PL/SQL to implement a trigger that automatically calculates and updates a running total column for a table whenever new rows are inserted.

```
-- ORDERS(ORDER_ID PK, AMOUNT, RUN_TOTAL)

CREATE OR REPLACE TRIGGER trg_orders_run_total

FOR INSERT ON orders COMPOUND TRIGGER  g_last NUMBER; BEFORE
STATEMENT IS

BEGIN SELECT NVL(MAX(run_total),0) INTO g_last FROM orders; END;

BEFORE EACH ROW IS

BEGIN :NEW.run_total := g_last + :NEW.amount; g_last := :NEW.run_total;

END;

END; /
```

Program 8

Write a code in PL/SQL to create a trigger that validates the availability of items before allowing an order to be placed, considering stock levels and pending orders.

```
-- ITEMS(ITEM_ID PK, STOCK_QTY); ORDER_ITEMS(OI_ID PK, ITEM_ID FK, QTY,
STATUS)
CREATE OR REPLACE TRIGGER trg_order_items_stock_check
BEFORE INSERT OR UPDATE OF item_id, qty ON order_items FOR EACH ROW
DECLARE v_stock NUMBER; v_reserved NUMBER; v_avail NUMBER;
BEGIN
    SELECT stock_qty INTO v_stock FROM items WHERE item_id = :NEW.item_id;
    SELECT NVL(SUM(qty),0) INTO v_reserved
    FROM order_items
    WHERE item_id = :NEW.item_id AND status='PENDING' AND NVL(oi_id,-1) <>
NVL(:NEW.oi_id,-1);
    v_avail := v_stock - v_reserved; IF :NEW.qty > v_avail THEN
        RAISE_APPLICATION_ERROR(-20008,'Insufficient stock.
Available='||v_avail);
    END IF;
END;
/
```


Evaluation Procedure	Marks awarded
PL/SQL Procedure(5)	
Program/Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	

MONGO DB

MONGO DB

MongoDB is a free and open-source cross-platform document-oriented database. Classified as a NoSQL database, MongoDB avoids the traditional table-based relational database structure in favor of JSON-like documents with dynamic schemas, making the integration of data in certain types of applications easier and faster.

Create Database using mongosh

After connecting to your database using mongosh, you can see which database you are using by typing db in your terminal.

If you have used the connection string provided from the MongoDB Atlas dashboard, you should be connected to the myFirstDatabase database.

Show all databases

To see all available databases, in your terminal type show dbs.

Notice that myFirstDatabase is not listed. This is because the database is empty. An empty database is essentially non-existent.

Change or Create a Database

You can change or create a new database by typing use then the name of the database.

Create Collection using mongosh

You can create a collection using the createCollection() database method.

Insert Documents

insertOne()


```
db.posts.insertOne({  
  
  title: "Post Title 1",  
  
  body: "Body of post.",  
  
  category: "News",  
  
  likes: 1,  
  
  tags: ["news", "events"],  
  
  date: Date()  
  
})
```

EXERCISE 18

Structure of 'restaurants' collection:

```
{  
  "address": {  
    "building": "1007",  
    "coord": [ -73.856077, 40.848447 ],  
    "street": "Morris Park Ave",  
    "zipcode": "10462"  
  },  
  "borough": "Bronx",  
  "cuisine": "Bakery",  
  "grades": [  
    { "date": { "$date": 1393804800000 }, "grade": "A", "score": 2 },  
    { "date": { "$date": 1378857600000 }, "grade": "A", "score": 6 },  
    { "date": { "$date": 1358985600000 }, "grade": "A", "score": 10 },  
    { "date": { "$date": 1322006400000 }, "grade": "A", "score": 9 },  
    { "date": { "$date": 1299715200000 }, "grade": "B", "score": 14 }  
  ],  
  "name": "Morris Park Bake Shop",  
}
```

```
"restaurant_id": "30075445"
}
```

1. Write a MongoDB query to find the restaurant Id, name, borough and cuisine for those restaurants which prepared dish except 'American' and 'Chinees' or restaurant's name begins with letter 'Wil'.

```
db.restaurants.find(
```

```
{ $or: [ { cuisine: { $nin: ["American", "Chinees"] } }, { name: /^Wil/i } ] },
```

```
{ restaurant_id:1, name:1, borough:1, cuisine:1, _id:0 } );
```

2. Write a MongoDB query to find the restaurant Id, name, and grades for those restaurants which achieved a grade of "A" and scored 11 on an ISODate "2014-08-11T00:00:00Z" among many of survey dates..

```
{ grades: { $elemMatch: { grade:"A", score:11, date: ISODate("2014-08- 11T00:00:00Z") }
}}, { restaurant_id:1, name:1, grades:1, _id:0 } );
```

3. Write a MongoDB query to find the restaurant Id, name and grades for those restaurants where the 2nd element of grades array contains a grade of "A" and score 9 on an ISODate "2014-08-11T00:00:00Z".

```
db.restaurants.find( { "grades.1.grade":"A", "grades.1.score":9, "grades.1.date":
ISODate("2014-08-11T00:00:00Z") }, restaurant_id:1, name:1, grades:1, _id:0 } );
```

4. Write a MongoDB query to find the restaurant Id, name, address and geographical location for those restaurants where 2nd element of coord array contains a value

which is more than 42 and upto 52..

```
db.restaurants.find( { "address.coord.1": { $gt:42, $lte:52 } }, { restaurant_id:1, name:1, address:1, "address.coord":1, _id:0 } );
```

5. Write a MongoDB query to arrange the name of the restaurants in ascending order along with all the columns.

```
db.restaurants.find({}).sort({ name: 1 });
```

6. Write a MongoDB query to arrange the name of the restaurants in descending along with all the columns.

```
db.restaurants.find({}).sort({ name: -1 });
```

7. Write a MongoDB query to arranged the name of the cuisine in ascending order and for that same cuisine borough should be in descending order.

```
db.restaurants.find({}, { name:1, cuisine:1, borough:1 }).sort({ cuisine:1, borough:-1 });
```

8. Write a MongoDB query to know whether all the addresses contains the street or not.

```
db.restaurants.find({ "address.street": { $exists: false } }, { name:1, address:1, _id:0 });
```

9. Write a MongoDB query which will select all documents in the restaurants collection where the coord field value is Double.

```
db.restaurants.find( { "address.coord.0": { $type: "double" }, "address.coord.1": { $type: "double" } } );
```

10. Write a MongoDB query which will select the restaurant Id, name and grades for those restaurants which returns 0 as a remainder after dividing the score by 7.

```
db.restaurants.find( { "grades.score": { $mod: [7, 0] } }, { restaurant_id:1, name:1, grades:1, _id:0 } );
```

11. Write a MongoDB query to find the restaurant name, borough, longitude and attitude and cuisine for those restaurants which contains 'mon' as three letters somewhere in its name.

```
db.restaurants.find( { name: /mon/i }, { name:1, borough:1, cuisine:1, "address.coord.0":1, "address.coord.1":1, _id:0 } );
```

12. Write a MongoDB query to find the restaurant name, borough, longitude and latitude and cuisine for those restaurants which contain 'Mad' as first three letters of its name.

```
db.restaurants.find( { name: /^Mad/i }, { name:1, borough:1, cuisine:1, "address.coord.0":1, "address.coord.1":1, _id:0 } );
```

13. Write a MongoDB query to find the restaurants that have at least one grade with a score of less than 5.

```
db.restaurants.find({ "grades.score": { $lt: 5 } });
```

14. Write a MongoDB query to find the restaurants that have at least one grade with a score of less than 5 and that are located in the borough of Manhattan.

```
db.restaurants.find({ "grades.score": { $lt:5 }, borough: "Manhattan" });
```

// 15) Score < 5 AND borough in Manhattan or Brooklyn

```
db.restaurants.find({"grades.score": { $lt:5 }, borough: {  
$in:["Manhattan","Brooklyn"] } }));
```

15. Write a MongoDB query to find the restaurants that have at least one grade with a score of less than 5 and that are located in the borough of Manhattan or Brooklyn.

```
db.restaurants.find({ "grades.score": { $lt:5 }, borough: {  
$in:["Manhattan","Brooklyn"] }, cuisine: { $ne: "American" } }));
```

16. Write a MongoDB query to find the restaurants that have at least one grade with a score of less than 5 and that are located in the borough of Manhattan or Brooklyn, and their cuisine is not American

```
db.restaurants.find({ "grades.score": { $lt:5 }, borough: {  
$in:["Manhattan","Brooklyn"] }, cuisine: { $ne: "American" } }));
```

17. Write a MongoDB query to find the restaurants that have at least one grade with a score of less than 5 and that are located in the borough of Manhattan or Brooklyn, and their cuisine is not American or Chinese.

```
db.restaurants.find({ "grades.score": { $lt:5 }, borough: {
$in:["Manhattan","Brooklyn"] }, cuisine: { $nin: ["American","Chinese"] }
});
```

18. Write a MongoDB query to find the restaurants that have a grade with a score of 2 and a grade with a score of 6.

```
db.restaurants.find({ grades: { $all: [ { $elemMatch: { score: 2 } }, { $elemMatch: {
score: 6 } } ] } }));
```

19. Write a MongoDB query to find the restaurants that have a grade with a score of 2 and a grade with a score of 6 and are located in the borough of Manhattan.

```
db.restaurants.find({ grades: { $all: [ { $elemMatch: { score: 2 } }, { $elemMatch: {
score: 6 } } ] } }));
```

21. Write a MongoDB query to find the restaurants that have a grade with a score of 2 and a grade with a score of 6 and are located in the borough of Manhattan or Brooklyn.

```
db.restaurants.find({ borough: { $in:["Manhattan","Brooklyn"] }, grades: { $all: [ {
$elemMatch: { score: 2 } }, { $elemMatch: { score: 6 } } ] } }));
```

22. Write a MongoDB query to find the restaurants that have a grade with a score of 2 and a grade with a score of 6 and are located in the borough of Manhattan or Brooklyn, and their cuisine is not American.

```
db.restaurants.find({ borough: { $in:["Manhattan","Brooklyn"] }, cuisine: { $ne:"American" }, grades: { $all: [ { $elemMatch: { score: 2 } }, { $elemMatch: { score: 6 } } ] } }));
```

22. Write a MongoDB query to find the restaurants that have a grade with a score of 2 and a grade with a score of 6 and are located in the borough of Manhattan or Brooklyn, and their cuisine is not American or Chinese.

23. Write a MongoDB query to find the restaurants that have a grade with a score of 2 or a grade with a score of 6.

```
db.restaurants.find({ borough: { $in:["Manhattan","Brooklyn"] }, cuisine: { $nin: ["American","Chinese"] }, grades: { $all: [ { $elemMatch: { score: 2 } }, { $elemMatch: { score: 6 } } ] } }));
```

Sample document of 'movies' collection

```
{
  _id: ObjectId("573a1390f29313caabcd42e8"),
  plot: 'A group of bandits stage a brazen train hold-up, only to find a determined posse hot on their heels.'
```

genres: ['Short', 'Western'],

runtime: 11,

cast: [

'A.C. Abadie',

"Gilbert M. 'Broncho Billy' Anderson",

'George Barnes',

'Justus D. Barnes'

],

poster: 'https://m.media-amazon.com/images/M/MV5BMTU3NjE5NzYtYTtyNS00MDVmLWlwYjgtMmYwYWlxdDYyNzU2XkEyXkFqcGdeQXVyNzQzNzQxNzI@._V1_SY1000_SX677_AL_.jpg',

title: 'The Great Train Robbery',

fullplot: "Among the earliest existing films in American cinema - notable as the first film that presented a narrative story to tell - it depicts a group of cowboy outlaws who hold up a train and rob the passengers. They are then pursued by a Sheriff's posse. Several scenes have color included - all hand tinted.",

languages: ['English'],

released: ISODate("1903-12-01T00:00:00.000Z"),

directors: ['Edwin S. Porter'],

rated: 'TV-G',

awards: { wins: 1, nominations: 0, text: '1 win.' },

lastupdated: '2015-08-13 00:27:59.177000000',

year: 1903,

imdb: { rating: 7.4, votes: 9847, id: 439 },

countries: ['USA'],

type: 'movie',

tomatoes: {


```
viewer: { rating: 3.7, numReviews: 2559, meter: 75 },  
fresh: 6,  
critic: { rating: 7.6, numReviews: 6, meter: 100 },  
rotten: 0,  
lastUpdated: ISODate("2015-08-08T19:16:10.000Z")  
}
```

1. Find all movies with full information from the 'movies' collection that released in the year 1893.

```
db.movies.find({ year: 1893 });
```

2. Find all movies with full information from the 'movies' collection that have a runtime greater than 120 minutes.

```
db.movies.find({ runtime: { $gt: 120 } });
```

3. Find all movies with full information from the 'movies' collection that have "Short" genre.

```
db.movies.find({ genres: "Short" });
```

4. Retrieve all movies from the 'movies' collection that were directed by "William K.L. Dickson" and include complete information for each movie.

```
db.movies.find({ directors: "William K.L. Dickson" });
```

5. Retrieve all movies from the 'movies' collection that were released in the USA and include complete information for each movie.

```
db.movies.find({ countries: "USA" });
```

6. Retrieve all movies from the 'movies' collection that have complete information and are rated as "UNRATED".

```
db.movies.find({ rated: "UNRATED" });
```

7. Retrieve all movies from the 'movies' collection that have complete information and have received more than 1000 votes on IMDb.

```
db.movies.find({ "imdb.votes": { $gt: 1000 } });
```

8. Retrieve all movies from the 'movies' collection that have complete information and have an IMDb rating higher than 7.

```
db.movies.find({ "imdb.rating": { $gt: 7 } });
```

9. Retrieve all movies from the 'movies' collection that have complete information and have a viewer rating higher than 4 on Tomatoes.

```
db.movies.find({ "tomatoes.viewer.rating": { $gt: 4 } });
```

10. Retrieve all movies from the 'movies' collection that have received an award.

```
db.movies.find({ $or: [ { "awards.wins": { $gt: 0 } }, { "awards.nominations": { $gt: 0 } } ] });
```

11. Find all movies with title, languages, released, directors, writers, awards, year, genres, runtime, cast, countries from the 'movies' collection in MongoDB that have at least one nomination.

```
db.movies.find( { "awards.nominations": { $gt: 0 } }, { _id: 0, title: 1, languages: 1, released: 1, directors: 1, writers: 1, awards: 1, year: 1, genres: 1, runtime: cast: 1, countries: 1 } );
```

12. Find all movies with title, languages, released, directors, writers, awards, year, genres, runtime, cast, countries from the 'movies' collection in MongoDB with cast including "Charles Kayser".

```
db.movies.find( { cast: "Charles Kayser" }, { _id: 0, title: 1, languages: 1, released: 1, directors: 1, writers: 1, awards: 1, year: 1, genres: 1, runtime: 1, cast: 1, countries: 1 } );
```

13. Retrieve all movies with title, languages, released, directors, writers, countries from the 'movies' collection in MongoDB that released on May 9, 1893.

```
db.movies.find( { released: ISODate("1893-05-09T00:00:00Z") }, { _id: 0, title: 1, languages: 1, released: 1, directors: 1, writers: 1, countries: 1 } );
```

14. Retrieve all movies with title, languages, released, directors, writers, countries from the 'movies' collection in MongoDB that have a word "scene" in the title.

```
db.movies.find( { title: { $regex: /\bscene\b/i } }, { _id: 0, title: 1, languages: 1, released: 1, directors: 1, writers: 1, countries: 1 } );
```

Evaluation Procedure	Marks awarded
PL/SQL Procedure(5)	
Program/Execution (5)	
Viva(5)	
Total (15)	
Faculty Signature	